Coding Interview Preparation Guide

Pattern-based problem set with Java 8+ solutions

Target level: Mid-level software engineering interviews at companies such as Booking.com, Airbnb, Microsoft, Agoda, UiPath, and organizations of similar range.

How to use this guide

This guide is organized **by pattern**, not by random problem lists. Interviewers at these companies rarely ask you to recall an exotic algorithm. Instead they test whether you recognize which of a dozen well-known *patterns* a problem belongs to, and whether you can implement it cleanly under time pressure.

For every problem you get:

1. **The idea in plain language** — what the problem is really asking.
2. **A worked example** — concrete input and output.
3. **A diagram** — a picture of the intuition (ASCII, prints fine in a PDF).
4. **Brute force in Java** — the obvious solution, so you understand the baseline.
5. **Optimized in Java** — the solution an interviewer wants, with the trick explained.
6. **Complexity table** — time and space for both versions.

All code targets **Java 8 or later**. It compiles as-is and favors clarity over cleverness.

A note on difficulty

Everything here sits in the **Easy-to-Medium** band, which is exactly what mid-level loops at these companies focus on. You will occasionally see a "Medium-Hard" problem (LRU Cache, Merge K Sorted Lists, Meeting Rooms II) because they show up often enough to be worth knowing cold.

The 12 patterns

#	Pattern	When you reach for it
1	Two Pointers	Sorted array, pair/triplet, in-place partition
2	Sliding Window	Longest/shortest contiguous subarray or substring
3	Hashing / Frequency	"Have I seen this before?", counting, grouping
4	Binary Search	Sorted data, or a monotonic answer space
5	Linked List	Pointer manipulation, cycle detection, LRU
6	Trees (BFS / DFS)	Hierarchy, recursion, level-by-level work
7	Graphs	Grids, dependencies, connectivity, ordering
8	Dynamic Programming	Overlapping subproblems, "count the ways / best value"

#	Pattern	When you reach for it
9	Backtracking	Generate all subsets / permutations / combinations
10	Heaps / Priority Queue	"Top K", "Kth largest", streaming merges
11	Intervals	Overlapping ranges, scheduling, meeting rooms
12	Stacks	Matching pairs, "next greater", monotonic structure

How to practice

- **Recognize before you code.** Spend the first 60 seconds asking: *which pattern is this?*
- **Say the brute force out loud first.** It buys goodwill and often reveals the optimization.
- **State complexity before you're asked.** It signals seniority.
- **Test on the worked example by hand** before declaring victory.

---

1. Two Pointers

The core idea. Instead of using nested loops that re-scan the same elements, you keep two indices ("pointers") that walk through the data in a coordinated way. They might start at opposite ends and move toward each other, or move in the same direction at different speeds. Each pointer moves only forward, so the whole scan is linear instead of quadratic.

Reach for it when: the array is **sorted** (or can be), you're looking for a **pair or triplet** that satisfies a condition, or you need to **partition / dedupe in place**.

1.1 Two Sum II — Input Array Is Sorted

Problem. Given an integer array `numbers` that is sorted in **non-decreasing** order and an integer `target`, return the **1-based** indices `[i, j]` of the two numbers whose sum equals `target`.

- **Constraints:** `2 <= numbers.length <= 3 * 10^4`; numbers may include negative values and duplicates; the array is already sorted ascending.
- **Clarifying assumptions:** exactly one valid pair exists (so you never need to handle "no solution"); each input element is used at most once (you can't pair an index with itself); return indices, not the values themselves.
- **Why it's tricky:** the naive fix is to binary-search for the complement of each element (O(n log n)), but the sortedness lets two converging pointers do it in one O(n) pass — recognizing *why* moving the correct pointer is always safe (it never skips the answer) is the actual insight being tested.

Example.

```
Input:  numbers = [2, 7, 11, 15], target = 9
Output: [1, 2]          (2 + 7 = 9)
```

Intuition (diagram). Put one pointer at the smallest value, one at the largest. Their sum tells you which way to move: too big → shrink from the right; too small → grow from the left. Because the array is sorted, each move rules out an entire set of impossible pairs.

```
index:      0    1    2    3
array:      2    7   11   15
            L              R   sum = 2+15 = 17 > 9  -> move R left
            L         R        sum = 2+11 = 13 > 9  -> move R left
            L    R             sum = 2+7  = 9  == 9 -> found (L, R)
```

Brute force

Check every pair with two loops.

```
public int[] twoSumBrute(int[] numbers, int target) {
    for (int i = 0; i < numbers.length; i++) {
        for (int j = i + 1; j < numbers.length; j++) {
            if (numbers[i] + numbers[j] == target) {
                return new int[] { i + 1, j + 1 };
            }
        }
    }
    return new int[] { -1, -1 };
}
```

Optimized (two pointers)

Exploit the sorted order: converge from both ends.

```
public int[] twoSumSorted(int[] numbers, int target) {
    int lo = 0, hi = numbers.length - 1;
    while (lo < hi) {
        int sum = numbers[lo] + numbers[hi];
        if (sum == target) {
            return new int[] { lo + 1, hi + 1 };
        } else if (sum < target) {
            lo++;               // need a bigger sum
        } else {
            hi--;               // need a smaller sum
        }
    }
    return new int[] { -1, -1 };
}
```

Version	Time	Space
Brute force	$O(n^2)$	O(1)
Two pointers	O(n)	O(1)

1.2 Valid Palindrome

Problem. Given a string `s`, return `true` if it reads the same forwards and backwards after considering **only alphanumeric characters** and **ignoring case** (punctuation, spaces, and symbols are skipped entirely, not just lowercased).

- **Constraints:** `1 <= s.length <= 2 * 10^5`; `s` consists of printable ASCII characters (letters, digits, spaces, punctuation).
- **Clarifying assumptions:** an empty string, or a string with no alphanumeric characters at all, counts as a palindrome (vacuously true); only ASCII letters/digits count as alphanumeric, not full Unicode categories.
- **Why it's tricky:** it's easy to solve with O(n) extra space (build a cleaned string, reverse, compare); the trap is doing it **in place** with two pointers while correctly skipping non-alphanumeric characters on *both* sides without overshooting each other.

Example.

```
Input:  "A man, a plan, a canal: Panama"
Output: true            (ignoring punctuation/case -> "amanaplanacanalpanama")
```

Intuition (diagram). One pointer from each end. Skip anything that isn't a letter or digit. Compare the two characters; if they ever differ, it's not a palindrome.

```
r a c e c a r
L           R  r == r  ok, move inward
  L       R    a == a  ok, move inward
    L   R      c == c  ok, move inward
      *        lo meets hi at 'e' -> palindrome
```

Brute force

Build a cleaned string, reverse it, compare.

```
public boolean isPalindromeBrute(String s) {
    StringBuilder sb = new StringBuilder();
    for (char c : s.toCharArray()) {
        if (Character.isLetterOrDigit(c)) {
            sb.append(Character.toLowerCase(c));
        }
    }
    String forward = sb.toString();
    String backward = sb.reverse().toString();
    return forward.equals(backward);
}
```

Optimized (two pointers, no extra string)

Compare in place from both ends.

```
public boolean isPalindrome(String s) {
    int lo = 0, hi = s.length() - 1;
    while (lo < hi) {
        while (lo < hi && !Character.isLetterOrDigit(s.charAt(lo))) lo++;
        while (lo < hi && !Character.isLetterOrDigit(s.charAt(hi))) hi--;
        if (Character.toLowerCase(s.charAt(lo)) != Character.toLowerCase(s.charAt(hi))) {
            return false;
        }
        lo++;
        hi--;
    }
    return true;
}
```

Version	Time	Space
Brute force	O(n)	O(n)
Two pointers	O(n)	O(1)

1.3 3Sum

Problem. Given an integer array `nums`, return all **unique** triplets `[nums[i], nums[j], nums[k]]` (distinct indices `i`, `j`, `k`) such that `nums[i] + nums[j] + nums[k] == 0`. The order of triplets, and the order of elements within a triplet, does not matter, but the result must not contain duplicate triplets.

- **Constraints:** `3 <= nums.length <= 3000`; elements can be positive, negative, or zero, and duplicates are common (`-10^5 <= nums[i] <= 10^5`).
- **Clarifying assumptions:** if no triplet sums to zero, return an empty list; the same value may be reused across *different* triplets, just not within the same triplet from the same index twice.
- **Why it's tricky:** the obvious approach is $O(n^3)$ with a set to dedupe, which is slow and wastes memory on duplicate work. The real challenge is combining sorting + two pointers to get $O(n^2)$, *and* correctly skipping duplicate anchors and duplicate `lo` / `hi` values so the same triplet isn't emitted twice — an easy source of off-by-one bugs.

Example.

```
Input:  [-1, 0, 1, 2, -1, -4]
Output: [[-1, -1, 2], [-1, 0, 1]]
```

Intuition (diagram). Sort first. Fix one number, then the problem becomes "Two Sum (sorted)" on the rest — solved with the converging-pointers trick. Sorting also makes it easy to skip duplicates so triplets aren't repeated.

```
index:      0   1   2   3   4   5
sorted:    -4  -1  -1   0   1   2
            i   L               R  fix i=-4, need L+R = 4
                i   L           R  fix i=-1, need L+R = 1
                i       L   R      L+R = 0+1 = 1 -> found [-1,0,1]
```

Approach (step by step)

Key idea / invariant. Sort the array first. Then fix the smallest element of a candidate triplet as an anchor `i`, and solve "find two numbers in `nums[i+1..n-1]` that sum to `-nums[i]`" — exactly the Two Sum II problem from 1.1 — with converging pointers `lo` and `hi`. Because the suffix is sorted, `lo` only ever needs to move right and `hi` only ever needs to move left, so each anchor `i` costs $O(n)$ instead of $O(n^2)$.

Numbered walk-through: 1. Sort `nums` ascending. 2. For each index `i` from `0` to `n - 3`: a. If `nums[i] == nums[i - 1]`, skip it — using the same anchor value again can only reproduce triplets already found with the previous `i`. b. Set `lo = i + 1`, `hi = n - 1`. c. While `lo < hi`: - Compute `sum = nums[i] + nums[lo] + nums[hi]`. - If `sum == 0`: record the triplet, then advance `lo` past any duplicate values and retreat `hi` past any duplicate values (so the next distinct pair is tried), then move both inward once more. - If `sum < 0`: the total is too small, so move `lo++` to increase it. - If `sum > 0`: the total is too big, so move `hi--` to decrease it. 3. Return the collected triplets.

Worked trace on `[-4, -1, -1, 0, 1, 2]` (already sorted): - `i=0` (`-4`): `lo=1(-1), hi=5(2)` → sum `-3` < 0 → `lo++` → `lo=2(-1), hi=5(2)` → sum `-3` < 0 → `lo++` → `lo=3(0), hi=5(2)` → sum `-2` < 0 → `lo++` → `lo=4(1), hi=5(2)` → sum `-1` < 0 → `lo++` → `lo == hi`, stop. No triplet for anchor `-4`. - `i=1` (`-1`): `lo=2(-1), hi=5(2)` → sum `0` → record `[-1, -1, 2]`; advance past dups → `lo=3(0),`

`hi=4(1)` → sum `0` → record `[-1, 0, 1]`; `lo++, hi--` → `lo == hi`, stop. - `i=2` (`-1`): equals `nums[i-1]`, skip. - Result: `[[-1, -1, 2], [-1, 0, 1]]`, matching the expected output.

Brute force

Three nested loops, dedupe with a set.

```
public List<List<Integer>> threeSumBrute(int[] nums) {
    Set<List<Integer>> seen = new HashSet<>();
    int n = nums.length;
    Arrays.sort(nums);                    // sort so each triplet has a canonical order
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            for (int k = j + 1; k < n; k++) {
                if (nums[i] + nums[j] + nums[k] == 0) {
                    seen.add(Arrays.asList(nums[i], nums[j], nums[k]));
                }
            }
        }
    }
    return new ArrayList<>(seen);
}
```

Optimized (sort + two pointers)

For each `i`, converge two pointers over the remaining suffix.

```
public List<List<Integer>> threeSum(int[] nums) {
    Arrays.sort(nums);
    List<List<Integer>> result = new ArrayList<>();
    for (int i = 0; i < nums.length - 2; i++) {
        if (i > 0 && nums[i] == nums[i - 1]) continue;      // skip duplicate anchors
        int lo = i + 1, hi = nums.length - 1;
        while (lo < hi) {
            int sum = nums[i] + nums[lo] + nums[hi];
            if (sum == 0) {
                result.add(Arrays.asList(nums[i], nums[lo], nums[hi]));
                while (lo < hi && nums[lo] == nums[lo + 1]) lo++;   // skip dup
                while (lo < hi && nums[hi] == nums[hi - 1]) hi--;   // skip dup
                lo++;
                hi--;
            } else if (sum < 0) {
                lo++;
            } else {
                hi--;
            }
        }
    }
    return result;
}
```

Version	Time	Space
Brute force	$O(n^3)$	O(n) for the result set
Sort + two pointers	$O(n^2)$	O(1) extra (ignoring output)

1.4 Container With Most Water

Problem. Given an integer array `height` where `height[i]` is the height of a vertical line drawn at `x = i`, choose two lines `i < j` that, together with the x-axis, form a container. Return the **maximum** amount of water that container can hold, where the area is `min(height[i], height[j]) * (j - i)`.

- **Constraints:** `2 <= height.length <= 10^5`; `0 <= height[i] <= 10^4`; heights can be zero (a line of height 0 holds no water on that side) but not negative.
- **Clarifying assumptions:** the container has no "floor" thickness — only the two chosen lines and the x-axis distance between them matter; you cannot tilt or slant the container.
- **Why it's tricky:** the brute force checks all O(n²) pairs. The key insight for the linear solution is a **greedy elimination argument**: for the current widest window, the *shorter* wall is strictly limiting the area, so keeping it and shrinking the window can never beat moving it inward — proving that it is always safe to discard the shorter wall (not the taller one) is the crux of the problem.

Example.

```
Input:  [1, 8, 6, 2, 5, 4, 8, 3, 7]
Output: 49          (lines at index 1 and 8: min(8,7) * (8-1) = 7 * 7 = 49)
```

Intuition (diagram). Area = width × the *shorter* of the two walls. Start as wide as possible. The short wall is the bottleneck, so moving the *taller* wall inward can only lose width without helping — instead move the **shorter** wall inward, hoping for a taller one.

```
index:      0   1   2   3   4   5   6   7   8
height:     1   8   6   2   5   4   8   3   7
            L                               R  area = min(1,7)*8 = 8   -> move L (shorter)
                L                           R  area = min(8,7)*7 = 49  <- best so far
```

Approach (step by step)

Key idea / invariant. Start with the widest possible container (`lo = 0`, `hi = n-1`) and shrink the window one step at a time. At every step, the area is capped by the **shorter** of the two current walls. If you move the taller wall inward, the width shrinks but the cap (the shorter wall) stays the same or gets worse — so that move can never produce a larger area than what you already have. Moving the **shorter** wall is the only move that has a *chance* of finding a taller wall and a bigger area. This lets you scan the array once instead of checking every pair.

Numbered walk-through: 1. Set `lo = 0`, `hi = height.length - 1`, `best = 0`. 2. While `lo < hi`: a. Compute `area = min(height[lo], height[hi]) * (hi - lo)`. b. Update `best = max(best, area)`. c. If `height[lo] < height[hi]`, move `lo++` (the left wall is the bottleneck). Otherwise move `hi--` (the right wall is the bottleneck, or they're tied). 3. Return `best` once `lo` and `hi` meet.

Worked trace on `[1, 8, 6, 2, 5, 4, 8, 3, 7]`: - `lo=0(1), hi=8(7)` → area `min(1,7)*8 = 8` → `best=8` → `height[lo] < height[hi]` → `lo++`. - `lo=1(8), hi=8(7)` → area `min(8,7)*7 = 49` → `best=49` → `height[lo] >= height[hi]` → `hi--`. - The window keeps shrinking, but no later pair beats an area of 49, so the answer is `49`.

Brute force

Try every pair of walls.

```
public int maxAreaBrute(int[] height) {
    int best = 0;
    for (int i = 0; i < height.length; i++) {
        for (int j = i + 1; j < height.length; j++) {
            int area = Math.min(height[i], height[j]) * (j - i);
            best = Math.max(best, area);
        }
    }
    return best;
}
```

Optimized (two pointers)

Start wide, always move the shorter wall.

```
public int maxArea(int[] height) {
    int lo = 0, hi = height.length - 1, best = 0;
    while (lo < hi) {
        int area = Math.min(height[lo], height[hi]) * (hi - lo);
        best = Math.max(best, area);
        if (height[lo] < height[hi]) {
            lo++;               // move the limiting (shorter) wall
        } else {
            hi--;
        }
    }
    return best;
}
```

Version	Time	Space
Brute force	$O(n^2)$	O(1)
Two pointers	O(n)	O(1)

---

2. Sliding Window

The core idea. Instead of re-scanning a range of elements from scratch every time you move forward, you keep a "window" — a contiguous slice of the array or string defined by two edges (`start` and `end`). As you slide the window forward, you only add the new element entering the window and remove the old element leaving it, reusing all the work you already did for the rest. This turns many $O(n^2)$ "check every subarray/substring" problems into $O(n)$.

Reach for it when: you need to find a **contiguous subarray or substring** that satisfies some condition (max/min sum, longest/shortest length, no repeats), and the window can **grow or shrink** based on what's currently inside it.

2.1 Best Time to Buy and Sell Stock

Problem. Given an array `prices` where `prices[i]` is the stock price on day `i`, find the maximum profit from buying one share on one day and selling it on a later day. You may buy only once and sell only once (a single transaction), and the sell must happen strictly after the buy. If no profit is possible, return `0`.

- **Input:** `prices`, an array of non-negative integers, `1 <= prices.length <= 10^5`, `0 <= prices[i] <= 10^4`.
- **Output:** a single integer, the maximum achievable profit (or `0`).
- **Clarifying assumptions:** exactly one buy and one sell (no multiple transactions); it's fine for the answer to be `0` if prices only fall; the array is not necessarily sorted.
- **Why it's tricky:** the naive instinct is to compare every pair of days ($O(n^2)$); the insight is that you only ever need the *lowest price seen so far*, not every past price.

Example.

```
Input:  prices = [7, 1, 5, 3, 6, 4]
Output: 5              (buy at 1, sell at 6 -> profit 5)
```

Intuition (diagram). Think of it as a window that always starts at the lowest price seen so far and ends at today. You don't need to remember every past price — only the **minimum so far**. As you walk forward one day at a time, ask "if I sold today, having bought at the cheapest point so far, what's my profit?" and keep the best answer.

```
day:     0  1  2  3  4  5
price:   7  1  5  3  6  4
         ^                 day0: minSoFar=7          profit=0
            ^              day1: minSoFar=1 (new low) profit=0
               ^           day2: minSoFar=1           profit=5-1=4
                     ^     day4: minSoFar=1           profit=6-1=5 <- best
```

Approach (step by step)

- **Key idea / invariant:** at every day `i`, `minPriceSoFar` holds the lowest price seen in `prices[0..i]`. Because a valid sale must happen *after* the buy, scanning left-to-right and always comparing "today's price minus the cheapest price so far" automatically considers every valid (buy, sell) pair without ever looking backward.
- **Walk-through:**
- Initialize `minPriceSoFar = +infinity` and `bestProfit = 0`.
- For each `price` in order: update `minPriceSoFar = min(minPriceSoFar, price)` — this represents "the best day to have bought, up to and including today."
- Compute the profit if you sold today: `price − minPriceSoFar`, and keep the running max in `bestProfit`.
- After the loop, `bestProfit` is the answer.
- **Worked trace** (`prices = [7, 1, 5, 3, 6, 4]`): `minSoFar` goes `7 -> 1 -> 1 -> 1 -> 1 -> 1`; profit-if-sold-today goes `0, 0, 4, 2, 5, 3`; the running best profit becomes `0, 0, 4, 4, 5, 5`, so the answer is `5`.

Brute force

Try every buy day and every later sell day.

```
public int maxProfitBrute(int[] prices) {
    int best = 0;
    for (int buy = 0; buy < prices.length; buy++) {
        for (int sell = buy + 1; sell < prices.length; sell++) {
            best = Math.max(best, prices[sell] - prices[buy]);
        }
    }
    return best;
}
```

Optimized (one-pass window)

Slide forward, tracking the lowest price seen so far.

```
public int maxProfit(int[] prices) {
    int minPriceSoFar = Integer.MAX_VALUE;
    int bestProfit = 0;
    for (int price : prices) {
        minPriceSoFar = Math.min(minPriceSoFar, price);   // cheapest "buy" so far
        bestProfit = Math.max(bestProfit, price - minPriceSoFar); // sell today
    }
    return bestProfit;
}
```

Version	Time	Space
Brute force	$O(n^2)$	O(1)
One-pass window	O(n)	O(1)

2.2 Longest Substring Without Repeating Characters

Problem. Given a string `s`, find the length of the longest substring that has no repeating characters, and return that length.

- **Input:** `s`, a string of `0 <= s.length <= 5 * 10^4`, made of ASCII characters (letters, digits, and/or symbols).
- **Output:** a single integer, the length of the longest repeat-free substring.
- **Clarifying assumptions:** an empty string returns `0`; the substring must be contiguous (not a subsequence); character comparison is case-sensitive.
- **Why it's tricky:** candidates often try to recheck the whole window for duplicates on every step; the key insight is a two-pointer window plus a "seen" set/map lets you grow and shrink in amortized O(1) per character instead of re-scanning.

Example.

```
Input:  s = "abcabcbb"
Output: 3              ("abc" is the longest substring with no repeats)
```

Intuition (diagram). Grow the window by moving `end` forward one character at a time. If the new character is already inside the window, shrink from the left (`start`) until the duplicate is gone. A hash set (or map of char -> last index) tells you what's currently in the window so you can detect duplicates in O(1).

```
s:      a b c a b c b b
idx:    0 1 2 3 4 5 6 7
        S   E           window [0,2]="abc"  len=3  <- best so far
          S   E         'a' repeats -> shrink: S moves 0->1
            S   E       'b' repeats -> shrink: S moves 1->2
```

Approach (step by step)

- **Key idea / invariant:** `start` and `end` mark a window `[start, end]` that, at the top of every iteration, contains **no duplicate characters** — that's the invariant the loop maintains. `end` only ever moves forward because we're scanning the string once; `start` only ever moves forward because once a character has been excluded from the window it can never help re-include an earlier position (shrinking never "undoes" progress).
- **Walk-through:**
- Initialize an empty set `window`, `start = 0`, `best = 0`.
- For each `end` from `0` to `s.length() - 1`: let `c = s.charAt(end)`.
- While `c` is already in `window`: remove `s.charAt(start)` from the set and increment `start` — this is the **shrink condition**, and it runs until the specific duplicate of `c` has left the window (not just once).
- Add `c` to `window` (now the window `[start, end]` is duplicate-free again).
- Update `best = max(best, end - start + 1)`.
- Return `best`.
- **Worked trace** (`s = "abcabcbb"`): window grows to `"abc"` (len 3, best=3) at `end=2`; at `end=3` the incoming `'a'` duplicates the one at index 0, so `start` shrinks from `0` to `1`, giving window

`"bca"` (len 3); at `end=4` the incoming `'b'` duplicates index 1, so `start` shrinks to `2`; the pattern repeats and `best` never exceeds `3`.

Brute force

Check every substring for repeated characters.

```
public int lengthOfLongestSubstringBrute(String s) {
    int best = 0;
    for (int i = 0; i < s.length(); i++) {
        Set<Character> seen = new HashSet<>();
        for (int j = i; j < s.length(); j++) {
            if (!seen.add(s.charAt(j))) {
                break;                      // duplicate found, stop growing
            }
            best = Math.max(best, j - i + 1);
        }
    }
    return best;
}
```

Optimized (variable window + hash set)

Grow with `end`, shrink from `start` only when a duplicate appears.

```
public int lengthOfLongestSubstring(String s) {
    Set<Character> window = new HashSet<>();
    int start = 0;
    int best = 0;
    for (int end = 0; end < s.length(); end++) {
        char c = s.charAt(end);
        while (window.contains(c)) {
            window.remove(s.charAt(start));  // shrink until duplicate is removed
            start++;
        }
        window.add(c);
        best = Math.max(best, end - start + 1);
    }
    return best;
}
```

Version	Time	Space
Brute force	O(n²)	O(n)
Variable window + hash set	O(n)	O(min(n, alphabet size))

2.3 Maximum Average Subarray I

Problem. Given an integer array `nums` and an integer `k`, find the contiguous subarray of length **exactly** `k` that has the maximum average value, and return that average.

- **Input:** `nums`, `1 <= nums.length <= 10^5`, `-10^4 <= nums[i] <= 10^4`; `k`, an integer with `1 <= k <= nums.length`.

- **Output:** a `double`, the maximum average over all length-`k` windows.
- **Clarifying assumptions:** `k` is guaranteed to be at most `nums.length`, so at least one valid window always exists; `nums` may contain negatives and duplicates and is not sorted.
- **Why it's tricky:** the window size is fixed, so the trap is recomputing the sum of each window from scratch (O(n·k)); the insight is that sliding by one position only changes two elements (one leaves, one enters), so the sum can be updated in O(1).

Example.

```
Input:  nums = [1, 12, -5, -6, 50, 3], k = 4
Output: 12.75          (subarray [12, -5, -6, 50] -> sum 51, avg 51/4 = 12.75)
```

Intuition (diagram). Since the window size is fixed at `k`, you don't need to recompute the sum from scratch each time. Slide the window one step right: **add** the new element entering on the right, **subtract** the element that just left on the left. That keeps the running sum in O(1) per step.

```
idx:      0   1   2   3   4   5
nums:     1  12  -5  -6  50   3
          [           ]          window[0,3] sum=1+12-5-6=2      avg=0.50
              [           ]      window[1,4] sum=2-1+50=51<-best  avg=12.75
                  [           ]  window[2,5] sum=51-12+3=42      avg=10.50
```

Approach (step by step)

- **Key idea / invariant:** `windowSum` always equals the sum of exactly the last `k` elements processed — i.e. `nums[end-k+1..end]`. The window has a **fixed width**, so instead of two independently moving pointers, think of it as one pointer `end` sliding forward, with an implicit left edge at `end - k + 1`.
- **Walk-through:**
- Compute `windowSum` as the sum of the first `k` elements (`nums[0..k-1]`); this is the first window. Set `bestSum = windowSum`.
- For each `end` from `k` to `nums.length - 1`: slide the window one step right by adding the incoming element `nums[end]` and subtracting the outgoing element `nums[end - k]`.
- Update `bestSum = max(bestSum, windowSum)`.
- Return `bestSum / k` as a `double`.
- **Worked trace** (`nums = [1, 12, -5, -6, 50, 3], k = 4`): first window sum = `1+12-5-6 = 2` (avg 0.50, best so far). Slide to `end=4`: `sum = 2 - 1 + 50 = 51` (avg 12.75, new best). Slide to `end=5`: `sum = 51 - 12 + 3 = 42` (avg 10.5, not better). Final answer: `51 / 4 = 12.75`.

Brute force

Recompute the sum of each length-`k` window from scratch.

```
public double findMaxAverageBrute(int[] nums, int k) {
    double best = Double.NEGATIVE_INFINITY;
    for (int i = 0; i + k <= nums.length; i++) {
        int sum = 0;
        for (int j = i; j < i + k; j++) {
            sum += nums[j];             // recompute every time -- wasteful
        }
        best = Math.max(best, (double) sum / k);
    }
    return best;
}
```

Optimized (fixed-size window sum)

Maintain a running sum; add the incoming element, drop the outgoing one.

```
public double findMaxAverage(int[] nums, int k) {
    int windowSum = 0;
    for (int i = 0; i < k; i++) {
        windowSum += nums[i];           // sum of the first window
    }
    int bestSum = windowSum;
    for (int end = k; end < nums.length; end++) {
        windowSum += nums[end] - nums[end - k];  // slide: add new, remove old
        bestSum = Math.max(bestSum, windowSum);
    }
    return (double) bestSum / k;
}
```

Version	Time	Space
Brute force	O(n·k)	O(1)
Fixed-size window sum	O(n)	O(1)

2.4 Minimum Size Subarray Sum

Problem. Given an array of positive integers `nums` and an integer `target`, return the length of the **shortest** contiguous subarray whose sum is greater than or equal to `target`. If no such subarray exists, return `0`.

- **Input:** `nums`, `1 <= nums.length <= 10^5`, `1 <= nums[i] <= 10^4` (all values **positive**); `target`, `1 <= target <= 10^9`.
- **Output:** a single integer, the minimum length of a qualifying subarray (or `0`).
- **Clarifying assumptions:** all elements are strictly positive (this is what makes the shrink logic safe — adding elements only ever increases the sum); the subarray must be contiguous and non-empty.
- **Why it's tricky:** the naive approach checks every subarray ($O(n^2)$); the insight is the monotonic "grow to become valid, then shrink to become minimal" pattern, which only works because every element is positive (so the window sum is monotonic as you move either edge).

Example.

```
Input:  nums = [2, 3, 1, 2, 4, 3], target = 7
Output: 2              ([4, 3] sums to 7, and it's the shortest such subarray)
```

Intuition (diagram). Grow the window from the right, adding to a running sum. As soon as the sum meets `target`, the window is "good" — but maybe it's bigger than it needs to be, so shrink from the left as long as it stays good, recording the shortest length along the way. This is the classic "grow to become valid, then shrink to become minimal" pattern.

```
idx:     0  1  2  3  4  5
nums:    2  3  1  2  4  3
         S        E        sum=2+3+1+2=8 >=7  len=4
            S     E        shrink: remove idx0 -> sum=6 <7 stop  best=4
            S        E     end=4: sum=6+4=10>=7  len=4
                  S  E     shrink twice -> sum=6<7 stop  best=3
                  S     E  end=5: sum=6+3=9>=7  len=3
                     S  E  shrink: remove idx3 -> sum=7>=7 len=2 <- best
```

Approach (step by step)

- **Key idea / invariant:** `sum` always equals the sum of the current window `[start, end]`. Because every element is positive, growing the window (moving `end` right) only ever increases `sum`, and shrinking it (moving `start` right) only ever decreases `sum` — this monotonicity is exactly what makes it safe to greedily shrink from the left as far as possible whenever the window is valid, without missing a shorter answer. Both pointers only move forward because each index is considered "added" once (by `end`) and "removed" once (by `start`), never revisited.
- **Walk-through:**
- Initialize `start = 0`, `sum = 0`, `best = infinity`.
- For each `end` from `0` to `nums.length - 1`: add `nums[end]` to `sum` (grow).
- **Shrink condition:** while `sum >= target`, the window is valid, so record `best = min(best, end - start + 1)`, then subtract `nums[start]` from `sum` and increment `start` — try to shrink further, since we want the *minimal* valid window ending at `end`.
- After the loop, if `best` was never updated, return `0`; otherwise return `best`.
- **Worked trace** (`nums = [2, 3, 1, 2, 4, 3], target = 7`): by `end=3`, `sum = 2+3+1+2 = 8 = 7, so shrink once (remove nums[0]=2, sum=6 < 7, stop); best = 4. At end=4, sum = 6+4 = 10 >= 7, shrink twice (remove nums[1]=3 -> sum=7, still valid, best=3; remove nums[2]=1 -> sum=6 < 7, stop). At end=5, sum = 6+3 = 9 >= 7, shrink once (remove nums[3]=2 -> sum=7 >= 7, valid, best=2); shrinking again gives sum=3 < 7, so stop. Final answer: 2`.

Brute force

Check every subarray's sum.

```
public int minSubArrayLenBrute(int target, int[] nums) {
    int best = Integer.MAX_VALUE;
    for (int i = 0; i < nums.length; i++) {
        int sum = 0;
        for (int j = i; j < nums.length; j++) {
            sum += nums[j];
            if (sum >= target) {
                best = Math.min(best, j - i + 1);
                break;                    // longer j only makes it worse from here
            }
        }
    }
    return best == Integer.MAX_VALUE ? 0 : best;
}
```

Optimized (variable window, shrink while valid)

Grow with `end`, shrink from `start` while the window still meets `target`.

```
public int minSubArrayLen(int target, int[] nums) {
    int start = 0;
    int sum = 0;
    int best = Integer.MAX_VALUE;
    for (int end = 0; end < nums.length; end++) {
        sum += nums[end];                     // grow the window
        while (sum >= target) {               // shrink while still valid
            best = Math.min(best, end - start + 1);
            sum -= nums[start];
            start++;
        }
    }
    return best == Integer.MAX_VALUE ? 0 : best;
}
```

Version	Time	Space
Brute force	$O(n^2)$	O(1)
Variable window	O(n)	O(1)

---

3. Hashing / Frequency

The core idea. A hash map (or hash set) gives you near-instant lookup: "have I seen this value before, and what did I store with it?" That turns an O(n) linear scan for "does this exist?" into an O(1) check. Most of these problems boil down to the same move: walk the data once, and use a hash map/set to remember what you've already seen so you never need a second nested loop.

Reach for it when: you need to check **existence or counts** quickly, you're matching things up by some **derived key** (like a sorted string or a character count), or you keep writing an `O(n^2)` nested loop just to compare every pair of elements.

3.1 Two Sum

Problem. Given an array of integers `nums` and an integer `target`, return the indices of the two numbers whose values add up exactly to `target`.

- **Constraints:** `2 <= nums.length <= 10^4`; `-10^9 <= nums[i], target <= 10^9`; exactly one valid pair is guaranteed to exist; you may not use the same array element (index) twice, though the same *value* can legally appear at two different indices.
- **Clarifying assumptions:** confirm exactly one solution exists (so you never need to handle "no pair found" or pick among several valid pairs), and confirm the return value is the pair of *indices*, not the two numbers themselves.
- **Why it's tricky:** the obvious approach checks every pair in O(n²); the key insight is reframing "does a partner exist?" as a hash-map membership question instead of a search, which collapses it to a single O(n) pass.

Example.

```
Input:  nums = [2, 7, 11, 15], target = 9
Output: [0, 1]          (nums[0] + nums[1] = 2 + 7 = 9)
```

Intuition (diagram). For each number, the "partner" you need is `target - num`. Instead of searching the whole array for that partner, keep a map of every value you've already visited to its index. By the time you reach the second half of a pair, the first half is already sitting in the map, ready to be found in O(1).

```
nums:   2   7  11  15      target = 9
map:   {}

i=0  num=2   need=7   not in map -> store {2:0}
i=1  num=7   need=2   2 IS in map (index 0) -> answer [0, 1]
```

Brute force

Check every pair with two nested loops.

```
public int[] twoSumBrute(int[] nums, int target) {
    for (int i = 0; i < nums.length; i++) {
        for (int j = i + 1; j < nums.length; j++) {
            if (nums[i] + nums[j] == target) {
                return new int[] { i, j };
            }
        }
    }
    return new int[] { -1, -1 };
}
```

Optimized (hashmap, one pass)

Remember each value's index as you go; look up its complement immediately.

```
public int[] twoSum(int[] nums, int target) {
    Map<Integer, Integer> valueToIndex = new HashMap<>();   // value -> index seen so far
    for (int i = 0; i < nums.length; i++) {
        int need = target - nums[i];
        if (valueToIndex.containsKey(need)) {
            return new int[] { valueToIndex.get(need), i };
        }
        valueToIndex.put(nums[i], i);   // record this value AFTER checking, avoids self-pa
    }
    return new int[] { -1, -1 };
}
```

Version	Time	Space
Brute force	$O(n^2)$	O(1)
Hashmap, one pass	O(n)	O(n)

3.2 Group Anagrams

Problem. Given an array of strings `strs`, group the strings that are anagrams of one another (same letters, same counts, any order) into lists, and return the list of groups. The groups themselves, and the strings within each group, may be returned in any order.

- **Constraints:** `1 <= strs.length <= 10^4`; `0 <= strs[i].length <= 100`; each `strs[i]` consists of lowercase English letters only.
- **Clarifying assumptions:** confirm the output order doesn't matter (grader typically compares groups as sets of sets), and confirm empty strings are valid input and should form their own group (all empty strings are anagrams of each other).
- **Why it's tricky:** the challenge is picking a canonical "key" that is identical for every anagram of a word but different for non-anagrams — get that key wrong (e.g. use the raw string) and nothing groups correctly.

Example.

```
Input:  ["eat", "tea", "tan", "ate", "nat", "bat"]
Output: [["eat","tea","ate"], ["tan","nat"], ["bat"]]
```

Intuition (diagram). Anagrams are just rearrangements of the same letters, so if you build a "canonical key" that ignores order (either the sorted string, or a count of each letter), every anagram maps to the identical key. Use that key as a hashmap key and bucket the words into lists.

```
word "eat" -> sorted "aet" -\
word "tea" -> sorted "aet"  |--> map["aet"] = ["eat","tea","ate"]
word "ate" -> sorted "aet" -/

word "tan" -> sorted "ant" -\
word "nat" -> sorted "ant"  |--> map["ant"] = ["tan","nat"]
```

Approach (step by step)

Key idea / invariant: two strings are anagrams of each other if and only if sorting their characters produces the same string. That sorted string is a canonical key: every member of an anagram group maps to the exact same key, and no non-member ever collides with it. A hashmap from key to list-of-words then does the grouping for free.

1. Create an empty map from `String` key to `List<String>` group.
2. For each word in the input array: a. Copy its characters into an array and sort that array. b. Turn the sorted array back into a string — this is the canonical key. c. Look up (or create) the list for that key in the map, and append the original word to it.
3. After the loop, the map's values are exactly the anagram groups. Return them as a list.

Worked trace (`["eat","tea","tan","ate","nat","bat"]`):

```
word "eat"  sorted "aet"  map = {"aet": ["eat"]}
word "tea"  sorted "aet"  map = {"aet": ["eat","tea"]}
word "tan"  sorted "ant"  map = {"aet": [...], "ant": ["tan"]}
word "ate"  sorted "aet"  map = {"aet": ["eat","tea","ate"], "ant": ["tan"]}
word "nat"  sorted "ant"  map = {"aet": [...],               "ant": ["tan","nat"]}
word "bat"  sorted "abt"  map = {..., "abt": ["bat"]}
```

Final groups = the map's values: `[["eat","tea","ate"], ["tan","nat"], ["bat"]]` .

Brute force

Compare every word against every group already found.

```
public List<List<String>> groupAnagramsBrute(String[] strs) {
    List<List<String>> groups = new ArrayList<>();
    for (String word : strs) {
        char[] sortedWord = word.toCharArray();
        Arrays.sort(sortedWord);
        boolean placed = false;
        for (List<String> group : groups) {
            char[] sortedFirst = group.get(0).toCharArray();
            Arrays.sort(sortedFirst);
            if (Arrays.equals(sortedWord, sortedFirst)) {
                group.add(word);
                placed = true;
                break;
            }
        }
        if (!placed) {
            List<String> newGroup = new ArrayList<>();
            newGroup.add(word);
            groups.add(newGroup);
        }
    }
    return groups;
}
```

Optimized (hashmap keyed by sorted string)

Sort each word once to build its key, then bucket in O(1) lookups.

```
public List<List<String>> groupAnagrams(String[] strs) {
    Map<String, List<String>> keyToGroup = new HashMap<>();
    for (String word : strs) {
        char[] chars = word.toCharArray();
        Arrays.sort(chars);                       // canonical form: same letters -> same k
        String key = new String(chars);
        keyToGroup.computeIfAbsent(key, k -> new ArrayList<>()).add(word);
    }
    return new ArrayList<>(keyToGroup.values());
}
```

Version	Time	Space
Brute force	$O(n^2 \cdot k \log k)$	$O(n \cdot k)$
Hashmap, sorted key	$O(n \cdot k \log k)$	$O(n \cdot k)$

(n = number of words, k = max word length. Using a 26-count key instead of sorting drops the per-word cost to O(k), but sorting is simpler to write correctly under pressure.)

3.3 Valid Anagram

Problem. Given two strings `s` and `t`, return `true` if `t` is an anagram of `s`, meaning `t` uses exactly the same letters as `s` with exactly the same counts, just possibly in a different order; otherwise return `false`.

- **Constraints:** `1 <= s.length, t.length <= 5 * 10^4`; both strings consist of lowercase English letters only.
- **Clarifying assumptions:** confirm the alphabet is lowercase a-z only (so a fixed 26-slot array is safe), and confirm different lengths immediately imply `false` (a useful early exit).
- **Why it's tricky:** it's tempting to sort both strings (O(n log n)); the faster insight is that "same letters, same counts" is exactly what a fixed-size counting array checks in a single O(n) pass, with no sorting needed.

Example.

```
Input:  s = "anagram", t = "nagaram"
Output: true
```

Intuition (diagram). Two strings are anagrams exactly when they have identical letter counts. Use a fixed-size array of 26 counters (one per lowercase letter): add 1 for every letter in `s`, subtract 1 for every letter in `t`. If they're anagrams, every counter lands back on zero.

```
 s = "anagram"     t = "nagaram"

 counts['a'] += 3 (from s), then -= 3 (from t) -> 0
 counts['n'] += 1 (from s), then -= 1 (from t) -> 0
 counts['g'] += 1 (from s), then -= 1 (from t) -> 0
 ... all 26 counters end at 0 -> true
```

Brute force

Sort both strings and compare.

```
public boolean isAnagramBrute(String s, String t) {
    if (s.length() != t.length()) return false;
    char[] sChars = s.toCharArray();
    char[] tChars = t.toCharArray();
    Arrays.sort(sChars);
    Arrays.sort(tChars);
    return Arrays.equals(sChars, tChars);
}
```

Optimized (26-length count array)

One pass to add, one pass to subtract; no sorting needed.

```
public boolean isAnagram(String s, String t) {
    if (s.length() != t.length()) return false;
    int[] counts = new int[26];                 // one bucket per lowercase letter
    for (int i = 0; i < s.length(); i++) {
        counts[s.charAt(i) - 'a']++;
        counts[t.charAt(i) - 'a']--;
    }
    for (int count : counts) {
        if (count != 0) return false;            // mismatch: some letter count differs
    }
    return true;
}
```

Version	Time	Space
Brute force (sort)	O(n log n)	O(n)
Count array	O(n)	O(1) (fixed 26 slots)

3.4 Contains Duplicate

Problem. Given an integer array `nums`, return `true` if any value appears at least twice in the array, and `false` if every element is distinct.

- **Constraints:** `1 <= nums.length <= 10^5`; `-10^9 <= nums[i] <= 10^9`; the array is unsorted and may contain negative numbers.
- **Clarifying assumptions:** confirm you only need a boolean (not the duplicate's value or index), and confirm the array isn't guaranteed sorted (if it were, adjacent-pair comparison would suffice without any extra space).
- **Why it's tricky:** it's a warm-up problem, but it's the canonical example of trading O(n) space for a hash set to avoid the naive O(n²) all-pairs comparison — the pattern every later problem in this section builds on.

Example.

```
Input:  [1, 2, 3, 1]
Output: true            (1 appears twice)
```

Intuition (diagram). Walk the array once. Before adding a number to your "seen" set, check if it's already there — if so, you've found your duplicate. A hash set gives O(1) membership checks, so this beats comparing every pair.

```
nums: 1  2  3  1
seen: {}

1 -> not in seen -> add -> seen {1}
2 -> not in seen -> add -> seen {1,2}
3 -> not in seen -> add -> seen {1,2,3}
1 -> ALREADY in seen -> return true
```

Brute force

Compare every pair of elements.

```
public boolean containsDuplicateBrute(int[] nums) {
    for (int i = 0; i < nums.length; i++) {
        for (int j = i + 1; j < nums.length; j++) {
            if (nums[i] == nums[j]) {
                return true;
            }
        }
    }
    return false;
}
```

Optimized (hash set)

Track everything seen so far; a repeat means an immediate hit.

```
public boolean containsDuplicate(int[] nums) {
    Set<Integer> seen = new HashSet<>();
    for (int num : nums) {
        if (!seen.add(num)) {         // add() returns false if the value was already prese
            return true;
        }
    }
    return false;
}
```

Version	Time	Space
Brute force	$O(n^2)$	O(1)
Hash set	O(n)	O(n)

3.5 Top K Frequent Elements

Problem. Given an integer array `nums` and an integer `k`, return an array of the `k` values that occur most frequently in `nums`. The answer may be returned in any order.

- **Constraints:** `1 <= nums.length <= 10^5`; `-10^4 <= nums[i] <= 10^4`; `k` is between `1` and the number of distinct elements in `nums`, so the answer is always well-defined (no need to handle `k` larger than the distinct-value count).
- **Clarifying assumptions:** confirm ties (two values with equal frequency, only one of which can fit in the top `k`) may be broken arbitrarily, and confirm the output need not be sorted by frequency.
- **Why it's tricky:** the obvious approach sorts all distinct values by frequency (O(n log n)); the faster insight is that frequency is bounded by `n`, so you can bucket values by frequency and read off the top ones in O(n), skipping the sort entirely.

Example.

```
Input:  nums = [1, 1, 1, 2, 2, 3], k = 2
Output: [1, 2]          (1 appears 3x, 2 appears 2x, 3 appears 1x)
```

Intuition (diagram). First count every value's frequency with a hashmap (that part is unavoidable). The naive next step is to sort by frequency, which costs O(n log n). Instead, notice frequency can only range from 1 to n, so create "buckets" indexed by frequency — bucket[3] holds every value that appears exactly 3 times. Then just walk the buckets from highest frequency down and collect values until you have k. (A min-heap of size k is a common alternative — O(n log k) — but bucket sort avoids the heap entirely here.)

```
nums = [1,1,1,2,2,3]     counts: {1:3, 2:2, 3:1}

buckets (index = frequency, value = list of nums with that frequency)

idx:      0      1      2      3
        +----+ +----+ +----+ +----+
bucket: |    | | [3]| | [2]| | [1]|
        +----+ +----+ +----+ +----+

walk idx from 3 down to 0, collecting values until k are found:
idx=3 -> take [1]        result=[1]        (k=2, need 1 more)
idx=2 -> take [2]        result=[1,2]      (k=2, done)
idx=1 -> (not reached, already have k)
```

Approach (step by step)

Key idea / invariant: frequency in `nums` can never exceed `nums.length`, so instead of sorting distinct values by frequency (O(n log n)), index an array of buckets by frequency itself: `buckets[f]` holds every value that occurs exactly `f` times. Walking the buckets from the highest index down visits values in descending frequency order "for free," with no comparisons at all.

1. Count every value's frequency into a hashmap `countOf` (one pass over `nums`).
2. Allocate `buckets`, an array of lists sized `nums.length + 1` (frequency ranges from `1` to `nums.length`).
3. For each `(value, freq)` entry in `countOf`, append `value` to `buckets[freq]`.
4. Walk `freq` from `buckets.length - 1` down to `0`. For each non-empty bucket, take its values into the result array until `k` values have been collected, then stop.
5. Return the result.

Worked trace (`nums = [1,1,1,2,2,3]`, `k = 2`):

```
countOf = {1:3, 2:2, 3:1}
buckets[1] = [3]
buckets[2] = [2]
buckets[3] = [1]

freq=3: buckets[3]=[1] -> result=[1]        filled=1
freq=2: buckets[2]=[2] -> result=[1,2]      filled=2 -> stop (k reached)
```

Brute force

Count frequencies, then sort all entries by frequency.

```
public int[] topKFrequentBrute(int[] nums, int k) {
    Map<Integer, Integer> countOf = new HashMap<>();
    for (int num : nums) {
        countOf.merge(num, 1, Integer::sum);
    }
    List<Integer> values = new ArrayList<>(countOf.keySet());
    values.sort((a, b) -> countOf.get(b) - countOf.get(a));   // O(n log n) sort by frequer
    int[] result = new int[k];
    for (int i = 0; i < k; i++) {
        result[i] = values.get(i);
    }
    return result;
}
```

Optimized (hashmap count + bucket sort)

Frequencies are bounded by `nums.length`, so bucket them instead of sorting.

```
public int[] topKFrequent(int[] nums, int k) {
    Map<Integer, Integer> countOf = new HashMap<>();
    for (int num : nums) {
        countOf.merge(num, 1, Integer::sum);       // count occurrences
    }

    // bucket[f] = list of values that occur exactly f times
    List<Integer>[] buckets = new List[nums.length + 1];
    for (Map.Entry<Integer, Integer> entry : countOf.entrySet()) {
        int freq = entry.getValue();
        if (buckets[freq] == null) {
            buckets[freq] = new ArrayList<>();
        }
        buckets[freq].add(entry.getKey());
    }

    int[] result = new int[k];
    int filled = 0;
    for (int freq = buckets.length - 1; freq >= 0 && filled < k; freq--) {
        if (buckets[freq] == null) continue;
        for (int value : buckets[freq]) {
            if (filled == k) break;
            result[filled++] = value;
        }
    }
    return result;
}
```

Version	Time	Space
Brute force (sort by frequency)	O(n log n)	O(n)
Hashmap + bucket sort	O(n)	O(n)
Min-heap of size k (alternative)	O(n log k)	O(n)

---

4. Binary Search

The core idea. If you can ask "is the answer to my left or my right?" about the middle element and get a reliable yes/no, you can throw away half the remaining search space with every question. Keep a `lo` / `hi` window, look at the midpoint, decide which half can still contain what you want, and shrink the window to that half. Each step is O(1), and the window halves every time, so the whole search is O(log n).

Reach for it when: the data is **sorted** (or has some other monotonic structure), or the problem can be rephrased as "find the smallest/largest value for which some yes/no condition first becomes true" — even if there's no array to search at all.

4.1 Binary Search

Problem. Given a sorted array of distinct integers `nums` (ascending order) and an integer `target`, return the index `i` such that `nums[i] == target`, or `-1` if `target` is not present in `nums`.

- **Constraints:** `1 <= nums.length <= 10^4`; `-10^4 <= nums[i], target <= 10^4`; `nums` is sorted in strictly ascending order (no duplicates).
- **Clarifying assumptions:** confirm the array is truly sorted ascending (not descending) and that values are distinct — duplicates would make "the" index ambiguous.
- **Why it's tricky:** off-by-one errors in the `lo` / `hi` update and mid-point overflow are the classic traps, not the algorithmic idea itself.

Example.

```
Input:  nums = [-1, 0, 3, 5, 9, 12], target = 9
Output: 4
```

Intuition (diagram). Look at the middle element. If it's the target, done. If the target is bigger, it can only be in the right half, so move `lo` past mid. If smaller, move `hi` before mid. Every comparison halves the window.

```
idx:       0   1   2   3   4   5
nums:     -1   0   3   5   9  12    target = 9
          lo     mid          hi    mid=2 -> nums[2]=3 < 9, search right
                      lo mid   hi    mid=4 -> nums[4]=9 == 9, found!
```

Approach (step by step)

Invariant: if `target` exists in `nums`, it is always within `nums[lo..hi]` (inclusive). Every iteration either finds `target` or shrinks this window by discarding a half that provably cannot contain it, while preserving the invariant for the half that remains.

1. Set `lo = 0`, `hi = nums.length - 1` — the invariant window starts as the whole array.
2. While `lo <= hi` (the window is non-empty), compute `mid = lo + (hi - lo) / 2`.

3. Compare `nums[mid]` to `target`:
4. Equal: return `mid` immediately.
5. `nums[mid] < target`: since the array is sorted ascending, everything at or left of `mid` is `<= nums[mid] < target`, so it can be discarded — set `lo = mid + 1`.
6. `nums[mid] > target`: symmetrically, everything at or right of `mid` is too big, so discard it — set `hi = mid - 1`.
7. **Loop termination:** the loop stops when `lo > hi`, meaning the window is empty and `target` cannot be anywhere in `nums` — return `-1`.

Worked trace on `nums = [-1, 0, 3, 5, 9, 12]`, `target = 9`: - `lo=0, hi=5` -> `mid=2`, `nums[2]=3 < 9` -> discard `[0..2]`, `lo=3`. - `lo=3, hi=5` -> `mid=4`, `nums[4]=9 == 9` -> return `4`.

Brute force

Scan every element linearly.

```
public int searchBrute(int[] nums, int target) {
    for (int i = 0; i < nums.length; i++) {
        if (nums[i] == target) {
            return i;
        }
    }
    return -1;
}
```

Optimized (binary search template)

Halve the search window each step. `lo + (hi - lo) / 2` avoids the overflow that `(lo + hi) / 2` risks when `lo` and `hi` are both close to `Integer.MAX_VALUE`.

```
public int search(int[] nums, int target) {
    int lo = 0, hi = nums.length - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;   // safe midpoint, no overflow
        if (nums[mid] == target) {
            return mid;
        } else if (nums[mid] < target) {
            lo = mid + 1;               // target is in the right half
        } else {
            hi = mid - 1;               // target is in the left half
        }
    }
    return -1;                          // window closed, not found
}
```

Version	Time	Space
Brute force	O(n)	O(1)
Binary search	O(log n)	O(1)

4.2 Search in Rotated Sorted Array

Problem. An ascending array of **distinct** integers was rotated between `1` and `nums.length` positions at some unknown pivot, e.g. `[0,1,2,4,5,6,7]` becomes `[4,5,6,7,0,1,2]`. Given the rotated array `nums` and an integer `target`, return the index of `target` in `nums`, or `-1` if it isn't present.

- **Constraints:** `1 <= nums.length <= 5000`; `-10^4 <= nums[i] <= 10^4`; all values in `nums` are **unique**; `nums` is guaranteed to be a rotation of an ascending array.
- **Clarifying assumptions:** confirm values are distinct (duplicates break the "which half is sorted" test — see the follow-up variant) and that the array can be rotated by 0 (i.e. it may already be fully sorted).
- **Why it's tricky:** the array as a whole is not sorted, so the naive "compare to mid and go left/right" rule doesn't directly apply — you must first figure out *which half is sorted* before deciding where `target` could be.

Example.

```
Input:  nums = [4, 5, 6, 7, 0, 1, 2], target = 0
Output: 4
```

Intuition (diagram). The array isn't fully sorted, but one of the two halves around any `mid` always is. Check which half is sorted by comparing `nums[lo]` to `nums[mid]`. Then check if `target` falls in that sorted half's range — if it does, search there; otherwise search the other half.

```
idx:       0   1   2   3   4   5   6
nums:      4   5   6   7   0   1   2    target = 0
          lo         mid          hi    left half [4..7] sorted, 0 not in [4,7] -> go right
                          lo mid  hi    left half [0..1] sorted, 0 is in [0,1] -> go left
                        lo/mid/hi        mid = nums[4] = 0 == target -> found!
```

Approach (step by step)

Invariant: at every step, at least one of `nums[lo..mid]` or `nums[mid..hi]` is a properly (non-rotated) ascending run. We identify that sorted half, check whether `target` can be in its range, and discard whichever half provably cannot contain `target`.

1. Set `lo = 0`, `hi = nums.length - 1`.
2. While `lo <= hi`, compute `mid = lo + (hi - lo) / 2`. If `nums[mid] == target`, return `mid`.
3. Decide which half is sorted by comparing `nums[lo]` to `nums[mid]`:
4. If `nums[lo] <= nums[mid]`, the **left half** `nums[lo..mid]` is sorted ascending.
    - If `target` falls inside `[nums[lo], nums[mid])`, it must be in the left half: `hi = mid - 1`.
    - Otherwise `target` (if present at all) is in the right half: `lo = mid + 1`.
5. Otherwise the **right half** `nums[mid..hi]` is sorted ascending.
    - If `target` falls inside `(nums[mid], nums[hi]]`, it must be in the right half: `lo = mid + 1`.
    - Otherwise it must be in the left half: `hi = mid - 1`.
6. **Loop termination:** the loop stops when `lo > hi` (window empty) — return `-1`, since every step only discards a half proven not to contain `target`.

Worked trace on `nums = [4, 5, 6, 7, 0, 1, 2]`, `target = 0`:- `lo=0, hi=6` -> `mid=3`, `nums[3]=7`. `nums[lo]=4 <= nums[mid]=7` so left half `[4,5,6,7]` is sorted. Is `0` in `[4, 7)`? No -> search right: `lo=4`.- `lo=4, hi=6` -> `mid=5`, `nums[5]=1`. `nums[lo]=0 <= nums[mid]=1` so left half `[0,1]` is sorted. Is `0` in `[0, 1)`? Yes -> search left: `hi=4`.- `lo=4, hi=4` -> `mid=4`, `nums[4]=0 == target` -> return `4`.

Brute force

Scan every element linearly (rotation doesn't matter if you don't need log n).

```
public int searchRotatedBrute(int[] nums, int target) {
    for (int i = 0; i < nums.length; i++) {
        if (nums[i] == target) {
            return i;
        }
    }
    return -1;
}
```

Optimized (binary search, identify the sorted half)

At every step at least one half is properly sorted; use that to decide where to look.

```
public int searchRotated(int[] nums, int target) {
    int lo = 0, hi = nums.length - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;   // safe midpoint, no overflow
        if (nums[mid] == target) {
            return mid;
        }
        if (nums[lo] <= nums[mid]) {
            // left half [lo..mid] is sorted
            if (nums[lo] <= target && target < nums[mid]) {
                hi = mid - 1;           // target in the sorted left half
            } else {
                lo = mid + 1;           // target must be in the right half
            }
        } else {
            // right half [mid..hi] is sorted
            if (nums[mid] < target && target <= nums[hi]) {
                lo = mid + 1;           // target in the sorted right half
            } else {
                hi = mid - 1;           // target must be in the left half
            }
        }
    }
    return -1;
}
```

Version	Time	Space
Brute force	O(n)	O(1)
Binary search	O(log n)	O(1)

4.3 Find Minimum in Rotated Sorted Array

Problem. An ascending array of **distinct** integers was rotated between `1` and `nums.length` positions at some unknown pivot. Given the rotated array `nums`, return its minimum element — this is exactly the value at the pivot where the rotation happened.

- **Constraints:** `1 <= nums.length <= 5000`; `-5000 <= nums[i] <= 5000`; all values are **unique**; `nums` is a rotation of an ascending array (rotation count may be 0).
- **Clarifying assumptions:** confirm values are distinct (with duplicates the standard `nums[mid]` vs `nums[hi]` comparison becomes ambiguous and needs a linear fallback), and that we only need the value, not its index.
- **Why it's tricky:** you must binary-search for a *structural boundary* (the rotation point) rather than a specific value, which changes the loop condition and the `hi` update rule (`hi = mid`, not `mid - 1`) versus a normal search.

Example.

```
Input:  nums = [4, 5, 6, 7, 0, 1, 2]
Output: 0
```

Intuition (diagram). The minimum is the only element smaller than the array's last element (unless the array isn't rotated at all). Compare `nums[mid]` to `nums[hi]`: if `nums[mid] > nums[hi]`, the pivot is to the right of `mid`, so it survives in the window; otherwise the pivot is at or before `mid`, so `mid` itself could be the answer and `hi` can shrink to `mid`.

```
idx:       0   1   2   3   4   5   6
nums:      4   5   6   7   0   1   2
          lo         mid          hi    nums[mid]=7 > nums[hi]=2 -> min is right of mid
                          lo mid  hi    nums[mid]=1 <= nums[hi]=2 -> min is at mid or left
                       lo/hi=mid        window closed -> nums[lo] = 0 is the minimum
```

Approach (step by step)

Invariant: the minimum (rotation pivot) always lies within `nums[lo..hi]` (inclusive). Unlike 4.1/4.2, we don't compare against a target value — we compare `nums[mid]` to `nums[hi]` to decide which side the *boundary* is on, and we never discard `mid` itself because it could be the answer.

1. Set `lo = 0`, `hi = nums.length - 1`.
2. While `lo < hi` (note: strict `<`, not `<=` — see termination below), compute `mid = lo + (hi - lo) / 2`.
3. Compare `nums[mid]` to `nums[hi]`:
4. `nums[mid] > nums[hi]`: the run from `lo` to `mid` is "too high," meaning the array wraps around somewhere in `(mid, hi]`, so the minimum is strictly right of `mid`. Set `lo = mid + 1` (safe to exclude `mid`, since `nums[mid] > nums[hi]` proves `mid` isn't the minimum).
5. `nums[mid] <= nums[hi]`: the run from `mid` to `hi` is already ascending (no wrap in `(mid, hi]`), so the minimum is at `mid` or to its left. Set `hi = mid` (cannot exclude `mid` — it might itself be the minimum).
6. **Loop termination:** because `hi` is set to `mid` (never `mid - 1`) in one branch, using `lo < hi` instead of `lo <= hi` avoids infinite looping — the window shrinks by at least one element every

iteration and stops exactly when `lo == hi`, which is the answer index.

Worked trace on `nums = [4, 5, 6, 7, 0, 1, 2]`: - `lo=0, hi=6` -> `mid=3`, `nums[3]=7 > nums[6]=2` -> min is right of mid -> `lo=4`. - `lo=4, hi=6` -> `mid=5`, `nums[5]=1 <= nums[6]=2` -> min is at mid or left -> `hi=5`. - `lo=4, hi=5` -> `mid=4`, `nums[4]=0 <= nums[5]=1` -> min is at mid or left -> `hi=4`. - `lo=4, hi=4` -> loop ends -> return `nums[4] = 0`.

Brute force

Scan for the smallest value directly.

```
public int findMinBrute(int[] nums) {
    int min = nums[0];
    for (int n : nums) {
        min = Math.min(min, n);
    }
    return min;
}
```

Optimized (binary search on the pivot)

Shrink toward the rotation point; `hi` moves to `mid` (not `mid - 1`) because `mid` might already be the minimum.

```
public int findMin(int[] nums) {
    int lo = 0, hi = nums.length - 1;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;   // safe midpoint, no overflow
        if (nums[mid] > nums[hi]) {
            lo = mid + 1;               // pivot (min) is to the right of mid
        } else {
            hi = mid;                   // pivot is at mid or to its left
        }
    }
    return nums[lo];                    // lo == hi, pointing at the minimum
}
```

Version	Time	Space
Brute force	O(n)	O(1)
Binary search	O(log n)	O(1)

4.4 Koko Eating Bananas

Problem. Koko has `n` piles of bananas, `piles[i]` bananas in the `i`-th pile, and must eat all of them within `h` hours. Each hour she picks exactly one pile and eats up to `speed` bananas from it; if the pile has fewer than `speed` bananas she finishes that pile and the rest of the hour is wasted (she cannot start a second pile in the same hour). Return the minimum integer `speed` such that she can finish all piles within `h` hours.

- **Constraints:** `1 <= piles.length <= 10^4`; `piles.length <= h <= 10^9`; `1 <= piles[i] <= 10^9`.
- **Clarifying assumptions:** confirm `h >= piles.length` is guaranteed (otherwise no speed works, since each pile needs at least one full hour), and that `speed` must be a positive integer.
- **Why it's tricky:** there's no array to binary search over — you must recognize that the *answer itself* (the speed) lives on a monotonic yes/no boundary, and binary search on that abstract range instead of on `piles`.

Example.

```
Input:  piles = [3, 6, 7, 11], h = 8
Output: 4
```

Intuition (diagram). This isn't a search over an array — it's a **search on the answer space**. The "answer" is the eating speed, ranging from `1` to `max(piles)`. For any given speed we can check in O(n) whether it's fast enough (a **monotonic predicate**: if speed `s` works, every speed greater than `s` also works). That monotonic yes/no is exactly what binary search needs — so binary search directly on the *speed*, not on the piles.

```
speed:     1   2   3   4   5   6   7  ...  max(piles)
works?     N   N   N   Y   Y   Y   Y  ...  Y
                       ^
                  first speed that works <- binary search finds this boundary
```

Approach (step by step)

Invariant: binary search over the range `[lo, hi] = [1, max(piles)]` of candidate speeds. The predicate `hoursNeeded(speed) <= h` is **monotonic** in `speed` (faster speed never needs more hours), so the set of working speeds is a contiguous suffix `[answer, max(piles)]`. We search for the left edge of that suffix — the smallest speed that still works.

1. Set `lo = 1` (slowest conceivable speed) and `hi = max(piles)` (eating an entire pile in one hour is always fast enough, so this speed always works).
2. While `lo < hi`, compute `mid = lo + (hi - lo) / 2` and evaluate `hoursNeeded(piles, mid)`.
3. Compare the hours needed at `mid` to the budget `h`:
4. `hoursNeeded(mid) <= h`: speed `mid` is fast enough. Because the predicate is monotonic, every speed above `mid` also works, so the true answer is at `mid` or slower — narrow the window to the left: `hi = mid` (keep `mid`, since it might be the minimum working speed).
5. `hoursNeeded(mid) > h`: speed `mid` is too slow; `mid` and everything slower is disqualified — narrow to the right: `lo = mid + 1`.
6. **Loop termination:** `hi = mid` (not `mid - 1`) mirrors 4.3's pivot search, so the loop uses `lo < hi` to guarantee termination; it ends when `lo == hi`, which is the smallest speed for which the predicate is true.

Worked trace on `piles = [3, 6, 7, 11]`, `h = 8`: - `lo=1, hi=11` -> `mid=6`: hours = ceil(3/6)+ceil(6/6)+ceil(7/6)+ceil(11/6) = 1+1+2+2 = 6 <= 8 -> works -> `hi=6`. - `lo=1, hi=6` -> `mid=3`: hours = 1+2+3+4 = 10 > 8 -> too slow -> `lo=4`. - `lo=4, hi=6` -> `mid=5`: hours = 1+2+2+3 = 8 <= 8 ->

works -> `hi=5`. - `lo=4, hi=5` -> `mid=4`: hours = 1+2+2+3 = 8 <= 8 -> works -> `hi=4`. - `lo=4, hi=4` -> loop ends -> return `4`.

Brute force

Try every speed from 1 upward until one finishes in time.

```
public int minEatingSpeedBrute(int[] piles, int h) {
    int maxPile = 0;
    for (int p : piles) maxPile = Math.max(maxPile, p);
    for (int speed = 1; speed <= maxPile; speed++) {
        if (hoursNeeded(piles, speed) <= h) {
            return speed;
        }
    }
    return maxPile;
}

private int hoursNeeded(int[] piles, int speed) {
    int hours = 0;
    for (int p : piles) {
        hours += (p + speed - 1) / speed;   // ceil(p / speed) without floats
    }
    return hours;
}
```

Optimized (binary search on the answer space)

Binary search over possible speeds `[1, max(piles)]`; the predicate "does this speed finish in time?" is monotonic, so we can shrink the window just like a sorted array.

```
public int minEatingSpeed(int[] piles, int h) {
    int lo = 1, hi = 0;
    for (int p : piles) hi = Math.max(hi, p);   // slowest useful speed = biggest pile

    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;   // safe midpoint, no overflow
        if (hoursNeeded(piles, mid) <= h) {
            hi = mid;                   // mid works: try to go slower
        } else {
            lo = mid + 1;               // mid too slow: must go faster
        }
    }
    return lo;                          // smallest speed that still meets the deadline
}

private int hoursNeeded(int[] piles, int speed) {
    int hours = 0;
    for (int p : piles) {
        hours += (p + speed - 1) / speed;   // ceil(p / speed) without floats
    }
    return hours;
}
```

Version	Time	Space
Brute force	O(n * max(piles))	O(1)

Version	Time	Space
Binary search on answer	O(n log(max(piles)))	O(1)

---

5. Linked List

The core idea. A linked list is a chain of nodes where each node only knows about the next one. You can't jump to an index like an array — you must walk pointer by pointer. Most linked-list tricks come down to carefully juggling a handful of node references (`prev`, `curr`, `next`, a `dummy` head, or two pointers moving at different speeds) without ever losing the thread back to the rest of the list.

Reach for it when: you need to **rewire pointers in place** instead of copying data, **merge or splice** multiple chains together, or detect a **cycle / find a position relative to the end** without knowing the length up front.

All problems below assume this node definition:

```
class ListNode { int val; ListNode next; ListNode(int v){ val = v; } }
```

5.1 Reverse Linked List

Problem. Given the head of a singly linked list, reverse the direction of every `next` pointer so the list is traversed in the opposite order, and return the new head (the old tail). Do this **in place** — don't allocate a second list of nodes.

- **Constraints:** the list has between `0` and `5000` nodes; node values fit in a 32-bit int and may repeat or be negative; the list is singly linked (no `prev` pointer available).
- **Clarifying assumptions:** an empty list (`head == null`) or a single-node list should be returned unchanged; you should mutate the existing nodes rather than copy values into new ones, since that's the point of the exercise.
- **Why it's tricky:** once you overwrite `curr.next` to point backward, you lose the only reference to the rest of the original list — you must save "what's next" *before* you rewire it.

Example.

```
Input:  1 -> 2 -> 3 -> 4 -> 5 -> null
Output: 5 -> 4 -> 3 -> 2 -> 1 -> null
```

Intuition (diagram). Walk the list once. At each node, flip its `next` pointer to point backward instead of forward, using three pointers — `prev`, `curr`, `next` — so you never lose track of the rest of the chain.

```
null -> [1] -> [2] -> [3] -> null      step 0: nothing reversed yet
prev    curr

null <- [1]    [2] -> [3] -> null      step 1: 1.next = prev; slide all three forward
prev    curr   next

null <- [1] <- [2]    [3] -> null      step 2: repeat until curr == null
               prev   curr
```

Brute force

Collect values into a list, then rebuild the list in reverse order.

```
public ListNode reverseListBrute(ListNode head) {
    List<Integer> values = new ArrayList<>();
    for (ListNode n = head; n != null; n = n.next) {
        values.add(n.val);                // record original order
    }
    ListNode dummy = new ListNode(0);
    ListNode tail = dummy;
    for (int i = values.size() - 1; i >= 0; i--) {
        tail.next = new ListNode(values.get(i));  // build new nodes, reversed
        tail = tail.next;
    }
    return dummy.next;
}
```

Optimized (3-pointer iterative reversal)

Rewire `next` pointers in a single pass, no extra nodes.

```
public ListNode reverseList(ListNode head) {
    ListNode prev = null;
    ListNode curr = head;
    while (curr != null) {
        ListNode next = curr.next;   // save the rest of the list before overwriting
        curr.next = prev;            // flip this node's pointer backward
        prev = curr;                 // advance prev
        curr = next;                 // advance curr
    }
    return prev;                     // prev ends up at the new head
}
```

Version	Time	Space
Brute force (rebuild)	O(n)	O(n) new nodes
3-pointer iterative	O(n)	O(1)

5.2 Merge Two Sorted Lists

Problem. Given the heads of two singly linked lists that are each already sorted in non-decreasing order, merge them into a single sorted list by splicing together the existing nodes, and return the head of the merged list.

- **Constraints:** each list has between `0` and `5000` nodes; values fit in a 32-bit int and may include duplicates (within or across the two lists).
- **Clarifying assumptions:** if one input is empty, return the other list unchanged; ties (equal values) can be resolved either way — the examples below take `l1`'s node first.
- **Why it's tricky:** you must produce the result by **relinking** existing nodes (not copying values), and the "first node of the result" is annoying to special-case — a dummy head sidesteps that.

Example.

```
Input:  l1 = 1 -> 3 -> 5 -> null,  l2 = 2 -> 4 -> 6 -> null
Output: 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> null
```

Intuition (diagram). Use a **dummy head** so you never special-case "what's the first node of the result." Keep a `tail` pointer; at each step, splice on whichever list's current node is smaller, then advance that list. No new nodes are allocated — you just relink existing ones.

```
l1:  1 -> 3 -> 5 -> null
l2:  2 -> 4 -> 6 -> null

dummy -> 1                    1 < 2: splice l1's node, advance l1
dummy -> 1 -> 2               2 < 3: splice l2's node, advance l2
dummy -> 1 -> 2 -> 3 -> ...   continue until one list runs out
```

Approach (step by step). Two pointers, `l1` and `l2`, each mark the *unmerged* head of their own list; the invariant is that everything before them has already been spliced into the result in order. 1. Create a `dummy` node and a `tail` pointer starting at `dummy`. 2. While both `l1` and `l2` are non-null, compare their values; splice the smaller node onto `tail.next`, then advance that list's pointer and `tail`. 3. When one list runs out, attach whatever remains of the other list directly — it's already sorted, so no more comparisons are needed. 4. Return `dummy.next` (the real head, skipping the placeholder).

Brute force

Copy every value into an array, sort it, then build a brand-new list.

```
public ListNode mergeTwoListsBrute(ListNode l1, ListNode l2) {
    List<Integer> values = new ArrayList<>();
    for (ListNode n = l1; n != null; n = n.next) values.add(n.val);
    for (ListNode n = l2; n != null; n = n.next) values.add(n.val);
    Collections.sort(values);                 // ignore that inputs were already sorted
    ListNode dummy = new ListNode(0);
    ListNode tail = dummy;
    for (int v : values) {
        tail.next = new ListNode(v);
        tail = tail.next;
    }
    return dummy.next;
}
```

Optimized (dummy head + splice)

Merge in place by relinking nodes from both lists.

```
public ListNode mergeTwoLists(ListNode l1, ListNode l2) {
    ListNode dummy = new ListNode(0);   // placeholder so result always has a "before head"
    ListNode tail = dummy;
    while (l1 != null && l2 != null) {
        if (l1.val <= l2.val) {
            tail.next = l1;             // splice l1's node into the result
            l1 = l1.next;
        } else {
            tail.next = l2;             // splice l2's node into the result
            l2 = l2.next;
        }
        tail = tail.next;
    }
    tail.next = (l1 != null) ? l1 : l2; // attach whichever list still has leftovers
    return dummy.next;
}
```

Version	Time	Space
Brute force (copy + sort)	O((n+m) log(n+m))	O(n+m)
Dummy head + splice	O(n+m)	O(1)

5.3 Linked List Cycle

Problem. Given the head of a linked list, determine whether it contains a cycle, i.e. whether following `next` pointers from some node eventually leads back to a node you've already visited, rather than terminating at `null`.

- **Constraints:** the list has between `0` and `10^4` nodes; there is at most one cycle, and if it exists it can loop back to any earlier node (not necessarily the head); you may **not** modify the list (e.g. by mutating `next` or node values to mark visits).
- **Clarifying assumptions:** an empty list or a list with no cycle should return `false`; you should assume you cannot rely on comparing node *values* to detect a repeat, since values may repeat legitimately — you must compare node *references* (identity).
- **Why it's tricky:** the naive fix (a `HashSet` of visited nodes) works but costs O(n) space; the elegant O(1)-space trick — two pointers moving at different speeds — isn't obvious until you've seen it, and proving *why* they must meet takes a little math.

Example.

```
Input:  [3] -> [2] -> [0] -> [-4]
                ^----------------+
Output: true   (the -4 node's next points back to the 2 node, forming a loop)
```

Intuition (diagram). A slow pointer moves one step at a time, a fast pointer moves two. If there's no cycle, fast simply reaches `null` first. If there **is** a cycle, fast will eventually lap the slow pointer inside the loop and they collide — this is Floyd's tortoise-and-hare trick.

```
slow/fast
[3] -> [2] -> [0] -> [-4]
       ^----------------+

slow advances 1 step, fast advances 2 steps every round
fast re-enters the loop and gains on slow by 1 node per round
eventually slow == fast (same node)  ->  cycle confirmed
```

Approach (step by step). The key invariant: once both pointers are inside the cycle, `fast` closes the gap to `slow` by exactly 1 node every round (it moves 2 steps to `slow`'s 1, inside a loop of fixed length). A gap that shrinks by 1 each round and wraps modulo the cycle length *must* hit 0 — so if a cycle exists, they are guaranteed to land on the same node eventually; they cannot skip past each other because the gap closes one step at a time. 1. Start `slow` and `fast` both at `head`. 2. On each iteration, move `slow` one step (`slow = slow.next`) and `fast` two steps (`fast = fast.next.next`). 3. If `fast` (or `fast.next`) becomes `null` , the list ended cleanly — no cycle, return `false` . 4. If at any point `slow == fast` (the same node object, not just equal value), the pointers collided inside a loop — return `true` .

Worked trace on `3 -> 2 -> 0 -> -4 -> (back to 2)` : - start: `slow=3, fast=3` - round 1: `slow=2, fast=0` - round 2: `slow=0, fast=2` (fast wrapped through the cycle back to 2, then to 0) - round 3: `slow=-4, fast=-4` -> `slow == fast` -> cycle confirmed.

Brute force

Track every node visited in a `HashSet` ; a repeat means a cycle.

```
public boolean hasCycleBrute(ListNode head) {
    Set<ListNode> seen = new HashSet<>();
    for (ListNode n = head; n != null; n = n.next) {
        if (!seen.add(n)) {          // add() returns false if already present
            return true;
        }
    }
    return false;
}
```

Optimized (Floyd's fast/slow pointers)

Two pointers, no extra memory.

```
public boolean hasCycle(ListNode head) {
    ListNode slow = head;
    ListNode fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;          // moves 1 step
        fast = fast.next.next;     // moves 2 steps
        if (slow == fast) {        // they met -> must be inside a cycle
            return true;
        }
    }
    return false;                  // fast hit null -> no cycle
}
```

Version	Time	Space
Brute force (hash set)	O(n)	O(n)
Fast/slow pointers	O(n)	O(1)

5.4 Remove Nth Node From End of List

Problem. Given the head of a singly linked list and an integer `n`, remove the `n`-th node counting from the **end** of the list (`n = 1` is the last node) and return the head of the resulting list. Do it ideally in a single pass, without first computing the length.

- **Constraints:** the list has between `1` and `5000` nodes; `1 <= n <= length of list` (`n` is always valid — you don't need to handle `n` out of range); values fit in a 32-bit int.
- **Clarifying assumptions:** removing the head itself (when `n == length`) must work without special-casing — this is exactly why a **dummy node** before the head is useful; assume there is exactly one node to remove (no duplicates-of-position ambiguity).
- **Why it's tricky:** you don't know the list's length until you've walked it once, so naively you'd need two passes. The one-pass trick — offsetting two pointers by a fixed gap of `n` — lets `fast` hitting the end tell you exactly where `slow` needs to stop.

Example.

```
Input:  1 -> 2 -> 3 -> 4 -> 5 -> null, n = 2
Output: 1 -> 2 -> 3 -> 5 -> null      (4 was the 2nd node from the end)
```

Intuition (diagram). The hard part is not knowing the length in advance. Fix it by first advancing a `fast` pointer `n` steps ahead, then moving `slow` and `fast` together — the constant gap of `n` between them means that when `fast` falls off the end, `slow` is sitting exactly at the node *before* the one to remove.

```
dummy -> [1] -> [2] -> [3] -> [4] -> [5] -> null    n = 2
slow/fast                                          both start at dummy

slow                   fast                        fast advanced n+1 = 3 steps ahead

                       slow                 fast   advance together; fast falls off the end

slow is now at [3], the node right before the target ([4])
unlink: slow.next = slow.next.next  ->  [3] -> [5]
```

Approach (step by step). The invariant maintained throughout is: `fast` is always exactly `n + 1` nodes ahead of `slow`. Because that gap never changes while both pointers advance one step at a time, the moment `fast` runs off the end (`null`), `slow` must be sitting exactly `n + 1` nodes before the end — i.e. one node before the target. 1. Create a `dummy` node pointing at `head` (so removing the real head needs no special case), and set `slow = fast = dummy`. 2. Advance `fast` by `n + 1` steps. Now the gap between `slow` and `fast` is fixed at `n + 1`. 3. Advance `slow` and `fast` one step at a time, together, until `fast` becomes `null`. The gap of `n + 1` is preserved, so `slow` ends up at the node just before the one to remove. 4. Unlink the target: `slow.next = slow.next.next`. 5. Return `dummy.next`.

Worked trace on `1 -> 2 -> 3 -> 4 -> 5`, `n = 2`: - `slow = fast = dummy`. - Advance `fast` by `n+1 = 3`: `fast` moves `dummy -> 1 -> 2 -> 3`, so `fast` is at node `3`. - Advance both together: `slow` moves `dummy -> 1`, `fast` moves `3 -> 4`; then `slow -> 2`, `fast -> 5`; then `slow -> 3`, `fast -> null`. Loop stops. - `slow` is at node `3`, so `slow.next` (node `4`) is unlinked: result is `1 -> 2 -> 3 -> 5`.

Brute force

First pass counts the length, second pass walks to the node just before the target.

```
public ListNode removeNthFromEndBrute(ListNode head, int n) {
    int length = 0;
    for (ListNode node = head; node != null; node = node.next) {
        length++;                       // pass 1: count total nodes
    }
    ListNode dummy = new ListNode(0);
    dummy.next = head;
    ListNode curr = dummy;
    for (int i = 0; i < length - n; i++) {
        curr = curr.next;               // pass 2: walk to node before the target
    }
    curr.next = curr.next.next;         // unlink target node
    return dummy.next;
}
```

Optimized (one-pass two-pointer gap of n)

Keep a fixed gap of `n` between two pointers so one pass suffices.

```
public ListNode removeNthFromEnd(ListNode head, int n) {
    ListNode dummy = new ListNode(0);
    dummy.next = head;
    ListNode slow = dummy;
    ListNode fast = dummy;
    for (int i = 0; i < n + 1; i++) {
        fast = fast.next;               // push fast n+1 ahead of slow
    }
    while (fast != null) {
        slow = slow.next;               // advance both, gap stays at n+1
        fast = fast.next;
    }
    slow.next = slow.next.next;         // slow is right before the target -> unlink it
    return dummy.next;
}
```

Version	Time	Space
Brute force (two passes)	O(n)	O(1)
One-pass two-pointer gap	O(n)	O(1)

---

6. Trees (BFS / DFS)

The core idea. A tree has no cycles, so you never need a "visited" set — you just need a strategy for the order you visit nodes in. **DFS** (recursion or a stack) dives to the bottom before backtracking, which is natural for problems phrased in terms of subtrees ("is the left subtree valid?", "combine results from both children"). **BFS** (a queue) visits nodes level by level, which is natural for problems phrased in terms of depth or "the whole row at once."

Reach for it when: the problem talks about **subtrees, height, or ancestors** (→ DFS), or about **levels, shortest path in an unweighted tree, or per-row output** (→ BFS).

Assume this node definition throughout:

```
class TreeNode { int val; TreeNode left, right; TreeNode(int v){ val = v; } }
```

6.1 Maximum Depth of Binary Tree

Problem. Given the root of a binary tree, return its maximum depth — the number of nodes along the longest path from the root down to the farthest leaf.

- **Input:** the `root` of a binary tree (may be `null` for an empty tree).
- **Output:** an `int`, the number of nodes on the longest root-to-leaf path (an empty tree has depth 0; a single-node tree has depth 1).
- **Constraints (typical):** up to ~10^4 nodes; node values are irrelevant to this problem (only the shape of the tree matters) and can be any 32-bit integer.
- **Clarifying assumptions:** confirm that "depth" counts nodes, not edges (a single root node has depth 1, not 0), and that an empty tree should return 0.
- **Why it's tricky:** it isn't tricky as a recursive definition — the trap is only for candidates who try to track depth with a mutable counter instead of letting the recursion return the answer.

Example.

```
Input:  root = [3, 9, 20, null, null, 15, 7]
Output: 3
```

Intuition (diagram). The depth of a tree is 1 (for the root) plus the depth of its deeper child. That's a recursive definition, so DFS falls out naturally. Equivalently, BFS can count how many "layers" of the queue it processes.

```
     3              depth(3)  = 1 + max(depth(9), depth(20))
   /   \
  9     20          depth(9)  = 1
       /  \
      15   7        depth(15) = depth(7) = 1, so depth(20) = 1 + max(1, 1) = 2
```

Approach 1 (recursive DFS)

Depth of a node = 1 + the larger depth of its two children. A `null` node has depth 0.

```
public int maxDepth(TreeNode root) {
    if (root == null) return 0;                          // base case: empty tree
    int leftDepth = maxDepth(root.left);
    int rightDepth = maxDepth(root.right);
    return 1 + Math.max(leftDepth, rightDepth);
}
```

Approach 2 (iterative BFS)

Process one full level at a time; each full pass through the queue is one level deeper.

```
public int maxDepthBFS(TreeNode root) {
    if (root == null) return 0;
    Queue<TreeNode> queue = new LinkedList<>();
    queue.offer(root);
    int depth = 0;
    while (!queue.isEmpty()) {
        int size = queue.size();          // number of nodes in the current level
        for (int i = 0; i < size; i++) {
            TreeNode node = queue.poll();
            if (node.left != null) queue.offer(node.left);
            if (node.right != null) queue.offer(node.right);
        }
        depth++;                          // finished one whole level
    }
    return depth;
}
```

Version	Time	Space
Recursive DFS	O(n)	O(h) call stack (h = height, worst case O(n))
Iterative BFS	O(n)	O(w) queue (w = max width, worst case O(n))

6.2 Invert Binary Tree

Problem. Given the root of a binary tree, invert it (mirror it left-right, swapping every left child with its sibling right child, recursively) and return the new root.

- **Input:** the `root` of a binary tree (may be `null`).
- **Output:** the `root` of the same tree with every node's left and right children swapped, at every level.
- **Constraints (typical):** up to ~100 nodes for interview purposes (LeetCode allows up to ~10^2–10^4); node values need not be unique.
- **Clarifying assumptions:** confirm whether to invert in place (mutate and return the same nodes, which is standard) or build a new tree — in-place is almost always expected and is what the code below does.

- **Why it's tricky:** it isn't algorithmically hard, but it's a good warm-up for the "return a transformed subtree from each recursive call" pattern used by harder tree problems (e.g. LCA below); the only real trap is swapping before vs. after recursing into the *already-swapped* pointers, which would invert twice.

Example.

```
Input:  root = [4, 2, 7, 1, 3, 6, 9]
Output:       [4, 7, 2, 9, 6, 3, 1]
```

Intuition (diagram). Swap every node's left and right children. Do this at every node, top-down or bottom-up — it doesn't matter, because each swap is independent of the others.

```
before:
              4
           /     \
      2             7
    /   \         /   \
  1       3     6       9

after:
              4
           /     \
      7             2
    /   \         /   \
  9       6     3       1
```

Approach 1 (recursive DFS)

Invert both subtrees, then swap them onto the current node.

```
public TreeNode invertTree(TreeNode root) {
    if (root == null) return null;                // nothing to invert
    TreeNode left = invertTree(root.left);
    TreeNode right = invertTree(root.right);
    root.left = right;                            // swap the (already inverted) children
    root.right = left;
    return root;
}
```

Approach 2 (iterative BFS)

Visit every node with a queue and swap its children as you go; order doesn't matter.

```
public TreeNode invertTreeBFS(TreeNode root) {
    if (root == null) return null;
    Queue<TreeNode> queue = new LinkedList<>();
    queue.offer(root);
    while (!queue.isEmpty()) {
        TreeNode node = queue.poll();
        TreeNode temp = node.left;                // swap children
        node.left = node.right;
        node.right = temp;
        if (node.left != null) queue.offer(node.left);
        if (node.right != null) queue.offer(node.right);
    }
    return root;
}
```

Version	Time	Space
Recursive DFS	O(n)	O(h) call stack
Iterative BFS	O(n)	O(w) queue

6.3 Binary Tree Level Order Traversal

Problem. Given the root of a binary tree, return the values of its nodes grouped by level, from top to bottom, left to right within each level.

- **Input:** the `root` of a binary tree (may be `null`).
- **Output:** a `List<List<Integer>>` — one inner list per level, values left to right.
- **Constraints (typical):** up to ~2000 nodes; node values fit in a 32-bit int and may repeat.
- **Clarifying assumptions:** confirm an empty tree should return an empty list (not a list containing one empty list), and that "level" means BFS depth from the root (root is level 0).
- **Why it's tricky:** the values themselves are trivial to collect — the trick is knowing *where one level ends and the next begins* inside a single queue, since BFS naturally interleaves levels unless you explicitly snapshot a boundary.

Example.

```
Input:  root = [3, 9, 20, null, null, 15, 7]
Output: [[3], [9, 20], [15, 7]]
```

Intuition (diagram). A queue naturally processes nodes in the order they were discovered — level by level. The trick is knowing where one level ends: snapshot the queue's size before the loop, and process exactly that many nodes as "this level."

```
      3              level 0: [3]
    /    \
  9          20      level 1: [9, 20]
           /   \
         15       7  level 2: [15, 7]
```

Approach (step by step)

Key idea / invariant: at the top of every `while` loop iteration, the queue holds *exactly* the nodes of one level, in left-to-right order — nothing more, nothing less. Capturing `size = queue.size()` before draining the queue is what freezes that boundary, even though children get `offer`'d into the same queue during the loop.

1. If the tree is empty, return an empty list immediately.
2. Push the root into the queue. It is level 0's only member.
3. While the queue isn't empty: a. Record `size = queue.size()` — this is how many nodes belong to the *current* level, captured before any children are added. b. Start a fresh `level` list. c. Poll exactly `size` nodes: for each one, add its value to `level`, and enqueue its non-null children (these belong to the *next* level, but they queue up safely behind the remaining current-level nodes). d. Append `level` to the result.
4. Return the result once the queue drains.

Worked trace on `[3, 9, 20, null, null, 15, 7]`:

```
queue = [3]                    size=1 -> level=[3]            result=[[3]]
queue = [9, 20]                size=2 -> level=[9, 20]        result=[[3],[9,20]]
queue = [15, 7]                size=2 -> level=[15, 7]        result=[[3],[9,20],[15,7]]
queue = []                     loop ends
```

Approach 2 (iterative BFS)

BFS is the natural fit here — grouping "by level" is exactly what BFS gives you for free.

```
public List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> result = new ArrayList<>();
    if (root == null) return result;
    Queue<TreeNode> queue = new LinkedList<>();
    queue.offer(root);
    while (!queue.isEmpty()) {
        int size = queue.size();                  // nodes belonging to this level
        List<Integer> level = new ArrayList<>();
        for (int i = 0; i < size; i++) {
            TreeNode node = queue.poll();
            level.add(node.val);
            if (node.left != null) queue.offer(node.left);
            if (node.right != null) queue.offer(node.right);
        }
        result.add(level);
    }
    return result;
}
```

Approach 1 (recursive DFS)

Also possible: DFS while tracking depth, appending each value into `result.get(depth)`.

```
public List<List<Integer>> levelOrderDFS(TreeNode root) {
    List<List<Integer>> result = new ArrayList<>();
    dfs(root, 0, result);
    return result;
}

private void dfs(TreeNode node, int depth, List<List<Integer>> result) {
    if (node == null) return;
    if (depth == result.size()) result.add(new ArrayList<>());  // start a new level
    result.get(depth).add(node.val);
    dfs(node.left, depth + 1, result);
    dfs(node.right, depth + 1, result);
}
```

Version	Time	Space
Iterative BFS	O(n)	O(w) queue + O(n) output
Recursive DFS	O(n)	O(h) call stack + O(n) output

6.4 Validate Binary Search Tree

Problem. Given the root of a binary tree, determine if it is a valid binary search tree (BST): every node's value must be strictly greater than all values in its left subtree and strictly less than all values in its right subtree.

- **Input:** the `root` of a binary tree (may be `null`, which counts as valid).
- **Output:** a `boolean` — `true` if the whole tree satisfies the BST property at every node, not just at each node's immediate children.
- **Constraints (typical):** up to ~10^4 nodes; node values are typically 32-bit ints and usually assumed unique (duplicates are a common follow-up: decide whether `==` goes left or right, and confirm that with the interviewer).
- **Clarifying assumptions:** confirm the ordering is *strict* (`<`, not `<=`) and that "left subtree" and "right subtree" mean *every* descendant, not just the direct child.
- **Why it's tricky:** the natural-looking check `left.val < node.val < right.val` (comparing a node only to its immediate children) is wrong — it misses violations from a *grandparent* or higher ancestor. The fix is to carry a shrinking `(min, max)` bound down from the root through every recursive call, not just compare neighbors.

Example.

```
Input:  root = [5, 1, 4, null, null, 3, 6]
Output: false        (4 is 5's right child, so everything under 4 must be > 5 — but 4
                      itself is not, so the tree is already invalid at node 4)
```

Intuition (diagram). The common bug is checking only `left.val < node.val < right.val` — a node's *direct* children — which misses violations further down. A node deep in the left subtree must still be less than every ancestor above it, not just its immediate parent. The fix is to carry a `(min, max)` bound down from the root and tighten it at every step.

```
     5               bounds for 5: (-inf, +inf)
   /     \
  1          4       bounds for 1: (-inf, 5)  -- ok, 1 < 5      bounds for 4: (5, +inf)
           /   \
          3       6   node 4 fails immediately: 4 must be in (5, +inf) since it's 5's
                       right child, but 4 < 5 -- so the whole tree is invalid here,
                       before recursion even reaches 3 or 6
```

Approach (step by step)

Key idea / invariant: every recursive call receives an open interval `(min, max)` — the range that *this* node's value is allowed to fall in, given every ancestor's constraint so far, not just its direct parent's. Going left tightens the *upper* bound to the parent's value (everything in a left subtree must be smaller); going right tightens the *lower* bound (everything in a right subtree must be larger). A `null` bound means "no constraint yet" (equivalent to -infinity or +infinity).

1. Call the helper on `root` with no bounds: `(min = null, max = null)`.
2. At each node: if `null`, it's a valid (empty) subtree — return `true`.
3. Check the current node's value against its inherited bounds: it must be strictly greater than `min` (if any) and strictly less than `max` (if any). If either check fails, the whole tree is invalid — return `false` immediately (short-circuits further recursion).
4. Recurse left with the *same* `min` but a *new* `max = node.val` (left subtree values must all be less than this node).
5. Recurse right with the *same* `max` but a *new* `min = node.val` (right subtree values must all be greater than this node).
6. The tree is valid only if both recursive calls return `true`.

Worked trace on `[5, 1, 4, null, null, 3, 6]`:

```
isValidBST(5, min=null, max=null)   -> 5 has no bounds yet, ok
  isValidBST(1, min=null, max=5)    -> 1 < 5, ok; 1 has no children, returns true
  isValidBST(4, min=5,    max=null) -> 4 must be > 5. It's not.  returns false
result: false (short-circuits without ever visiting node 3 or node 6)
```

Brute force

For each node, gather all values in its left subtree and all in its right subtree and check the ordering directly — correct but expensive.

```
public boolean isValidBSTBrute(TreeNode root) {
    if (root == null) return true;
    List<Integer> leftVals = new ArrayList<>();
    List<Integer> rightVals = new ArrayList<>();
    collect(root.left, leftVals);
    collect(root.right, rightVals);
    for (int v : leftVals) if (v >= root.val) return false;
    for (int v : rightVals) if (v <= root.val) return false;
    return isValidBSTBrute(root.left) && isValidBSTBrute(root.right);
}

private void collect(TreeNode node, List<Integer> out) {
    if (node == null) return;
    out.add(node.val);
    collect(node.left, out);
    collect(node.right, out);
}
```

Optimized (pass down min/max bounds)

Each recursive call knows the open range `(min, max)` its value must fall in, tightened by every ancestor above it — not just its direct parent.

```
public boolean isValidBST(TreeNode root) {
    return isValidBST(root, null, null);
}

private boolean isValidBST(TreeNode node, Integer min, Integer max) {
    if (node == null) return true;
    if (min != null && node.val <= min) return false;   // must be > every left-ancestor bo
    if (max != null && node.val >= max) return false;    // must be < every right-ancestor
    return isValidBST(node.left, min, node.val)          // tighten upper bound going left
        && isValidBST(node.right, node.val, max);        // tighten lower bound going right
}
```

Version	Time	Space
Brute force	O(n²) worst case	O(n) per subtree collection
Min/max bounds	O(n)	O(h) call stack

6.5 Lowest Common Ancestor of a Binary Tree

Problem. Given the root of a binary tree and two nodes `p` and `q` that exist in the tree, find their lowest common ancestor (LCA) — the deepest node that has both `p` and `q` as descendants (a node can be a descendant of itself).

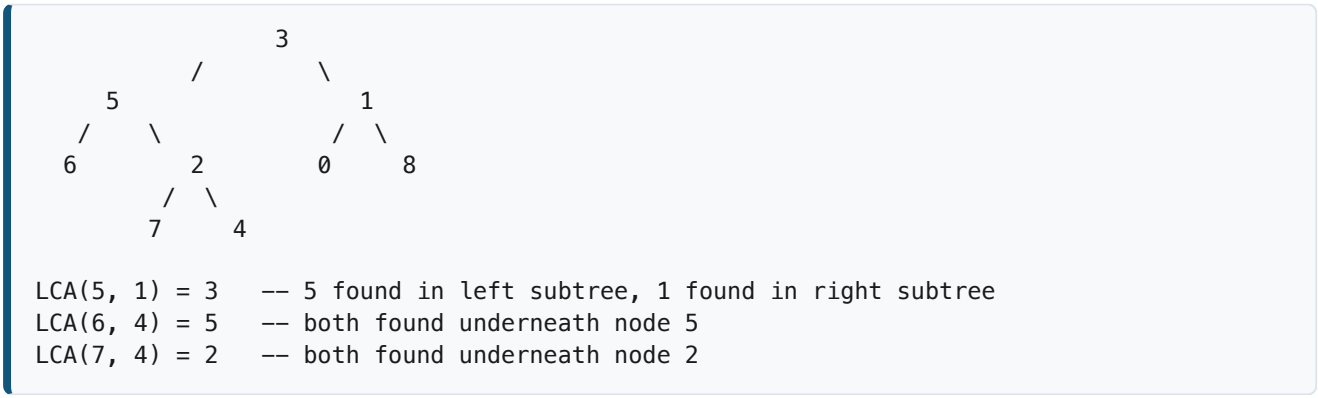
- **Input:** the `root` of a binary tree, and two `TreeNode` references `p` and `q` that are guaranteed to exist somewhere in the tree.
- **Output:** the `TreeNode` that is the deepest common ancestor of `p` and `q`.
- **Constraints (typical):** up to ~10^5 nodes; all node values are typically assumed unique unless stated otherwise; `p != q`.

- **Clarifying assumptions:** confirm that a node **can** be its own ancestor (e.g. `LCA(5, 6) = 5` when `6` is a descendant of `5`), and that both `p` and `q` are guaranteed to be present — you don't need to handle the "not found" case.
- **Why it's tricky:** the answer isn't produced at the moment you "find" `p` or `q` — it only becomes knowable once results from *both* children have bubbled back up to a common ancestor. The key insight is trusting the recursion: a call returns non-null as soon as its subtree contains `p`, `q`, or (once discovered) their LCA, and the parent combines the two children's answers to decide whether *it* is the LCA.

Example.

```
Input:  root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 1
Output: 3          (5 and 1 only share the root as a common ancestor)
```

Intuition (diagram). Search both subtrees. If `p` and `q` turn up in *different* subtrees of a node, that node is the LCA — it's the first place their paths converge. If the current node itself is `p` or `q`, it's an answer candidate right away.



Approach (step by step)

Key idea / invariant: each recursive call on a subtree returns one of three things: `null` (neither `p` nor `q` is anywhere in this subtree), `p` or `q` itself (this subtree contains exactly one target, found so far), or the LCA node (this subtree contains *both* targets, and their meeting point has already been found). The magic happens at the parent level, on the way back up (post-order): a node only knows it is the LCA when it sees a non-null result from *both* its left and right calls.

1. Base case: if the current node is `null`, `p`, or `q`, return it immediately — `null` means "nothing here," and returning `p` / `q` itself signals "found a target."
2. Recurse into the left subtree, then the right subtree, collecting `left` and `right`.
3. **Combine (post-order step):**
4. If both `left` and `right` are non-null, the targets were found in *different* branches beneath this node — this node is the LCA. Return it.
5. Otherwise, at most one side found anything — bubble that one result upward unchanged (`left` if non-null, else `right`, which may itself be `null`).
6. The very last call to return (at the root, or wherever the split happens) carries the final answer up through every ancestor above it unchanged, since once an LCA is found, every node above it just sees "one non-null child" and passes it straight through.

Worked trace for `LCA(5, 1)` on the example tree:

```
call(3): left = call(5), right = call(1)
  call(5): 5 == p, so returns 5 immediately (doesn't need to search under 5)
  call(1): 1 == q, so returns 1 immediately
call(3): left=5, right=1 -- both non-null -> 3 is the LCA, return 3
```

Approach 1 (recursive DFS)

Search left and right. If a node is `p` / `q`, or if `p` and `q` were found in different branches beneath it, that node is the answer — return it up the call stack.

```
public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
    if (root == null || root == p || root == q) {
        return root;                            // found one target, or hit the bottom
    }
    TreeNode left = lowestCommonAncestor(root.left, p, q);
    TreeNode right = lowestCommonAncestor(root.right, p, q);
    if (left != null && right != null) {
        return root;                            // p and q split across both sides -> here
    }
    return left != null ? left : right;         // bubble up whichever side found somethir
}
```

Approach 2 (iterative BFS + parent pointers)

Map every node to its parent via BFS, then walk both `p` and `q` up to the root collecting ancestor chains, and take the deepest node common to both.

```
public TreeNode lowestCommonAncestorBFS(TreeNode root, TreeNode p, TreeNode q) {
    Map<TreeNode, TreeNode> parent = new HashMap<>();
    Queue<TreeNode> queue = new LinkedList<>();
    queue.offer(root);
    parent.put(root, null);
    while (!parent.containsKey(p) || !parent.containsKey(q)) {
        TreeNode node = queue.poll();
        if (node.left != null) { parent.put(node.left, node); queue.offer(node.left); }
        if (node.right != null) { parent.put(node.right, node); queue.offer(node.right); }
    }
    Set<TreeNode> ancestorsOfP = new HashSet<>();
    while (p != null) { ancestorsOfP.add(p); p = parent.get(p); }
    while (!ancestorsOfP.contains(q)) { q = parent.get(q); }  // walk q up until it hits p'
    return q;
}
```

Version	Time	Space
Recursive DFS	O(n)	O(h) call stack
BFS + parent map	O(n)	O(n) for the parent map

---

7. Graphs

The core idea. A graph is just nodes connected by edges. Two traversal strategies cover almost every interview problem: **BFS** (explore level by level using a queue — good for "shortest path" / "minimum steps") and **DFS** (explore as deep as possible using recursion or a stack — good for "does a path exist" / "explore all of a region"). A **2D grid** is secretly a graph where each cell is a node and its up/down/left/right neighbors are edges. An explicit graph is usually given as an **adjacency list** (`Map<Node, List<Node>>` or `List<List<Integer>>`). In both cases you need a **visited set** (or a mutated grid) so you never process the same node twice and never loop forever.

Reach for it when: you see a **grid** with regions/paths to explore, an explicit graph of **nodes and edges**, a **dependency/ordering** problem, or anything asking for **shortest path**, **connectivity**, or **spreading/propagation** over time.

7.1 Number of Islands

Problem. Given an `m x n` grid where every cell is `'1'` (land) or `'0'` (water), return the number of islands. An island is a maximal group of `'1'` cells connected 4-directionally (up/down/left/right); diagonal neighbors don't count.

Constraints - `1 <= m, n <= 300` - `grid[i][j]` is the character `'1'` or `'0'` (a `char[][]`, not booleans or ints). - The grid can contain zero, one, or many islands, and may be all water.

Clarifying assumptions - Confirm connectivity is 4-directional only, not 8-directional (diagonals don't join islands). - Assume it's fine to mutate the input grid in place to mark visited cells; if the interviewer wants the input preserved, use a separate `boolean[][] visited` instead.

Why it's tricky: it looks like "count the 1's," but the real task is grouping connected cells into components without re-counting a cell that's already part of an island you've already found — a connected-components problem hiding inside a grid.

Example.

```
Input:
11000
11000
00100
00011
Output: 3
```

Intuition (diagram). Scan every cell. When you find unvisited land, that's a brand-new island — flood-fill outward (DFS or BFS) marking every connected land cell as visited so the outer scan never counts it again.

```
grid (row, col):
row 0:  1  1  0  0  0
row 1:  1  1  0  0  0
row 2:  0  0  1  0  0
row 3:  0  0  0  1  1

island A: (0,0) (0,1) (1,0) (1,1)   top-left block
island B: (2,2)                     single cell
island C: (3,3) (3,4)               bottom-right pair

the outer scan runs row-major; the first unvisited '1' it meets
starts a flood fill that sinks every 4-directionally connected
'1' to '0' before the scan resumes.
```

Approach (step by step)

Key idea / invariant. Every cell is visited at most once. As soon as a cell is sunk (flipped to `'0'`), it can never start a new island and can never be re-entered by a later flood fill. Because the outer scan and the flood fill share this invariant, each connected component of land is discovered exactly once — at the first unvisited cell that belongs to it.

1. Scan the grid row by row, left to right.
2. When the scan meets a cell that's still `'1'`, no earlier flood fill has claimed it, so it must be the start of a brand-new island — increment the island count.
3. Flood-fill outward from that cell (DFS or BFS), and mark each cell visited (sink it to `'0'`) the moment it's discovered, not after exploring its neighbors. This ordering is the visited invariant: a cell is marked before it can be reached a second time, so it's never pushed onto the stack/queue twice.
4. Resume the outer scan; already-sunk cells simply read as `'0'` and are skipped for free.
5. Return the total island count once the scan reaches the end of the grid.

BFS vs. DFS. Either works — this problem only needs *reachability*, not shortest distance, so there's no per-cell "distance from source" to track. DFS (recursion) is shorter to write but can overflow the call stack on one giant island (up to 300 x 300 = 90,000 cells); BFS trades that for an explicit queue, which is safer at scale.

Worked trace on the example grid: - Scan hits `(0,0) = '1'` → new island, count = 1 → sink `(0,0)`, `(0,1)`,`(1,0)`,`(1,1)` to `'0'`. - Scan resumes, skips sunk cells, hits `(2,2) = '1'` → count = 2 → sink just `(2,2)` (no land neighbors). - Scan resumes, hits `(3,3) = '1'` → count = 3 → sink `(3,3)` and `(3,4)`. - Scan finishes → return `3`.

Approach (DFS)

For each unvisited land cell, sink the whole connected blob before moving on.

```
public int numIslands(char[][] grid) {
    int rows = grid.length, cols = grid[0].length;
    int islands = 0;
    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            if (grid[r][c] == '1') {
                islands++;
                sink(grid, r, c);          // mark this whole island visited
            }
        }
    }
    return islands;
}

private void sink(char[][] grid, int r, int c) {
    int rows = grid.length, cols = grid[0].length;
    if (r < 0 || r >= rows || c < 0 || c >= cols || grid[r][c] != '1') {
        return;                            // out of bounds or already water/visited
    }
    grid[r][c] = '0';                      // mark visited by sinking it
    sink(grid, r + 1, c);
    sink(grid, r - 1, c);
    sink(grid, r, c + 1);
    sink(grid, r, c - 1);
}
```

Approach (BFS)

Same idea, but flood-fill with a queue instead of recursion (avoids stack overflow on huge grids).

```
public int numIslandsBfs(char[][] grid) {
    int rows = grid.length, cols = grid[0].length;
    int islands = 0;
    int[][] dirs = { { 1, 0 }, { -1, 0 }, { 0, 1 }, { 0, -1 } };
    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            if (grid[r][c] != '1') continue;
            islands++;
            Queue<int[]> queue = new LinkedList<>();
            queue.add(new int[] { r, c });
            grid[r][c] = '0';
            while (!queue.isEmpty()) {
                int[] cell = queue.poll();
                for (int[] d : dirs) {
                    int nr = cell[0] + d[0], nc = cell[1] + d[1];
                    if (nr >= 0 && nr < rows && nc >= 0 && nc < cols && grid[nr][nc] == '1'
                        grid[nr][nc] = '0';           // mark visited before enqueueing
                        queue.add(new int[] { nr, nc });
                    }
                }
            }
        }
    }
    return islands;
}
```

Version	Time	Space
DFS flood fill	O(rows × cols)	O(rows × cols) worst-case recursion stack
BFS flood fill	O(rows × cols)	O(min(rows, cols)) queue, plus grid mutation

---

7.2 Clone Graph

Problem. Given a reference to a node in a **connected** undirected graph, return a **deep copy** of the graph — a brand-new set of `Node` objects with the same values and the same edges, sharing no references with the original. Each `Node` has an integer value and a list of neighbor references.

Constraints - `1 <= number of nodes <= 100`, node values are unique integers. - The graph has no repeated edges and no self-loops, but it can contain cycles (it's a general undirected graph, not a tree). - If the input node is `null`, return `null`.

Clarifying assumptions - Confirm the graph is connected (every node reachable from the given start node) — the problem guarantees this, so one traversal from `node` reaches everything. - Assume "deep copy" means new `Node` objects and new neighbor lists; the `int val` fields themselves are copied by value, not referenced.

Why it's tricky: the graph can have cycles, so a naive recursive copy that doesn't remember what it's already cloned will recurse forever bouncing back and forth across an edge.

```
class Node {
    public int val;
    public List<Node> neighbors;

    public Node(int val) {
        this.val = val;
        this.neighbors = new ArrayList<>();
    }
}
```

Example.

```
Input:  adjList = [[2,4],[1,3],[2,4],[1,3]]   (node 1 -> 2,4 ; node 2 -> 1,3 ; ...)
Output: a fully separate graph with the same shape and values
```

Intuition (diagram). Walk the original graph (DFS or BFS). The first time you see a node, create its clone and store the mapping `original -> clone` in a `HashMap`. When you later hit an edge to a node you've already cloned, just reuse the map — this is what stops infinite loops on cycles and what lets you wire up neighbor lists correctly.

```
original graph:        clone graph:          map (old -> new):
  1 -- 2                 1'-- 2'              1 -> 1'
  |    |                 |    |               2 -> 2'
  4 -- 3                 4'-- 3'              3 -> 3'
                                               4 -> 4'
                        (map entry created the first time
                         DFS/BFS visits each original node)
```

Approach (step by step)

Key idea / invariant. A `HashMap<Node, Node>` maps every original node to its clone, and a node is added to the map at the moment it is first discovered — *before* its neighbors are processed. That means: whenever the traversal reaches an edge back to a node already in the map (a cycle, or just a node reached via a second path), it looks up the existing clone instead of creating a new one and instead of recursing/enqueueing it again. This is exactly what turns an unbounded graph walk into one that visits each node exactly once.

1. If `node` is `null`, return `null` immediately.
2. Create the clone of the start node and put `node -> clone` in the map before doing anything else, so a cycle back to the start is already handled.
3. Traverse (DFS: recurse into each neighbor; BFS: enqueue each neighbor) — for every neighbor:
4. if it's **already in the map**, reuse its clone (this is the invariant firing).
5. if it's **not in the map yet**, create its clone, record the mapping, and continue the traversal from it (recurse, or enqueue for later).
6. Wire up `clone.neighbors` by appending the clone of every original neighbor.
7. Once the traversal is exhausted (recursion returns / queue empties), the map contains a full clone for every reachable node with all neighbor lists filled in; return the clone of the start node.

BFS vs. DFS. Both visit each node once thanks to the map, so either is correct; DFS is slightly more natural here because "clone this node, then its neighbors" mirrors the recursive structure, while BFS avoids recursion depth concerns on a graph with 100 nodes in a long chain.

Worked trace on the diagram above, DFS from node `1`: - Visit `1`: not in map → create `1'`, map `{1:1'}`. Recurse into neighbors `2`, `4`. - Visit `2`: not in map → create `2'`, map `{1:1', 2:2'}`. Recurse into neighbors `1`, `3`. - Neighbor `1` is already in the map → reuse `1'` (this closes the cycle, no infinite loop). - Visit `3`: not in map → create `3'`, map adds `3:3'`. Recurse into `2`, `4` — `2` reused. - Visit `4`: not in map → create `4'`, map adds `4:4'`. Recurse into `1`, `3` — both reused. - All neighbor lists wired; return `1'`, whose neighbors are `[2', 4']`.

Approach (DFS)

Recurse, cloning a node the first time it's seen and reusing the map otherwise.

```
public Node cloneGraph(Node node) {
    if (node == null) return null;
    return dfs(node, new HashMap<>());
}

private Node dfs(Node node, Map<Node, Node> visited) {
    if (visited.containsKey(node)) {
        return visited.get(node);          // already cloned, reuse it (breaks cycles)
    }
    Node clone = new Node(node.val);
    visited.put(node, clone);              // map BEFORE recursing into neighbors
    for (Node neighbor : node.neighbors) {
        clone.neighbors.add(dfs(neighbor, visited));
    }
    return clone;
}
```

Approach (BFS)

Same map trick, iterative with a queue.

```
public Node cloneGraphBfs(Node node) {
    if (node == null) return null;
    Map<Node, Node> visited = new HashMap<>();
    visited.put(node, new Node(node.val));
    Queue<Node> queue = new LinkedList<>();
    queue.add(node);
    while (!queue.isEmpty()) {
        Node curr = queue.poll();
        for (Node neighbor : curr.neighbors) {
            if (!visited.containsKey(neighbor)) {
                visited.put(neighbor, new Node(neighbor.val));
                queue.add(neighbor);
            }
            visited.get(curr).neighbors.add(visited.get(neighbor));
        }
    }
    return visited.get(node);
}
```

Version	Time	Space
DFS with map	O(V + E)	O(V) for the map and recursion stack
BFS with map	O(V + E)	O(V) for the map and queue

7.3 Course Schedule

Problem. There are `numCourses` courses labeled `0` to `numCourses - 1`. Some courses have prerequisites, given as pairs `[a, b]` meaning "to take `a` you must first take `b`". Given `numCourses` and the list `prerequisites`, return `true` if it's possible to take every course (equivalently: the prerequisite graph has no cycle), or `false` otherwise.

Constraints - `1 <= numCourses <= 2000`, `0 <= prerequisites.length <= 5000`. - `prerequisites[i] = [a, b]` with `0 <= a, b < numCourses` and `a != b`. - There may be duplicate prerequisite pairs; treat them as the same edge.

Clarifying assumptions - Confirm the relation direction: `[a, b]` means `b` is a prerequisite of `a` (edge `b -> a`), not the reverse — this is easy to flip by accident. - Assume you only need a yes/no answer here, not the actual course order (that's the "Course Schedule II" follow-up, which this same algorithm produces as a side effect).

Why it's tricky: it's really "does this directed graph have a cycle?" in disguise, and the two standard tools for that — Kahn's BFS topological sort (indegree counting) and DFS with a 3-color visited state — look completely different but rely on the same underlying idea.

Example.

```
Input:  numCourses = 2, prerequisites = [[1, 0]]
Output: true            (take 0, then 1)

Input:  numCourses = 2, prerequisites = [[1, 0], [0, 1]]
Output: false           (0 needs 1, and 1 needs 0 -> cycle, impossible)
```

Intuition (diagram). This is cycle detection in a **directed** graph. A **topological sort** is just "process nodes in an order where every prerequisite comes before what depends on it." Kahn's algorithm does this with BFS: repeatedly take a node whose prerequisites are all satisfied (**indegree 0**), "remove" it, and see if that frees up new nodes. If you can process every node this way, there's no cycle; if some nodes are never freed up, they're stuck in a cycle.

```
courses graph:   0 -> 1 -> 2

indegree table:  course:    0   1   2
                 indegree:  0   1   1

step 1: indegree-0 courses are ready now -> queue = [0]
step 2: process 0, decrement 1's indegree (1 -> 0) -> queue = [1]
step 3: process 1, decrement 2's indegree (1 -> 0) -> queue = [2]
step 4: process 2, queue empty, processed = 3 of 3 -> no cycle -> true
```

Approach (step by step)

Key idea / invariant. A course is only safe to "take" once every prerequisite edge pointing into it has been satisfied — tracked as `indegree[course]` reaching `0`. The queue invariant is: at any point, the queue holds exactly the courses whose indegree is currently `0` and that haven't been processed yet. Processing a course means removing it from the graph (conceptually) and decrementing the indegree of everything it points to, which may free up new courses. If this process can eventually remove every node, the graph is a DAG (no cycle); if some nodes' indegree never reaches `0`, they — and whatever depends on them — are stuck inside a cycle and never get dequeued.

1. Build an adjacency list `needs -> [courses that depend on needs]` and an `indegree[]` array counting, for each course, how many prerequisites it still has outstanding.
2. Seed the queue with every course whose `indegree == 0` (no prerequisites at all — ready immediately).
3. Repeatedly poll a course from the queue, count it as processed, and for each course it unlocks, decrement that course's indegree; if that indegree hits `0`, enqueue it (it just became ready).
4. Stop when the queue is empty.
5. Compare `processed` to `numCourses`. If they're equal, every course was eventually freed up → no cycle → `true`. If `processed < numCourses`, some courses were never unlocked because they (or a course they depend on) sit inside a cycle → `false`.

BFS vs. DFS. Kahn's algorithm (BFS) processes nodes in true topological order and is easy to reason about via indegree counts. The DFS alternative below detects a cycle by coloring nodes as it walks down the current recursion path: reaching a node still marked "in progress" (state `1`) means the path has looped back on itself — a cycle. Both are `O(V + E)`; BFS is usually preferred when you also want the actual course order as output.

Worked trace on `numCourses = 2, prerequisites = [[1, 0], [0, 1]]` (the cyclic example): - Graph: edge `0 -> 1` (from `[1,0]`) and edge `1 -> 0` (from `[0,1]`). `indegree = [1, 1]`. - No course has indegree `0` → queue starts **empty**. - Loop body never runs → `processed = 0`. - `processed (0) != numCourses (2)` → return `false`. The indegree of both courses never drops to `0` because each is waiting on the other — exactly the stuck-in-a-cycle case.

Approach (BFS — Kahn's topological sort)

Build the graph and indegree counts, then repeatedly peel off indegree-0 nodes.

```
public boolean canFinish(int numCourses, int[][] prerequisites) {
    List<List<Integer>> graph = new ArrayList<>();
    for (int i = 0; i < numCourses; i++) {
        graph.add(new ArrayList<>());
    }
    int[] indegree = new int[numCourses];
    for (int[] pre : prerequisites) {
        int course = pre[0], needs = pre[1];
        graph.get(needs).add(course);      // needs -> course (course depends on needs)
        indegree[course]++;
    }

    Queue<Integer> queue = new LinkedList<>();
    for (int c = 0; c < numCourses; c++) {
        if (indegree[c] == 0) queue.add(c); // no prerequisites, ready now
    }

    int processed = 0;
    while (!queue.isEmpty()) {
        int course = queue.poll();
        processed++;
        for (int next : graph.get(course)) {
            indegree[next]--;               // one prerequisite satisfied
            if (indegree[next] == 0) {
                queue.add(next);             // now ready
            }
        }
    }
    return processed == numCourses;         // if not all processed, a cycle blocked some
}
```

Approach (DFS — cycle detection with states)

Alternative: DFS each node tracking 0=unvisited, 1=in current path, 2=fully done. Hitting a node marked "1" means a cycle.

```
public boolean canFinishDfs(int numCourses, int[][] prerequisites) {
    List<List<Integer>> graph = new ArrayList<>();
    for (int i = 0; i < numCourses; i++) graph.add(new ArrayList<>());
    for (int[] pre : prerequisites) graph.get(pre[1]).add(pre[0]);

    int[] state = new int[numCourses];      // 0 = unvisited, 1 = visiting, 2 = done
    for (int c = 0; c < numCourses; c++) {
        if (state[c] == 0 && hasCycle(graph, state, c)) {
            return false;
        }
    }
    return true;
}

private boolean hasCycle(List<List<Integer>> graph, int[] state, int course) {
    state[course] = 1;                      // mark as "currently on the stack"
    for (int next : graph.get(course)) {
        if (state[next] == 1) return true;  // reached a node still on the stack -> cycle
        if (state[next] == 0 && hasCycle(graph, state, next)) return true;
    }
    state[course] = 2;                      // fully explored, safe
    return false;
}
```

Version	Time	Space
BFS (Kahn's topo sort)	O(V + E)	O(V + E)
DFS (3-color cycle check)	O(V + E)	O(V + E)

7.4 Rotting Oranges

Problem. A grid contains `0` (empty cell), `1` (fresh orange), or `2` (rotten orange). Every minute, each rotten orange rots any of its 4-directional neighbors that is fresh, and it does this simultaneously for all rotten oranges at once. Return the minimum number of minutes until no fresh orange remains, or `-1` if some fresh orange is unreachable and can never rot.

Constraints - `1 <= rows, cols <= 10`, each cell is `0`, `1`, or `2`. - There can be zero, one, or many rotten oranges to start, and zero or more fresh oranges.

Clarifying assumptions - Confirm rotting is simultaneous across all currently-rotten oranges each minute (not one orange rotting neighbors one at a time) — this is what makes it multi-source BFS rather than several independent single-source BFS runs. - If there are zero fresh oranges at the start, the answer is `0` minutes.

Why it's tricky: it looks like ordinary BFS, but there are many simultaneous sources, and "minutes elapsed" only maps correctly to "BFS depth" if you process the queue one full level ("wave") at a time rather than one cell at a time.

Example.

```
Input:
2 1 1
1 1 0
0 1 1
Output: 4
```

Intuition (diagram). All rotten oranges spread **simultaneously**, so this is a **multi-source BFS**: load every rotten cell into the queue as minute 0, then expand outward level by level. Each full "wave" of the queue represents exactly one minute passing.

```
minute 0:        minute 1:        minute 2:
2 1 1            2 2 1            2 2 2
1 1 0    -->     2 1 0    -->     2 2 0     -> keeps spreading until
0 1 1            0 1 1            0 2 1        no fresh (1) cell remains
```

Approach (step by step)

Key idea / invariant. The queue holds exactly the rotten cells whose neighbors haven't been processed yet, grouped by the minute they became rotten. Processing one full wave of the queue (all cells that were rotten *at the start of this minute*) and only then moving to the next minute is what keeps the BFS "layer number" equal to "elapsed minutes" — this is the same level-by-level BFS used for shortest-path-in-unweighted-graph problems, just seeded from many sources instead of one.

1. Scan the grid once: push every already-rotten cell into the queue (these are all minute-0 sources), and count the total number of fresh oranges.
2. If there are no fresh oranges at all, return `0` immediately — nothing needs to rot.
3. Loop while the queue is non-empty **and** fresh oranges remain: capture `size = queue.size()` (the current wave), then pop exactly `size` cells. For each, rot any fresh 4-directional neighbor — flip it to `2`, decrement the fresh counter, and enqueue it (it becomes part of the *next* wave).
4. After finishing that wave, increment `minutes` by 1 — one full wave popped is one minute elapsed.
5. When the loop ends, either `fresh == 0` (everything rotted — return `minutes`) or the queue ran dry with fresh oranges still left (they're unreachable — return `-1`).

BFS vs. DFS. This must be BFS, not DFS: the question asks for the minimum time for the rot to *finish spreading everywhere*, which is a shortest-distance-from-multiple-sources question. DFS would explore one branch to completion before backtracking and gives no natural way to read off "which minute" a cell rotted on.

Worked trace on the example (`fresh` starts at 6): - Wave 0 (queue = `[(0,0)]`, size 1): rot `(0,1)` and `(1,0)` → fresh = 4. `minutes = 1`. - Wave 1 (queue = `[(0,1),(1,0)]`, size 2): from `(0,1)` rot `(0,2)`; from `(1,0)` rot `(1,1)` → fresh = 2. `minutes = 2`. - Wave 2 (queue = `[(0,2),(1,1)]`, size 2): from `(1,1)` rot `(2,1)`; `(0,2)` has no fresh neighbor → fresh = 1. `minutes = 3`. - Wave 3 (queue = `[(2,1)]`, size 1): rot `(2,2)` → fresh = 0. `minutes = 4`. - Queue still has `(2,2)` but `fresh == 0` stops the loop → return `4`.

Approach (multi-source BFS)

Seed the queue with all rotten oranges, then BFS level by level counting minutes.

```
public int orangesRotting(int[][] grid) {
    int rows = grid.length, cols = grid[0].length;
    Queue<int[]> queue = new LinkedList<>();
    int fresh = 0;

    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            if (grid[r][c] == 2) {
                queue.add(new int[] { r, c });   // every rotten orange starts at minute 0
            } else if (grid[r][c] == 1) {
                fresh++;
            }
        }
    }

    int[][] dirs = { { 1, 0 }, { -1, 0 }, { 0, 1 }, { 0, -1 } };
    int minutes = 0;
    while (!queue.isEmpty() && fresh > 0) {
        int size = queue.size();               // process one full "wave" = one minute
        for (int i = 0; i < size; i++) {
            int[] cell = queue.poll();
            for (int[] d : dirs) {
                int nr = cell[0] + d[0], nc = cell[1] + d[1];
                if (nr >= 0 && nr < rows && nc >= 0 && nc < cols && grid[nr][nc] == 1) {
                    grid[nr][nc] = 2;           // rot it
                    fresh--;
                    queue.add(new int[] { nr, nc });
                }
            }
        }
        minutes++;
    }
    return fresh == 0 ? minutes : -1;           // fresh left over -> unreachable
}
```

Version	Time	Space
Multi-source BFS	O(rows × cols)	O(rows × cols) queue worst case

---

7.5 Flood Fill

Problem. Given an image as an `m x n` grid of integer colors, a starting pixel `(sr, sc)`, and a `newColor`, repaint the starting pixel and every pixel reachable from it by 4-directional moves through pixels that share the **starting pixel's original color**. Return the modified image.

Constraints - `1 <= m, n <= 50`, `0 <= image[i][j], newColor < 2^16`. - `0 <= sr < m`, `0 <= sc < n` (the start pixel is always in bounds).

Clarifying assumptions - Confirm connectivity is 4-directional, matching Number of Islands / Rotting Oranges. - Ask what happens if `newColor` equals the starting color already — the expected behavior is a no-op (return the image unchanged), not an infinite repaint loop.

Why it's tricky: the classic trap is forgetting the `newColor == startColor` guard — if you repaint in place without it, every pixel already matches the "new" color after the first repaint, so the flood fill never terminates.

Example.

```
Input:  image = [[1,1,1],[1,1,0],[1,0,1]], sr = 1, sc = 1, newColor = 2
Output:         [[2,2,2],[2,2,0],[2,0,1]]
```

Intuition (diagram). Remember the original color at the start, then DFS/BFS outward, repainting any neighbor that still has that original color. Stop at boundaries or pixels that don't match — those are the edges of the region.

```
before:                 after:                  start = (1,1), color 1 -> 2
row 0:  1  1  1         row 0:  2  2  2
row 1:  1  1  0         row 1:  2  2  0         the 0's at (1,2) and (2,1)
row 2:  1  0  1         row 2:  2  0  1         block the spread; (2,2) sits
                                                  past both blockers, so it
                                                  stays unreached
```

Approach (step by step)

Key idea / invariant. Save `startColor` once, up front. From then on, the invariant is: a pixel is repainted (and only repainted) if it currently still equals `startColor` — this single check does double duty as both the "is this part of the region" test and the "have I already visited this pixel" test, because once a pixel is repainted it no longer equals `startColor` and will fail the check if reached again.

1. Read `startColor = image[sr][sc]`.
2. Guard: if `startColor == newColor`, return the image as-is — repainting would be a no-op, and without this check the flood fill below never terminates (every repainted neighbor would still "match" and get re-enqueued forever).
3. Repaint `(sr, sc)` to `newColor` immediately (DFS: right before recursing; BFS: right before enqueueing) — this is what marks it visited.
4. Explore its 4-directional neighbors (DFS: recurse; BFS: enqueue). For each neighbor still in bounds and still equal to `startColor`, repaint it and continue exploring from it.
5. Stop expanding at any neighbor that's out of bounds or doesn't equal `startColor` — that's either the image edge or the boundary of the region.
6. Return the image once every reachable same-colored pixel has been repainted.

BFS vs. DFS. Both are correct and the choice barely matters at this problem's scale (`m, n <= 50`, at most 2500 pixels); DFS reads slightly more naturally as "spread out and repaint," while BFS is the safer default habit for very large images to avoid deep recursion.

Worked trace on the example, DFS from `(1,1)`, `startColor = 1`, `newColor = 2`: - `(1,1)`: is `1` → repaint to `2`. Recurse into `(2,1)`, `(0,1)`, `(1,2)`, `(1,0)`. - `(2,1)`: is `0`, not `1` → stop (region boundary). - `(0,1)`: is `1` → repaint to `2`. Recurse into neighbors: `(0,0)` and `(0,2)` are `1`, repaint both; `(1,1)` is already `2` → stop. - `(1,2)`: is `0` → stop. - `(1,0)`: is `1` → repaint to `2`. Recurse into `(2,0)`: it's `1` and reachable via `(1,0)`, so it gets repainted to `2` as well; its only other neighbor, `(2,1)`, is `0` → stop there. - Final repainted set: `(1,1),(0,1),(0,0),(0,2),(1,0),(2,0)` all become `2`; `(1,2)` and `(2,1)` stay `0`; `(2,2)` stays `1` since it's not 4-connected to the repainted region — matching the expected output.

Approach (DFS)

Guard against repainting when `newColor == startColor` (would infinite-loop otherwise).

```
public int[][] floodFill(int[][] image, int sr, int sc, int newColor) {
    int startColor = image[sr][sc];
    if (startColor != newColor) {          // avoid infinite recursion on a no-op fill
        fill(image, sr, sc, startColor, newColor);
    }
    return image;
}

private void fill(int[][] image, int r, int c, int startColor, int newColor) {
    int rows = image.length, cols = image[0].length;
    if (r < 0 || r >= rows || c < 0 || c >= cols || image[r][c] != startColor) {
        return;                            // out of bounds or not part of this region
    }
    image[r][c] = newColor;                // repaint, which also marks it "visited"
    fill(image, r + 1, c, startColor, newColor);
    fill(image, r - 1, c, startColor, newColor);
    fill(image, r, c + 1, startColor, newColor);
    fill(image, r, c - 1, startColor, newColor);
}
```

Approach (BFS)

Same guard, iterative with a queue — better for very large regions.

```
public int[][] floodFillBfs(int[][] image, int sr, int sc, int newColor) {
    int startColor = image[sr][sc];
    if (startColor == newColor) return image;

    int rows = image.length, cols = image[0].length;
    int[][] dirs = { { 1, 0 }, { -1, 0 }, { 0, 1 }, { 0, -1 } };
    Queue<int[]> queue = new LinkedList<>();
    queue.add(new int[] { sr, sc });
    image[sr][sc] = newColor;

    while (!queue.isEmpty()) {
        int[] cell = queue.poll();
        for (int[] d : dirs) {
            int nr = cell[0] + d[0], nc = cell[1] + d[1];
            if (nr >= 0 && nr < rows && nc >= 0 && nc < cols && image[nr][nc] == startColor
                image[nr][nc] = newColor;
                queue.add(new int[] { nr, nc });
            }
        }
    }
    return image;
}
```

Version	Time	Space
DFS flood fill	O(rows × cols)	O(rows × cols) worst-case recursion stack
BFS flood fill	O(rows × cols)	O(rows × cols) worst-case queue

8. Dynamic Programming

The core idea. Dynamic programming solves a problem by breaking it into smaller subproblems that **overlap** — the same subproblem gets asked again and again if you just use plain recursion. Instead of recomputing an answer you've already found, you save it and reuse it. There are two ways to organize that saving: **memoization (top-down)** starts from the original question, recurses naturally, and caches each subproblem's answer the first time it's computed (usually in a map or array indexed by the subproblem's parameters). **Tabulation (bottom-up)** flips the direction — you start from the smallest subproblems, fill in a table iteratively, and build up to the answer you actually want, so nothing ever needs to be revisited or looked up twice.

Reach for it when: the problem asks for an optimal value (min/max/count) or a yes/no feasibility answer, a brute-force recursion **recomputes the same subproblem** many times, and the problem can be described by a **recurrence relation** — the answer to a bigger instance is built from the answers to smaller instances.

8.1 Climbing Stairs (LC 70)

Problem. You are climbing a staircase that has `n` steps. From any position you can advance either 1 or 2 steps at a time. Return the number of **distinct sequences of moves** that reach exactly step `n` (order matters — `1 then 2` and `2 then 1` are counted as two different ways).

- **Input:** a single integer `n`.
- **Output:** an integer — the count of distinct move-sequences that land on step `n`.
- **Constraints:** `1 <= n <= 45` (the standard LeetCode bound; the answer fits in a 32-bit int at this size).
- **Clarifying assumptions:** confirm each move is exactly 1 or 2 steps (never 0, never more than 2), and that reaching `n = 0` counts as exactly one way — "take no steps."

Why it's tricky: it reads like a combinatorics/counting question, but the number of ways to land on step `i` depends on nothing except the number of ways to land on the two steps right before it — spotting that hidden Fibonacci-style recurrence is the entire problem.

Example.

```
Input:  n = 4
Output: 5
Ways: 1+1+1+1, 1+1+2, 1+2+1, 2+1+1, 2+2
```

Intuition (diagram). To reach step `i`, your last move was either a 1-step from `i−1` or a 2-step from `i−2`, so the ways to reach `i` are the ways to reach `i−1` plus the ways to reach `i−2`.

```
index:      0    1    2    3    4
dp[i]:      1    1    2    3    5
```

Approach (step by step)

- **Key idea:** `dp[i]` = number of distinct move-sequences that land exactly on step `i`. Because the only two moves are 1 and 2 steps, every sequence reaching `i` arrives either from `i−1` (via a final 1-step) or from `i−2` (via a final 2-step), and those two cases never overlap — so they simply add.
- **Recurrence in words:** "the ways to reach step `i` equal the ways to reach step `i−1` (then take one more single step) plus the ways to reach step `i−2` (then take one more double step)."
- **What `dp[i]` means:** the count of distinct 1-or-2-step sequences whose moves sum to exactly `i`.
- **Base case:** `dp[0] = 1` (one way to already be at the ground — take zero steps) and `dp[1] = 1` (only one way: a single 1-step).
- **Fill order:** left to right, `i = 2 .. n`, since `dp[i]` needs `dp[i−1]` and `dp[i−2]`, both of which are already filled by the time you reach `i`.
- **Worked trace for `n = 4`:**
- `dp[0] = 1`, `dp[1] = 1` (base cases).
- `dp[2] = dp[1] + dp[0] = 1 + 1 = 2`.
- `dp[3] = dp[2] + dp[1] = 2 + 1 = 3`.
- `dp[4] = dp[3] + dp[2] = 3 + 2 = 5` — this is the answer.

Brute force

Plain recursion — recomputes the same step count over and over (exponential blowup).

```
public int climbStairsBrute(int n) {
    if (n <= 1) return 1;                       // 0 or 1 step: exactly 1 way
    return climbStairsBrute(n - 1) + climbStairsBrute(n - 2);
}
```

Optimized (O(1) space, bottom-up)

Recurrence: `ways(n) = ways(n−1) + ways(n−2)`. We only ever need the last two values, so skip the array entirely and roll two variables forward.

```
public int climbStairs(int n) {
    if (n <= 1) return 1;
    int prev2 = 1, prev1 = 1;                    // ways(0), ways(1)
    for (int i = 2; i <= n; i++) {
        int current = prev1 + prev2;             // combine the two smaller subproblems
        prev2 = prev1;
        prev1 = current;
    }
    return prev1;
}
```

Version	Time	Space
Brute force	O(2^n)	O(n) call stack
Bottom-up, O(1) space	O(n)	O(1)

8.2 House Robber (LC 198)

Problem. You are given an array `nums` where `nums[i]` is the amount of money stashed in house `i`, all houses arranged in a single row. Return the **maximum total money** you can rob such that no two robbed houses are **adjacent** (robbing adjacent houses trips the alarm).

- **Input:** an integer array `nums`.
- **Output:** an integer — the maximum sum achievable with no two chosen indices adjacent.
- **Constraints:** `1 <= nums.length <= 100`, `0 <= nums[i] <= 400` (standard LeetCode bounds; values are non-negative).
- **Clarifying assumptions:** confirm houses are in a fixed line (not a circle — that's a separate follow-up variant), values can be `0` (an empty house), and you may choose to rob zero houses if that somehow maximizes the total (it never does when all values are non-negative, but it's worth stating).

Why it's tricky: the greedy instinct ("always take the bigger house") fails — taking a big house early can block two smaller-but-combined-larger houses later. You need to compare, at every house, the best total *with* it against the best total *without* it.

Example.

```
Input:  [2, 7, 9, 3, 1]
Output: 12          (rob houses 0, 2, 4: 2 + 9 + 1 = 12)
```

Intuition (diagram). At each house `i` you make a choice: **skip** it (best you can do is whatever you already had at `i-1`), or **take** it (its value plus the best you could do at `i-2`, since `i-1` must then be skipped). Keep the better of the two at every step.

```
index:      0    1    2    3    4
houses:     2    7    9    3    1
dp[i]:      2    7   11   11   12
```

Approach (step by step)

- **Key idea:** `dp[i]` = the maximum money obtainable from houses `0..i` (inclusive), given the no-adjacent-houses rule. At every house you have exactly two options, and `dp[i]` is the better of the two.
- **Recurrence in words:** "the best total through house `i` is either the best total through house `i-1` (skip house `i` entirely), or house `i`'s value plus the best total through house `i-2` (take house `i`, which forces skipping `i-1`) — whichever is larger."
- **What `dp[i]` means:** the maximum robbery total considering only houses `0` through `i`.
- **Base case:** `dp[0] = nums[0]` (only one house available, so take it); conceptually `dp[-1] = 0` (no houses, no money) for the "take" branch when `i = 1`.
- **Fill order:** left to right, `i = 1 .. n-1`, since `dp[i]` needs `dp[i-1]` and `dp[i-2]`.
- **Worked trace for `[2, 7, 9, 3, 1]`:**
- `dp[0] = 2` (base case).
- `dp[1] = max(dp[0], nums[1]) = max(2, 7) = 7` (skip 0 and take 1 outright).
- `dp[2] = max(dp[1], nums[2] + dp[0]) = max(7, 9 + 2) = 11`.

- `dp[3] = max(dp[2], nums[3] + dp[1]) = max(11, 3 + 7) = 11`.
- `dp[4] = max(dp[3], nums[4] + dp[2]) = max(11, 1 + 11) = 12` — this is the answer.

Brute force

Recurse on "take this house" vs "skip it", recomputing overlapping suffixes.

```
public int robBrute(int[] nums, int i) {
    if (i >= nums.length) return 0;
    int take = nums[i] + robBrute(nums, i + 2);  // rob i, must skip i+1
    int skip = robBrute(nums, i + 1);             // leave i alone
    return Math.max(take, skip);
}

public int robBrute(int[] nums) {
    return robBrute(nums, 0);
}
```

Optimized (bottom-up, O(1) space)

Recurrence: `dp[i] = max(dp[i-1], dp[i-2] + nums[i])`. Track only the last two best totals.

```
public int rob(int[] nums) {
    int prev2 = 0, prev1 = 0;                    // best up through i-2, i-1
    for (int num : nums) {
        int current = Math.max(prev1, prev2 + num);
        prev2 = prev1;
        prev1 = current;
    }
    return prev1;
}
```

Version	Time	Space
Brute force	O(2^n)	O(n) call stack
Bottom-up, O(1) space	O(n)	O(1)

8.3 Coin Change (LC 322)

Problem. You are given an array `coins` of distinct coin denominations and an integer `amount`. You have an **unlimited supply** of each coin. Return the **fewest number of coins** needed to make up exactly `amount`, or `-1` if no combination of coins sums to it.

- **Input:** an integer array `coins`, an integer `amount`.
- **Output:** an integer — the minimum coin count, or `-1` if `amount` is unreachable.
- **Constraints:** `1 <= coins.length <= 12`, `1 <= coins[i] <= 2^31 - 1`, `0 <= amount <= 10^4` (standard LeetCode bounds).
- **Clarifying assumptions:** confirm the supply of each coin is unlimited (this is *not* the "each coin usable once" variant), coin values are positive, and `amount = 0` returns `0` (no coins needed).

Why it's tricky: greedily grabbing the largest coin first does not always minimize the count (e.g. coins `[1, 3, 4]`, amount `6`: greedy picks `4 + 1 + 1` = 3 coins, but `3 + 3` = 2 coins is better). You must consider every coin choice at every amount, which is exactly what makes it a DP over the amount.

Example.

```
Input:  coins = [1, 2, 5], amount = 11
Output: 3            (5 + 5 + 1)
```

Intuition (diagram). For each amount `a`, try every coin `c`: if you use one coin `c`, the rest of the amount is `a - c`, which is a smaller subproblem you've already solved. Take the coin choice that minimizes `1 + dp[a - c]`.

```
amount:      0    1    2    3    4    5    6    7    8    9   10   11
dp[a]:       0    1    1    2    2    1    2    2    3    3    2    3
```

Approach (step by step)

- **Key idea:** `dp[a]` = the fewest coins needed to make exactly amount `a`. For every coin `c <= a`, using one of that coin leaves the smaller subproblem `a - c`, so `dp[a]` is `1` plus the best (smallest) `dp[a - c]` over all usable coins.
- **Recurrence in words:** "the fewest coins to make amount `a` is one more than the fewest coins to make `a - c`, minimized over every coin `c` that is no larger than `a`."
- **What `dp[a]` means:** the minimum number of coins that sum to exactly `a` (an unreachable-so-far sentinel like `amount + 1` stands in for "impossible").
- **Base case:** `dp[0] = 0` — zero coins are needed to make amount `0`.
- **Fill order:** left to right over amounts, `a = 1 .. amount`, since `dp[a]` only depends on `dp[a - c]` for `c <= a`, i.e. strictly smaller amounts that are already filled.
- **Worked trace for `coins = [1, 2, 5]`, up through `a = 6`:**
- `dp[0] = 0` (base case).
- `dp[1] = 1 + dp[0] = 1` (use one 1-coin).
- `dp[2] = 1 + dp[0] = 1` (use one 2-coin — better than two 1-coins).
- `dp[3] = 1 + min(dp[2], dp[1]) = 1 + 1 = 2` (a 1 + a 2).
- `dp[4] = 1 + min(dp[3], dp[2]) = 1 + 1 = 2` (two 2-coins).
- `dp[5] = 1 + dp[0] = 1` (one 5-coin beats any smaller combination).
- `dp[6] = 1 + min(dp[5], dp[4], dp[1]) = 1 + 1 = 2` (a 5 + a 1).
- ... continuing the same pattern up to `dp[11] = 3` (5 + 5 + 1), the final answer.

Brute force

Try every coin at every remaining amount — massive overlap between subtrees.

```
public int coinChangeBrute(int[] coins, int amount) {
    if (amount == 0) return 0;
    if (amount < 0) return Integer.MAX_VALUE;    // invalid path
    int best = Integer.MAX_VALUE;
    for (int coin : coins) {
        int sub = coinChangeBrute(coins, amount - coin);
        if (sub != Integer.MAX_VALUE) {
            best = Math.min(best, sub + 1);
        }
    }
    return best;
}
```

Optimized (bottom-up dp array)

Recurrence: `dp[a] = 1 + min(dp[a - coin])` over every coin that fits, with `dp[0] = 0`.

```
public int coinChange(int[] coins, int amount) {
    int[] dp = new int[amount + 1];
    Arrays.fill(dp, amount + 1);                 // amount+1 acts as "infinity"
    dp[0] = 0;                                    // 0 coins needed to make 0
    for (int a = 1; a <= amount; a++) {
        for (int coin : coins) {
            if (coin <= a) {
                dp[a] = Math.min(dp[a], dp[a - coin] + 1);
            }
        }
    }
    return dp[amount] > amount ? -1 : dp[amount];
}
```

Version	Time	Space
Brute force	O(coins^amount)	O(amount) call stack
Bottom-up dp array	O(amount * coins)	O(amount)

8.4 Longest Increasing Subsequence (LC 300)

Problem. Given an integer array `nums`, return the length of the **longest strictly increasing subsequence** — a subsequence keeps the original relative order but elements do not need to be contiguous, and each next element must be strictly greater than the previous.

- **Input:** an integer array `nums`.
- **Output:** an integer — the length of the longest strictly increasing subsequence.
- **Constraints:** `1 <= nums.length <= 2500`, `-10^4 <= nums[i] <= 10^4` (standard LeetCode bounds).
- **Clarifying assumptions:** confirm "increasing" means **strictly** increasing (equal values do not extend the subsequence), duplicates may appear in `nums`, and you only need to return the length, not the subsequence itself.

Why it's tricky: the number of subsequences is exponential, and it's tempting to think you can track just "the best subsequence so far," but the best subsequence *ending at each index* is what matters — a later, smaller value might still extend into a longer chain than whatever was globally best up to that point.

Example.

```
Input:  [10, 9, 2, 5, 3, 7, 101, 18]
Output: 4            ([2, 3, 7, 101] or [2, 3, 7, 18])
```

Intuition (diagram). `dp[i]` is the length of the longest increasing subsequence that *ends* at index `i`. To compute it, look back at every earlier index `j` with a smaller value, and extend that subsequence: `dp[i] = 1 + max(dp[j])` over all valid `j < i`.

```
index:       0    1    2    3    4    5    6    7
nums[i]:    10    9    2    5    3    7  101   18
dp[i]:       1    1    1    2    2    3    4    4
```

Approach (step by step)

- **Key idea:** `dp[i]` = the length of the longest strictly increasing subsequence that ends exactly at index `i` (using `nums[i]` as the last element). The overall answer is the maximum value anywhere in the `dp` array, since the longest subsequence overall must end *somewhere*.
- **Recurrence in words:** "the longest increasing subsequence ending at `i` is one longer than the best subsequence ending at any earlier index `j` whose value is smaller than `nums[i]` — or just `1` (itself alone) if no earlier index qualifies."
- **What `dp[i]` means:** the length of the best strictly-increasing run of elements that finishes at position `i`.
- **Base case:** every `dp[i]` starts at `1`, because any single element is trivially an increasing subsequence of length 1.
- **Fill order:** left to right, `i = 1 .. n−1`; for each `i`, scan every `j < i` (all strictly earlier, already-finalized indices).
- **Worked trace for `[10, 9, 2, 5, 3, 7, 101, 18]`:**
- `dp[0] = 1` (base case, "10" alone).
- `dp[1] = 1` — `9 < 10`, no earlier smaller value extends it.
- `dp[2] = 1` — `2` is smaller than everything before it.
- `dp[3] = 1 + dp[2] = 2` — `5 > 2`, extend the subsequence ending at index 2.
- `dp[4] = 1 + dp[2] = 2` — `3 > 2`, same extension; `3` is not `> 5`, so index 3 doesn't help here.
- `dp[5] = 1 + max(dp[3], dp[4]) = 1 + 2 = 3` — `7` is greater than both `5` and `3`.
- `dp[6] = 1 + dp[5] = 4` — `101` extends the length-3 chain ending at index 5.
- `dp[7] = 1 + dp[5] = 4` — `18` also extends that same chain (a different, equally long subsequence).
- Answer = `max(dp) = 4`.

Brute force

Try every subsequence via recursion: at each index, either include it (if it extends the chain) or skip it — exponential number of choices.

```
public int lisBrute(int[] nums, int i, int prevIndex) {
    if (i == nums.length) return 0;
    int skip = lisBrute(nums, i + 1, prevIndex);              // don't take nums[i]
    int take = 0;
    if (prevIndex == -1 || nums[i] > nums[prevIndex]) {
        take = 1 + lisBrute(nums, i + 1, i);                  // take nums[i]
    }
    return Math.max(skip, take);
}

public int lengthOfLISBrute(int[] nums) {
    return lisBrute(nums, 0, -1);
}
```

Optimized (O(n^2) tabulation)

Recurrence: `dp[i] = 1 + max(dp[j])` for every `j < i` with `nums[j] < nums[i]`; each `dp[i]` starts at 1 (the element alone). The answer is the max over the whole table. (A faster O(n log n) patience-sorting approach with binary search also exists, but this quadratic version is the one to know cold.)

```
public int lengthOfLIS(int[] nums) {
    int n = nums.length;
    int[] dp = new int[n];
    Arrays.fill(dp, 1);                           // every element is a subsequence of lengt
    int best = 1;
    for (int i = 1; i < n; i++) {
        for (int j = 0; j < i; j++) {
            if (nums[j] < nums[i]) {
                dp[i] = Math.max(dp[i], dp[j] + 1);
            }
        }
        best = Math.max(best, dp[i]);
    }
    return best;
}
```

Version	Time	Space
Brute force	O(2^n)	O(n) call stack
O(n^2) tabulation	O(n^2)	O(n)

8.5 Word Break (LC 139)

Problem. Given a string `s` and a dictionary of strings `wordDict`, return `true` if `s` can be segmented into a space-separated sequence of one or more dictionary words (words may be reused any number of times).

- **Input:** a string `s`, a list of strings `wordDict`.
- **Output:** a boolean — whether `s` can be fully segmented using words from `wordDict`.
- **Constraints:** `1 <= s.length <= 300`, `1 <= wordDict.length <= 1000`, dictionary words are non-empty and made of lowercase letters (standard LeetCode bounds); `wordDict` may contain

duplicates, and the same word may be reused multiple times in the segmentation.

- **Clarifying assumptions:** confirm word reuse is allowed (it's an unlimited-supply problem, similar in spirit to Coin Change) and that matching is case-sensitive / exact.

Why it's tricky: the naive recursive approach re-examines the same trailing substrings over and over as it tries different split points, causing exponential blowup; the fix is to notice there are only `n + 1` distinct prefixes, so cache breakability per prefix instead of per substring choice.

Example.

```
Input:  s = "leetcode", wordDict = ["leet", "code"]
Output: true          ("leet" + "code")
```

Intuition (diagram). `dp[i]` means "the prefix `s[0..i)` can be fully broken into dictionary words." To fill `dp[i]`, look at every earlier split point `j`: if `dp[j]` is true and `s[j..i)` is a dictionary word, then `dp[i]` is true too.

```
char (s[i]):   l    e    e    t    c    o    d    e
index i:       0    1    2    3    4    5    6    7
```

```
prefix length: 0    1    2    3    4    5    6    7    8
dp[i]:         T    F    F    F    T    F    F    F    T
```

Approach (step by step)

- **Key idea:** `dp[i]` = "is the prefix consisting of the first `i` characters of `s` (i.e. `s[0..i)`) fully breakable into dictionary words?" `dp[n]` (the whole string) is the final answer.
- **Recurrence in words:** "the prefix of length `i` is breakable if there is some earlier split point `j` (with `0 <= j < i`) where the prefix of length `j` is already breakable *and* the substring from `j` to `i` is itself a dictionary word."
- **What `dp[i]` means:** whether `s[0..i)` can be fully partitioned into dictionary words, with no leftover characters.
- **Base case:** `dp[0] = true` — the empty prefix requires zero words, so it's trivially breakable.
- **Fill order:** left to right, `i = 1 .. n`; for each `i`, scan every split point `j = 0 .. i−1` (all previously-computed, smaller prefixes).
- **Worked trace for `s = "leetcode"`, `wordDict = ["leet", "code"]`:**
- `dp[0] = true` (base case).
- `dp[1..3]` (prefixes `"l"`, `"le"`, `"lee"`) all stay `false` — none is a dictionary word, and no valid split point makes them breakable.
- `dp[4] = true` — split at `j = 0`: `dp[0]` is true and `s[0..4) = "leet"` is in the dictionary.
- `dp[5..7]` (prefixes ending in `"leetc"`, `"leetco"`, `"leetcod"`) stay `false` — no split point produces both a true `dp[j]` and a dictionary word for the remainder.
- `dp[8] = true` — split at `j = 4`: `dp[4]` is true and `s[4..8) = "code"` is in the dictionary. `dp[8]` is the answer for the full string: `true`.

Brute force

Recurse: try every prefix that matches a dictionary word, then recurse on the remainder — the same remaining substrings get re-examined many times.

```
public boolean wordBreakBrute(String s, Set<String> dict) {
    if (s.isEmpty()) return true;
    for (int end = 1; end <= s.length(); end++) {
        String prefix = s.substring(0, end);
        if (dict.contains(prefix) && wordBreakBrute(s.substring(end), dict)) {
            return true;                          // prefix works, and so does the rest
        }
    }
    return false;
}
```

Optimized (bottom-up boolean dp)

Recurrence: `dp[i] = true` if there exists `j < i` with `dp[j] == true` and `s[j..i)` in the dictionary; `dp[0] = true` (empty prefix needs no words).

```
public boolean wordBreak(String s, List<String> wordDict) {
    Set<String> dict = new HashSet<>(wordDict);
    int n = s.length();
    boolean[] dp = new boolean[n + 1];
    dp[0] = true;                                 // empty prefix is trivially breakable
    for (int i = 1; i <= n; i++) {
        for (int j = 0; j < i; j++) {
            if (dp[j] && dict.contains(s.substring(j, i))) {
                dp[i] = true;                      // prefix [0,i) breaks via split at j
                break;
            }
        }
    }
    return dp[n];
}
```

Version	Time	Space
Brute force	O(2^n)	O(n) call stack
Bottom-up boolean dp	$O(n^2)$	O(n)

---

9. Backtracking

The core idea. Backtracking explores a decision tree: at each step you **choose** one option, **explore** everything that follows from that choice (usually by recursing), and then **un-choose** it (undo the choice) before trying the next option. That "undo" step is what makes it backtracking — you're reusing the same path/state variable for every branch instead of allocating a fresh copy, so you must clean up after yourself before moving on. The whole process is just a depth-first walk over a tree of partial solutions, where each node is a decision ("include this element?", "place this number here?") and leaves are complete (or invalid) candidates.

```
choose(option) -> explore(rest of the problem) -> un-choose(option)
```

Reach for it when: the problem asks for **all** subsets/permutations/combinations that satisfy a condition, you need to **enumerate** rather than count, or a greedy/DP shortcut doesn't exist because you must try every branch and abandon (**prune**) the ones that can't work.

9.1 Subsets

Problem. Given an array of `nums`, return **all possible subsets** (the power set) — every combination of elements you can form by including or excluding each element, plus the empty subset. The solution set must not contain duplicate subsets; subsets may be returned in any order.

- **Constraints:** `1 <= nums.length <= 10`; `-10 <= nums[i] <= 10`; all elements of `nums` are **distinct**.
- **Clarifying assumptions:** confirm that the empty subset `[]` and the full array itself both count as valid subsets (the power set always includes both), and that subset order — and element order within each subset — doesn't matter.
- **Why it's tricky:** it's easy to under- or over-count. The power set has exactly `2^n` members, and the safe way to hit that number exactly once each is to make the include/exclude decision explicit at every level and trust the recursion, rather than trying to build subsets iteratively.

Example.

```
Input:  nums = [1, 2, 3]
Output: [[], [1], [1,2], [1,2,3], [1,3], [2], [2,3], [3]]
```

Intuition (diagram). For each number, decide: include it or not. That's a binary decision at every level of the tree, so the tree has `2^n` leaves — one per subset. The recursion tree below shows the call at each node and which subset gets recorded at each leaf.

```
backtrack(index=0, path=[])
  +- choose 1 -> backtrack(index=1, path=[1])
  |    +- choose 2 -> backtrack(index=2, path=[1,2])
  |    |    +- choose 3 -> index=3, path=[1,2,3]  => record [1,2,3]
  |    |    +- skip 3   -> index=3, path=[1,2]    => record [1,2]
  |    +- skip 2   -> backtrack(index=2, path=[1])
  |         +- choose 3 -> index=3, path=[1,3]    => record [1,3]
  |         +- skip 3   -> index=3, path=[1]      => record [1]
  +- skip 1   -> backtrack(index=1, path=[])
       +- choose 2 -> backtrack(index=2, path=[2])
       |    +- choose 3 -> index=3, path=[2,3]    => record [2,3]
       |    +- skip 3   -> index=3, path=[2]      => record [2]
       +- skip 2   -> backtrack(index=2, path=[])
            +- choose 3 -> index=3, path=[3]      => record [3]
            +- skip 3   -> index=3, path=[]       => record []
```

Approach (step by step)

Key idea / invariant. `index` means "nums[0..index-1] have already been decided (each either included in or excluded from `path`); nums[index..n-1] are still undecided." `path` always holds exactly the elements included by the decisions made so far. Because the invariant is maintained on every call, the base case `index == nums.length` means *every* element has been decided, so `path` is one complete, valid subset.

Choose / explore / un-choose, step by step: 1. Base case first: if `index == nums.length`, every decision has been made — append a **copy** of `path` to `result` (copy because `path` keeps mutating after we return) and return. This single line is what actually produces the `2^n` recorded subsets, one per leaf of the tree above. 2. **Choose:** add `nums[index]` to `path` — this is the "include it" branch. 3. **Explore:** recurse into `backtrack(index + 1, path)`. This fully expands every subset that agrees with "include everything chosen on this path so far." 4. **Un-choose:** remove the last element from `path` (undo step 2), restoring `path` to exactly what it was before — this is the step that makes the algorithm "backtracking" instead of building `2^n` separate copies of `path`. 5. **Explore the sibling branch:** call `backtrack(index + 1, path)` again, this time without `nums[index]` in `path` — the "exclude it" branch. Nothing needs to be un-chosen here because nothing was added. 6. Because each of the `n` levels makes one independent include/exclude choice, the tree has depth `n` and `2^n` leaves — exactly the size of the power set.

Duplicates/order. `nums` is distinct here, so no dedup logic is needed and subset order never matters. (If duplicates were allowed, you'd sort `nums` first and skip a candidate that equals its previous sibling at the same recursion level, so the same subset isn't recorded twice — the same trick used for pruning in Combination Sum below.)

Worked trace on `[1,2,3]`: choose 1 -> choose 2 -> choose 3 -> record `[1,2,3]` -> un-choose 3 -> skip 3 -> record `[1,2]` -> un-choose 2 -> skip 2 -> choose 3 -> record `[1,3]` -> un-choose 3 -> skip 3 -> record `[1]` -> un-choose 1 -> the "skip 1" branch mirrors all of the above using only `{2, 3}`, producing `[2,3]`, `[2]`, `[3]`, `[]`.

```
public List<List<Integer>> subsets(int[] nums) {
    List<List<Integer>> result = new ArrayList<>();
    backtrack(nums, 0, new ArrayList<>(), result);
    return result;
}

private void backtrack(int[] nums, int index, List<Integer> path,
                       List<List<Integer>> result) {
    if (index == nums.length) {
        result.add(new ArrayList<>(path));   // record a copy of the current subset
        return;
    }
    // choose: include nums[index]
    path.add(nums[index]);
    backtrack(nums, index + 1, path, result);
    path.remove(path.size() - 1);             // un-choose

    // choose: exclude nums[index] (explore directly, nothing to undo)
    backtrack(nums, index + 1, path, result);
}
```

Version	Time	Space
Backtracking	$O(n \cdot 2^n)$ — 2^n subsets, O(n) to copy each	O(n) recursion depth (excl. output)

9.2 Permutations

Problem. Given an array of `nums`, return **all possible permutations** (every distinct ordering of all `n` elements), in any order.

- **Constraints:** `1 <= nums.length <= 6`; `-10 <= nums[i] <= 10`; all elements of `nums` are **distinct**.
- **Clarifying assumptions:** confirm the array has no duplicate values (if it did, the same set of positions could otherwise produce identical output permutations that need to be de-duplicated) and that every permutation must use all `n` elements, not a prefix.
- **Why it's tricky:** unlike Subsets, the choice at each level is "which value goes in this position," and the set of legal choices **shrinks** as you go deeper (whatever is already placed can't be reused) — so you need explicit bookkeeping (a `used[]` array) rather than a simple index that only moves forward.

Example.

```
Input:  nums = [1, 2, 3]
Output: [[1,2,3], [1,3,2], [2,1,3], [2,3,1], [3,1,2], [3,2,1]]
```

Intuition (diagram). At each position of the output, choose any number that hasn't been used yet, mark it used, recurse to fill the next position, then unmark it (un-choose) so it's available for other branches.

```
                []
      pick1/    pick2|    pick3\
       [1]         [2]         [3]
      /   \       /   \       /   \
  pick2   pick3 pick1 pick3 pick1 pick2
  [1,2]  [1,3] [2,1] [2,3] [3,1] [3,2]
    |      |     |     |     |     |
[1,2,3][1,3,2][2,1,3][2,3,1][3,1,2][3,2,1]
```

Approach (step by step)

Key idea / invariant. `path` holds the numbers placed in positions `0..path.size()-1` in order, and `used[i]` is true exactly when `nums[i]` already appears somewhere in `path`. Together they guarantee that at every recursive call the set of legal next choices is "every index `i` with `used[i] == false`" — no repeats, and every value eventually gets a turn in every position across the whole tree.

Choose / explore / un-choose, step by step: 1. Base case first: if `path.size() == nums.length`, every position has been filled — record a **copy** of `path` as one full permutation and return. This is the only place output is produced, and it fires once per leaf of the tree above. 2. Otherwise, loop over every index `i` in `nums`: 3. Skip `i` if `used[i]` is true — that value is already committed to an earlier position on this path, so it isn't a legal choice here. 4. **Choose:** set `used[i] = true` and append `nums[i]` to `path` — this commits `nums[i]` to the next open position. 5. **Explore:** recurse to fill the remaining positions with whatever is still unused. 6. **Un-choose:** remove the last element from `path` and reset `used[i] = false` — this frees `nums[i]` back up so the **next** iteration of the loop (a sibling branch) can try placing a different number in this same position, and so `nums[i]` is available again to fill *later* positions in other branches of the tree. 7. Because position 0 has `n` legal choices, position 1 has `n-1` (whichever wasn't already used), and so on down to 1 choice at the last position, the tree has `n!` leaves — one per permutation, matching the `n * (n-1) * ... * 1` fan-out visible in the diagram.

Duplicates/order. `nums` is distinct, so no permutation can be produced by two different root-to-leaf paths — every leaf is unique automatically. (If duplicates were allowed, you'd sort first and skip a candidate equal to the previous sibling *unless* the previous one is still `used`, the standard "skip equal siblings" rule for avoiding duplicate permutations.)

Worked trace on `[1,2,3]`: choose 1 (`used=[T,F,F]`) -> choose 2 (`used=[T,T,F]`) -> choose 3 -> record `[1,2,3]` -> un-choose 3 -> un-choose 2 -> choose 3 (`used=[T,F,T]`) -> choose 2 -> record `[1,3,2]` -> un-choose all the way back to `[]` -> the loop moves to `i=1` (start with 2) and `i=2` (start with 3), each mirroring the same pattern to produce the remaining four permutations.

```
public List<List<Integer>> permute(int[] nums) {
    List<List<Integer>> result = new ArrayList<>();
    boolean[] used = new boolean[nums.length];
    backtrack(nums, used, new ArrayList<>(), result);
    return result;
}

private void backtrack(int[] nums, boolean[] used, List<Integer> path,
                       List<List<Integer>> result) {
    if (path.size() == nums.length) {
        result.add(new ArrayList<>(path));   // full permutation found
        return;
    }
    for (int i = 0; i < nums.length; i++) {
        if (used[i]) continue;                // skip numbers already placed
        used[i] = true;                       // choose
        path.add(nums[i]);
        backtrack(nums, used, path, result);  // explore
        path.remove(path.size() - 1);         // un-choose
        used[i] = false;
    }
}
```

Version	Time	Space
Backtracking	O(n · n!) — n! permutations, O(n) to build each	O(n) recursion depth + used[] (excl. output)

9.3 Combination Sum

Problem. Given an array of `candidates` and an integer `target`, return **all unique combinations** of `candidates` where the chosen numbers sum to exactly `target`. The **same** number may be chosen from `candidates` an unlimited number of times (reuse is allowed); two combinations are the same if they have the same multiset of numbers.

- **Constraints:** `1 <= candidates.length <= 30`; `2 <= candidates[i] <= 40`; all elements of `candidates` are **distinct**; `1 <= target <= 40`; all `candidates[i]` are positive.
- **Clarifying assumptions:** confirm numbers are strictly positive (so the running sum only ever increases and pruning on "sum exceeds target" is safe) and that the same combination should never appear twice even though a number can repeat within it.
- **Why it's tricky:** it's the first problem in this section where "reuse the same element" and "avoid duplicate combinations" pull in different directions — you must let the recursive call revisit the **same index** (for reuse) while never revisiting an **earlier** index (which is what actually prevents `[2,3]` and `[3,2]` from both being recorded). Pruning is also essential: without cutting a branch once the sum overshoots, the search degenerates badly.

Example.

```
Input:  candidates = [2, 3, 6, 7], target = 7
Output: [[2,2,3], [7]]
```

Intuition (diagram). At each candidate index, choose to add it again (reuse allowed, so the recursive call stays at the same index) or move on to the next candidate. Prune (stop exploring) as soon as the running sum exceeds `target` — no point going deeper.

```
                    sum=0, start=0
        take 2 /                     \ skip 2, start=1
    sum=2,start=0                   sum=0,start=1
   take2/    \skip2                 take3/   \skip3
sum=4        sum=2,start=1       sum=3      sum=0,start=2
.../...        ...               take3\      ...
                              sum=6,start=1
                              take3: sum=9 > 7 -> prune
                              skip3, start=2 -> sum=6, take6 -> sum=12>7 prune
                                                  ... eventually [2,2,3] and [7] found
```

Approach (step by step)

Key idea / invariant. `start` means "candidates before index `start` are no longer choosable on this path" — it only ever moves forward or stays put, never backward. `sum` is always the total of everything currently in `path`. Sorting `candidates` first means that once `candidates[i]` alone would push `sum` past `target`, every later `candidates[j]` (`j > i`) is even bigger and would too, so the loop can `break` instead of just `continue`.

Choose / explore / un-choose, step by step: 1. Base case first: if `sum == target`, `path` is a valid combination — record a copy and return (there is no need to keep exploring deeper once the sum matches exactly, since all candidates are positive and adding more would only overshoot). 2. Loop `i` from `start` to the end of `candidates`: 3. **Prune:** if `sum + candidates[i] > target`, `break` out of the loop entirely (not just skip this one) — because the array is sorted, every remaining candidate is at least as large, so none of them can work either. 4. **Choose:** append `candidates[i]` to `path`. 5. **Explore:** recurse with `start = i` (not `i + 1`) and `sum + candidates[i]` — passing `i` again is exactly what allows `candidates[i]` to be picked again on the next level, giving reuse. 6. **Un-choose:** remove the last element from `path` before the loop's next iteration moves on to `candidates[i + 1]`. 7. Because the inner recursive call only ever starts at `i` or later — never at an index before `i` — a number that appears earlier in `candidates` can never be added *after* a later one in the same combination. That single constraint is what prevents `[2,3]` and `[3,2]` from both being generated, without needing a separate dedup step.

Duplicates/order. Candidates are distinct here, so the "never go to an earlier index" rule above is the entire duplicate-avoidance mechanism — no `used[]` array or "skip equal sibling" check is needed. Reuse is what gives you `[2,2,3]` (index 0 chosen twice before advancing to index 1).

Worked trace on `[2,3,6,7]`, target 7: take 2 (`sum=2,start=0`) -> take 2 again (`sum=4,start=0`) -> take 2 again (`sum=6,start=0`) -> take 2 again would give `sum=8>7`, prune; try 3 -> `sum=9>7`, prune; try 6 -> `sum=12>7`, prune -> back up, at `sum=4,start=0` try 3 -> `sum=7` -> record `[2,2,3]` -> back up further to `sum=0,start=0`, move past 2 to `start=1` -> take 3 -> ... eventually reaches `sum=0,start=3`, take 7 -> `sum=7` -> record `[7]`.

```
public List<List<Integer>> combinationSum(int[] candidates, int target) {
    List<List<Integer>> result = new ArrayList<>();
    Arrays.sort(candidates);                 // enables early pruning below
    backtrack(candidates, target, 0, 0, new ArrayList<>(), result);
    return result;
}

private void backtrack(int[] candidates, int target, int start, int sum,
                       List<Integer> path, List<List<Integer>> result) {
    if (sum == target) {
        result.add(new ArrayList<>(path));    // exact match found
        return;
    }
    for (int i = start; i < candidates.length; i++) {
        if (sum + candidates[i] > target) break;   // prune: sorted, so rest are worse too
        path.add(candidates[i]);                    // choose
        backtrack(candidates, target, i, sum + candidates[i], path, result);  // explore (i
        path.remove(path.size() - 1);               // un-choose
    }
}
```

Version	Time	Space
Backtracking (with pruning)	O(2^target) worst case — bounded by number of valid combinations explored	O(target) recursion depth (excl. output)

9.4 Letter Combinations of a Phone Number

Problem. Given a string `digits` containing only the digits `2`-`9`, return **all possible letter combinations** that the number could represent, where each digit maps to its letters on a standard phone keypad (`2 -> "abc"`, `3 -> "def"`, ..., `7 -> "pqrs"`, `9 -> "wxyz"`; note `7` and `9` map to **4** letters, all others map to 3). Return the combinations in any order.

- **Constraints:** `0 <= digits.length <= 4`; every character of `digits` is one of `2`-`9`.
- **Clarifying assumptions:** confirm what to return when `digits` is empty — the standard answer is an empty list `[]`, not a list containing an empty string — and confirm the digit-to-letters mapping matches the classic keypad (especially that `7` and `9` have 4 letters, not 3, which is the easiest detail to get wrong).
- **Why it's tricky:** it looks like simple Cartesian-product enumeration, but the branching factor is **non-uniform** (3 or 4 depending on the digit) and the empty-input edge case is a common off-by-one trap — without an explicit guard, the base case would fire immediately and incorrectly emit `[""]` instead of `[]`.

Example.

```
Input:  digits = "23"
Output: ["ad","ae","af","bd","be","bf","cd","ce","cf"]
```

Intuition (diagram). Each digit position is a decision level in the tree: pick one letter for that digit, recurse into the next digit, then remove the letter (un-choose) before trying the next one at this level.

```
digit '2' -> a, b, c        digit '3' -> d, e, f

                    []
        a  /        |b        \c
      [a]           [b]         [c]
     /  |  \      /  |  \      /  |  \
    d   e   f    d   e   f    d   e   f
  [ad][ae][af]  [bd][be][bf]  [cd][ce][cf]
```

Approach (step by step)

Key idea / invariant. `index` means "digits[0..index-1] already have a letter chosen for them, and that letter is appended to `path` at the matching position." `path` is therefore always a prefix of length `index`, built from one letter per digit processed so far. The base case `index == digits.length()` means every digit has contributed a letter, so `path` is one complete, valid combination.

Choose / explore / un-choose, step by step: 1. Guard first: if `digits` is null or empty, return the empty list immediately — there is no digit to recurse on, and without this check the base case below would trivially match on the first call and wrongly emit a single empty string. 2. Base case: if `index == digits.length()`, `path` has one letter per digit — record `path.toString()` and return. 3. Look up `letters = KEYPAD[digits.charAt(index) - '0']`, the 3-4 candidate letters for the current digit. 4. Loop over each character `c` in `letters`: 5. **Choose:** append `c` to `path`. 6. **Explore:** recurse to `index + 1` to resolve every remaining digit given this choice of `c`. 7. **Un-choose:** delete the last character of `path` (undo step 5) before the loop tries the next letter for this same digit — this is what lets `path` be reused across all 3-4 branches at this level instead of allocating a new buffer each time. 8. Because digit `i` contributes a factor of 3 or 4 to the branching, the tree has depth `digits.length()` and up to `4^n` leaves, matching the complexity bound below.

Duplicates/order. No duplicates are possible: each tree level corresponds to one fixed digit's letters, which are all distinct from each other, so every root-to-leaf path spells a distinct string. Order of the output follows the order letters appear in `KEYPAD`, which is why `"23"` produces `ad, ae, af, bd, ...` rather than a different ordering.

Worked trace on `"23"`: digit `2` gives letters `a,b,c`; for `a`: choose `a` -> digit `3` gives `d,e,f` -> choose `d` -> record `"ad"` -> un-choose `d` -> choose `e` -> record `"ae"` -> un-choose `e` -> choose `f` -> record `"af"` -> un-choose `f`, un-choose `a` -> repeat the same three-letter pattern for `b` (`"bd","be","bf"`) and `c` (`"cd","ce","cf"`).

Approach (step by step, continued: implementation)

Map each digit to its letters, then for the digit at the current position choose one letter, append it, recurse to the next digit, and remove it (un-choose) before trying the next letter.

```
private static final String[] KEYPAD = {
    "", "", "abc", "def", "ghi", "jkl", "mno", "pqrs", "tuv", "wxyz"
};

public List<String> letterCombinations(String digits) {
    List<String> result = new ArrayList<>();
    if (digits == null || digits.isEmpty()) return result;
    backtrack(digits, 0, new StringBuilder(), result);
    return result;
}

private void backtrack(String digits, int index, StringBuilder path, List<String> result) {
    if (index == digits.length()) {
        result.add(path.toString());          // full combination built
        return;
    }
    String letters = KEYPAD[digits.charAt(index) - '0'];
    for (char c : letters.toCharArray()) {
        path.append(c);                        // choose
        backtrack(digits, index + 1, path, result);  // explore
        path.deleteCharAt(path.length() - 1);  // un-choose
    }
}
```

Version	Time	Space
Backtracking	$O(4^n \cdot n)$ — up to 4 letters/digit, n chars to build each string	O(n) recursion depth (excl. output)

---

10. Heaps / Priority Queue

The core idea. A heap is a tree-shaped structure (usually stored in a flat array) that keeps one property invariant: in a **min-heap**, every parent is `<=` its children, so the smallest element is always at the root; in a **max-heap**, every parent is `>=` its children, so the largest element is always at the root. You never get full sorted order for free, but you always get instant access to the *extreme* element. Java's `PriorityQueue` is a min-heap by default — pass a comparator to flip it into a max-heap, or to order by a custom key. Insert (`offer`) and remove-the-extreme (`poll`) are both `O(log n)` because the heap only has to fix a single root-to-leaf path, not re-sort everything; peeking at the extreme (`peek`) is `O(1)`.

Reach for it when: you repeatedly need the **smallest/largest/closest** item out of a growing or shrinking collection — "top k", "k closest", "kth largest so far", or **merging multiple sorted sources** where you must always pick the next-smallest head.

10.1 Kth Largest Element in an Array

Problem. Given an integer array `nums` and an integer `k` (`1 <= k <= nums.length`), return the `k`th largest element of `nums` when the array is considered in sorted-descending order. This is the `k`th largest *value by rank*, not the `k`th distinct value — duplicates each occupy their own rank.

- **Constraints:** `1 <= nums.length <= 10^5`; `-10^4 <= nums[i] <= 10^4`; `nums` is unsorted and may contain duplicates and negative numbers; `k` is always valid (`1 <= k <= nums.length`).
- **Clarifying assumptions:** confirm that "kth largest" counts duplicates separately (e.g. `[3,3,3]`, `k=2` returns `3`, not "no 2nd distinct value"), and that the array is not sorted and not guaranteed to fit in memory sorted twice.
- **Why it's tricky:** the obvious full sort is `O(n log n)`; the insight is that you never need the other `n - k` elements fully ordered — a heap that only ever tracks the current top `k` gets you the answer in `O(n log k)`, which is much better when `k << n`.

Example.

```
Input:  nums = [3, 2, 1, 5, 6, 4], k = 2
Output: 5              (sorted desc: 6, 5, 4, 3, 2, 1 -> 2nd largest is 5)
```

Intuition (diagram). Keep a **min-heap of size k**. Walk the array; push every number, and whenever the heap grows past size `k`, pop the smallest. The heap only ever holds the `k` largest numbers seen so far, and its root — the smallest of those `k` — is exactly the answer once you've processed everything.

```
nums: 3 2 1 5 6 4,  k = 2

push 3   -> heap [3]
push 2   -> heap [2, 3]
push 1   -> heap [1, 2, 3]   size 3 > k=2, pop min 1 -> [2, 3]
push 5   -> heap [2, 3, 5]   size 3 > k=2, pop min 2 -> [3, 5]
push 6   -> heap [3, 5, 6]   size 3 > k=2, pop min 3 -> [5, 6]
push 4   -> heap [4, 5, 6]   size 3 > k=2, pop min 4 -> [5, 6]

root of final heap = 5   <- 2nd largest
```

Approach (step by step)

Key idea / invariant. A min-heap capped at size `k` always holds exactly the `k` *largest* elements seen so far — never more, never fewer. Because it's a min-heap, the weakest member of that top-`k` set (the smallest of the `k` largest) sits at the root, ready to be evicted the instant a bigger number shows up. When the whole array has been consumed, the heap holds the true top `k` values, and its root (the smallest of them) is by definition the `k`th largest overall.

1. Create an empty min-heap (Java's `PriorityQueue` is a min-heap by default).
2. For each number in `nums`: a. Push it onto the heap (`offer`) — cost `O(log h)` where `h` is the current heap size. b. If the heap's size just exceeded `k`, pop the minimum (`poll`) — also `O(log k)` — to discard the weakest member of the current top-`k` set.
3. After the array is exhausted, the heap has exactly `k` elements: the `k` largest values from `nums`. Its root is the smallest of those `k`, i.e. the `k`th largest overall — return `peek()` in `O(1)`.

Each of the `n` numbers causes at most one push and one pop, and the heap never holds more than `k + 1` elements, so every heap operation costs `O(log k)`; total time is `O(n log k)`.

Worked trace (`nums = [3, 2, 1, 5, 6, 4]`, `k = 2`):

```
step | push | heap after push | size > k? | action
-----|------|-----------------|-----------|--------------------
1    | 3    | [3]             | no        | -
2    | 2    | [2, 3]          | no        | -
3    | 1    | [1, 2, 3]       | yes       | pop min 1 -> [2, 3]
4    | 5    | [2, 3, 5]       | yes       | pop min 2 -> [3, 5]
5    | 6    | [3, 5, 6]       | yes       | pop min 3 -> [5, 6]
6    | 4    | [4, 5, 6]       | yes       | pop min 4 -> [5, 6]
```

Final heap `[5, 6]` has root `5` — the 2nd largest element.

Brute force

Sort descending, index into position `k - 1`.

```
public int findKthLargestBrute(int[] nums, int k) {
    int[] copy = nums.clone();
    Arrays.sort(copy);                      // ascending
    return copy[copy.length - k];           // kth from the end
}
```

Optimized (heap)

Min-heap capped at size `k`; its root is the `k`th largest.

```
public int findKthLargest(int[] nums, int k) {
    // min-heap: smallest of the "top k so far" sits at the root
    PriorityQueue<Integer> minHeap = new PriorityQueue<>((a, b) -> Integer.compare(a, b));

    for (int num : nums) {
        minHeap.offer(num);
        if (minHeap.size() > k) {
            minHeap.poll();                 // evict the smallest, keep only top k
        }
    }
    return minHeap.peek();                  // root = kth largest
}
```

Version	Time	Space
Sort	O(n log n)	O(n) (or O(1) if sorting in place)
Min-heap of size k	O(n log k)	O(k)

10.2 K Closest Points to Origin

Problem. Given an array of points `[x, y]` on a 2-D plane and an integer `k`, return the `k` points closest to the origin `(0, 0)`, in any order. Distance is Euclidean, but since `sqrt` is monotonic you can compare squared distances directly and avoid floating point.

- **Constraints:** `1 <= k <= points.length <= 10^4`; `-10^4 <= x, y <= 10^4`; points may repeat and may include the origin itself; `k` never exceeds the number of points.
- **Clarifying assumptions:** confirm ties (equal distances) can be returned in any order, and that the output does not need to be sorted by distance — just be the correct *set* of `k` closest points.
- **Why it's tricky:** using true `sqrt` distances risks floating-point error; using `int` differences in a comparator risks overflow since squared coordinates can reach `2*10^8`. The heap trick (max-heap capped at `k`) also inverts the usual "min-heap for smallest k" instinct — you evict the *largest*, not the smallest, because you're keeping the closest.

Example.

```
Input:  points = [[1,3],[-2,2]], k = 1
Output: [[-2,2]]        (dist² = 1+9=10 vs 4+4=8, so [-2,2] is closer)
```

Intuition (diagram). Keep a **max-heap of size k**, ordered by distance. Push each point; once the heap exceeds size `k`, pop the *farthest* one. The heap's root is always the current worst of the kept points, so popping it keeps only the `k` closest overall.

```
points by dist^2: (1,3)=10  (-2,2)=8   k = 1

push (1,3)   dist=10   -> heap [10]
push (-2,2)  dist=8    -> heap [10, 8]   size 2 > k=1, pop max 10 -> [8]

remaining heap = [(-2,2)]   <- the k closest points
```

Approach (step by step)

Key idea / invariant. A max-heap capped at size `k` holds the `k` *closest* points seen so far, with the current farthest of those `k` sitting at the root. Any time a new point turns out to be closer than the current worst kept point, the worst one is evicted. When all points have been processed, the heap contains exactly the `k` closest points overall — here only the *membership* of the heap matters, not its root value (unlike 10.1, where the root itself was the answer).

1. Create an empty max-heap ordered by squared distance from the origin (reverse the comparator so the *largest* distance floats to the root).
2. For each point: a. Push it onto the heap — `O(log h)`. b. If the heap's size just exceeded `k`, pop the max (farthest point) — `O(log k)`.
3. After all points are processed, the heap holds exactly `k` points: the closest `k` to the origin. Drain it into the result array — order doesn't matter since the problem accepts any order.

Every point causes at most one push and one pop against a heap that never exceeds `k + 1` elements, so total time is `O(n log k)`.

Worked trace (`points = [(1,3), (-2,2)]`, `k = 1`):

```
step | push point | dist^2 | heap after push | size > k? | action
-----|------------|--------|-----------------|-----------|------------------
1    | (1,3)      | 10     | [10]            | no        | -
2    | (-2,2)     | 8      | [10, 8]         | yes       | pop max 10 -> [8]
```

Final heap holds only `(-2, 2)` — the single closest point.

Brute force

Sort all points by squared distance, take the first `k`.

```
public int[][] kClosestBrute(int[][] points, int k) {
    Arrays.sort(points, (a, b) -> {
        long da = (long) a[0] * a[0] + (long) a[1] * a[1];
        long db = (long) b[0] * b[0] + (long) b[1] * b[1];
        return Long.compare(da, db);
    });
    return Arrays.copyOfRange(points, 0, k);
}
```

Optimized (heap)

Max-heap capped at size `k`, ordered by squared distance (use `long` / `Long.compare` — never subtract raw values in a comparator, since `int` differences can silently overflow).

```
public int[][] kClosest(int[][] points, int k) {
    // max-heap: farthest of the "closest k so far" sits at the root
    PriorityQueue<int[]> maxHeap = new PriorityQueue<>((a, b) -> {
        long da = (long) a[0] * a[0] + (long) a[1] * a[1];
        long db = (long) b[0] * b[0] + (long) b[1] * b[1];
        return Long.compare(db, da);              // reversed -> largest dist at root
    });

    for (int[] point : points) {
        maxHeap.offer(point);
        if (maxHeap.size() > k) {
            maxHeap.poll();                        // evict the farthest, keep k closest
        }
    }
    return maxHeap.toArray(new int[maxHeap.size()][]);
}
```

Version	Time	Space
Sort	O(n log n)	O(n)
Max-heap of size k	O(n log k)	O(k)

10.3 Merge k Sorted Lists

Problem. You're given an array `lists` of `k` linked lists, each already sorted in ascending order. Merge all of them into one sorted linked list and return its head.

```
class ListNode {
    int val;
    ListNode next;
    ListNode(int val) { this.val = val; }
}
```

- **Constraints:** `0 <= k <= 10^4`; total number of nodes across all lists is `0 <= n <= 10^4`; `-10^4 <= ListNode.val <= 10^4`; each individual list is already sorted ascending; any list (or the whole array) may be empty.
- **Clarifying assumptions:** confirm the lists may have very different lengths (including zero), that `lists` itself can be empty (return `null`), and that stability among equal values doesn't matter for correctness.
- **Why it's tricky:** naively merging two-at-a-time is `O(kn)` in the worst case (repeatedly re-scanning); the key insight is to keep exactly one "candidate" node per list alive in a min-heap at all times, so you only ever compare the current heads, never rescan a list from its start.

Example.

```
Input:  lists = [1->4->5, 1->3->4, 2->6]
Output: 1->1->2->3->4->4->5->6
```

Intuition (diagram). Put the **head node of every list** into a min-heap ordered by `val`. Repeatedly pop the smallest head, append it to the result, and — if that node has a `next` — push `next` into the heap.

The heap always holds one "candidate" per remaining list, so its root is always the next-smallest value across all lists.

```
lists:  a: 1 -> 4 -> 5      b: 1 -> 3 -> 4      c: 2 -> 6

heap starts with each list's head: [1(a), 1(b), 2(c)]
pop 1(a) -> append, push 4(a)      heap: [1(b), 2(c), 4(a)]
pop 1(b) -> append, push 3(b)      heap: [2(c), 3(b), 4(a)]
pop 2(c) -> append, push 6(c)      heap: [3(b), 4(a), 6(c)]
pop 3(b) -> append, push 4(b)      heap: [4(a), 4(b), 6(c)]
... continues until heap empties -> 1, 1, 2, 3, 4, 4, 5, 6
```

Approach (step by step)

Key idea / invariant — the "heap of heads" technique. At every point in the algorithm, the heap contains at most one node from each list still not fully consumed: the smallest unconsumed node of that list (its current "head"). Because every list is individually sorted, the smallest node in a list is always its current head — so the heap's root, the smallest value across all current heads, is guaranteed to be the smallest value across *all* remaining nodes in *all* lists. That means it's always safe to pop the root and append it to the output.

1. Push the head node of every non-empty list into a min-heap ordered by `val`. The heap now holds up to `k` nodes, one per list.
2. While the heap is not empty: a. Pop the minimum node (`poll`, `O(log k)`) — this is provably the next-smallest value across every remaining node in every list, so append it directly to the result. b. If the popped node has a `next`, push `next` into the heap (`offer`, `O(log k)`) — this restores the invariant by supplying that list's new current head.
3. When the heap empties, every node from every list has been consumed exactly once, in sorted order.

The heap never holds more than `k` nodes, and every one of the `n` total nodes is pushed and popped exactly once, so total time is `O(n log k)`.

Worked trace (`lists = [1->4->5, 1->3->4, 2->6]`, lists labeled a, b, c):

```
step | pop (min) | result so far          | push (popped.next) | heap after
-----|-----------|-----------------------|--------------------|-------------------
0    | -         | -                     | -                  | [1(a), 1(b), 2(c)]
1    | 1(a)      | 1                     | 4(a)               | [1(b), 2(c), 4(a)]
2    | 1(b)      | 1, 1                  | 3(b)               | [2(c), 3(b), 4(a)]
3    | 2(c)      | 1, 1, 2               | 6(c)               | [3(b), 4(a), 6(c)]
4    | 3(b)      | 1, 1, 2, 3            | 4(b)               | [4(a), 4(b), 6(c)]
5    | 4(a)      | 1, 1, 2, 3, 4         | 5(a)               | [4(b), 5(a), 6(c)]
6    | 4(b)      | 1, 1, 2, 3, 4, 4      | none               | [5(a), 6(c)]
7    | 5(a)      | 1, 1, 2, 3, 4, 4, 5   | none               | [6(c)]
8    | 6(c)      | 1, 1, 2, 3, 4, 4, 5, 6 | none              | []
```

The heap empties with the result fully merged and sorted: `1,1,2,3,4,4,5,6`.

Brute force

Dump every node's value into a list, sort it, rebuild a linked list.

```
public ListNode mergeKListsBrute(ListNode[] lists) {
    List<Integer> values = new ArrayList<>();
    for (ListNode node : lists) {
        while (node != null) {
            values.add(node.val);
            node = node.next;
        }
    }
    Collections.sort(values);

    ListNode dummy = new ListNode(0);
    ListNode tail = dummy;
    for (int v : values) {
        tail.next = new ListNode(v);
        tail = tail.next;
    }
    return dummy.next;
}
```

Optimized (heap)

Min-heap of "current head" nodes, one per list.

```
public ListNode mergeKLists(ListNode[] lists) {
    // min-heap: smallest unconsumed head across all k lists sits at the root
    PriorityQueue<ListNode> minHeap = new PriorityQueue<>((a, b) -> Integer.compare(a.val,

    for (ListNode head : lists) {
        if (head != null) {
            minHeap.offer(head);
        }
    }

    ListNode dummy = new ListNode(0);
    ListNode tail = dummy;
    while (!minHeap.isEmpty()) {
        ListNode smallest = minHeap.poll();
        tail.next = smallest;
        tail = tail.next;
        if (smallest.next != null) {
            minHeap.offer(smallest.next);       // advance that list by one node
        }
    }
    return dummy.next;
}
```

Version	Time	Space
Collect + sort	O(n log n)	O(n) (n = total nodes)
Min-heap of k heads	O(n log k)	O(k)

10.4 Kth Largest Element in a Stream

Problem. Design a class `KthLargest` that is constructed with an integer `k` and an initial array `nums`, and supports `add(val)`: it adds `val` to the running stream and returns the `k`th largest element among *all* values added so far (including the initial `nums`).

- **Constraints:** `1 <= k <= 10^4`; `0 <= nums.length <= 10^4`; `-10^4 <= nums[i], val <= 10^4`; it is guaranteed that there will be at least `k` elements in the stream when each `add` is called (so the answer is always well-defined).
- **Clarifying assumptions:** confirm the initial `nums` array can have fewer than `k` elements (the guarantee only applies *after* enough `add` calls, not necessarily at construction), and that values (including duplicates) all count toward rank, same as 10.1.
- **Why it's tricky:** this is 10.1's algorithm but turned into *stateful, amortized* streaming — you must not re-sort or re-scan the whole history on every call, so the min- heap has to persist as instance state across calls instead of being rebuilt each time.

Example.

```
KthLargest kth = new KthLargest(3, new int[]{4, 5, 8, 2});
kth.add(3);   // stream: 4,5,8,2,3      -> 3rd largest = 4
kth.add(5);   // stream: 4,5,8,2,3,5    -> 3rd largest = 5
kth.add(10);  // stream: ...,10         -> 3rd largest = 5
kth.add(9);   // stream: ...,9          -> 3rd largest = 8
kth.add(4);   // stream: ...,4          -> 3rd largest = 8
```

Intuition (diagram). Same trick as 10.1, but now it's *persistent* across calls: keep a **min-heap of size k** as instance state. Every `add` pushes the new value, and if the heap grows past `k`, pops the smallest. The root is always the current `k`th largest — no need to ever look at the full history again.

```
k = 3, heap so far holds the 3 largest seen: [4, 5, 8]   (root = 4)

add(3): push 3 -> [3, 4, 5, 8]   size 4 > k=3, pop min 3 -> [4, 5, 8]   root=4
add(5): push 5 -> [4, 5, 5, 8]   size 4 > k=3, pop min 4 -> [5, 5, 8]   root=5
```

Approach (step by step)

Key idea / invariant. Exactly like 10.1, a min-heap capped at size `k` always contains the `k` largest values seen across the *entire lifetime* of the object, with the smallest of those `k` (the `k`th largest overall) sitting at the root. The only difference from 10.1 is that the heap is a field on the object rather than a local variable, so the invariant is maintained *across calls* instead of within a single pass.

1. In the constructor, create the min-heap once and seed it by calling `add` for every value in the initial `nums` array — this reuses the exact same push/evict logic needed later.
2. On every `add(val)`: a. Push `val` onto the heap — `O(log k)`. b. If the heap's size just exceeded `k`, pop the minimum — `O(log k)` — discarding whichever value is currently weakest among the top `k`. c. Return `peek()`, the current root, in `O(1)` — this is the `k`th largest value seen so far, by the same argument as 10.1.
3. Because the heap never grows past `k + 1` elements even as the stream grows unbounded, each `add` call costs `O(log k)` regardless of how many values have been seen in total.

Worked trace (`k = 3`, seeded with `[4, 5, 8, 2]`, then `add(3)`, `add(5)`):

```
call      | push    | heap after push | size > k? | action                     | return (rc
----------|---------|-----------------|-----------|----------------------------|-----------
init seed | 4,5,8,2 | [4, 5, 8]       | n/a       | keep top 3 of the 4 seeded | 4
add(3)    | 3       | [3, 4, 5, 8]    | yes       | pop min 3 -> [4, 5, 8]     | 4
add(5)    | 5       | [4, 5, 5, 8]    | yes       | pop min 4 -> [5, 5, 8]     | 5
```

(During seeding, `2` is pushed and then evicted once the 4th value makes the heap exceed size `k = 3`, leaving `[4, 5, 8]`.)

Brute force

Store every value seen; on each `add`, sort and index.

```
class KthLargestBrute {
    private final List<Integer> stream = new ArrayList<>();
    private final int k;

    public KthLargestBrute(int k, int[] nums) {
        this.k = k;
        for (int n : nums) stream.add(n);
    }

    public int add(int val) {
        stream.add(val);
        List<Integer> sorted = new ArrayList<>(stream);
        Collections.sort(sorted);                    // ascending
        return sorted.get(sorted.size() - k);        // kth from the end
    }
}
```

Optimized (heap)

Min-heap of size `k` kept alive as instance state across calls.

```
class KthLargest {
    private final PriorityQueue<Integer> minHeap;
    private final int k;

    public KthLargest(int k, int[] nums) {
        this.k = k;
        // min-heap: smallest of the "top k so far" sits at the root
        this.minHeap = new PriorityQueue<>((a, b) -> Integer.compare(a, b));
        for (int n : nums) {
            add(n);                                  // reuse add() to seed the heap
        }
    }

    public int add(int val) {
        minHeap.offer(val);
        if (minHeap.size() > k) {
            minHeap.poll();                           // evict the smallest, keep top k
        }
        return minHeap.peek();                        // root = kth largest so far
    }
}
```

Version	Time	Space
Re-sort on every add	O(n log n) per add	O(n)
Min-heap of size k	O(log k) per add	O(k)

---

11. Intervals

The core idea. Represent each interval as `int[]{start, end}`. Almost every interval problem starts the same way: **sort by start time**, then sweep left to right, comparing each interval to the one currently "open." Two intervals overlap when `a.start <= b.end` (using the previously merged/open interval `a` and the next one `b`). Sorting turns an $O(n^2)$ pairwise comparison into a single linear pass.

Reach for it when: you're merging, inserting, counting removals among, or scheduling **ranges** — anything phrased as "intervals," "meetings," "bookings," or "ranges that overlap."

11.1 Merge Intervals

Problem. Given an array `intervals` where `intervals[i] = [start_i, end_i]`, merge all overlapping intervals and return an array of the non-overlapping intervals that cover all the intervals in the input, sorted by start.

- **Constraints:** `1 <= intervals.length <= 10^4`; `intervals[i].length == 2`; `0 <= start_i <= end_i <= 10^4`. Intervals may be given in any order and may already overlap in complex ways (e.g. one fully containing another).
- **Clarifying assumptions:** treat touching intervals (`a.end == b.start`) as overlapping and merge them; the output order doesn't matter to the grader as long as it's sorted by start and each interval is `[start, end]`.
- **Why it's tricky:** the naive approach re-scans after every merge; the insight that *sorting by start* turns "does this overlap anything" into "does this overlap the single most-recently-closed interval" is what gets you from $O(n^2)$/$O(n^3)$ to $O(n \log n)$.

Example.

```
Input:  [[1,3], [2,6], [8,10], [15,18]]
Output: [[1,6], [8,10], [15,18]]
```

Intuition (diagram). Sort by start. Keep a "current merged" interval; if the next interval's start is `<=` the current end, stretch the current end; otherwise close it out and start a new one.

```
0 1 2 3     6   8  10        15    18
  [---]
    [-------]
                [---]
                               [-----]
```

```
result (after merging):
  [---------]
                [---]
                               [-----]
```

Approach (step by step)

- **Key idea / invariant:** once intervals are sorted by start, any interval that overlaps the "current" merged interval must appear immediately after it in the sorted order — you never need to look back. The invariant maintained is: `current` always represents the fully-merged range of every interval seen so far that chains back to it.
- **Numbered walk-through:**
- Sort all intervals by `start`.
- Initialize `current` to the first interval.
- For each subsequent interval `next`: if `next.start <= current.end`, they overlap (or touch) — extend `current.end = max(current.end, next.end)`.
- Otherwise there's a gap — push `current` to the result and set `current = next`.
- After the loop, push the final `current` (the last interval is never closed out inside the loop).
- **Worked trace** on `[[1,3],[2,6],[8,10],[15,18]]` (already sorted by start):
- `current = [1,3]`
- `next=[2,6]`: `2 <= 3` -> overlap -> `current = [1,6]`
- `next=[8,10]`: `8 <= 6` ? no -> gap -> push `[1,6]`, `current = [8,10]`
- `next=[15,18]`: `15 <= 10` ? no -> gap -> push `[8,10]`, `current = [15,18]`
- loop ends -> push `[15,18]`
- result: `[[1,6],[8,10],[15,18]]`

Brute force

Repeatedly scan for any overlapping pair and merge it, until nothing changes.

```
public int[][] mergeBrute(int[][] intervals) {
    List<int[]> current = new ArrayList<>(Arrays.asList(intervals));
    boolean mergedAny = true;
    while (mergedAny) {
        mergedAny = false;
        for (int i = 0; i < current.size() && !mergedAny; i++) {
            for (int j = i + 1; j < current.size(); j++) {
                int[] a = current.get(i), b = current.get(j);
                if (a[0] <= b[1] && b[0] <= a[1]) {           // overlap either way
                    int[] merged = { Math.min(a[0], b[0]), Math.max(a[1], b[1]) };
                    current.remove(j);
                    current.remove(i);
                    current.add(merged);
                    mergedAny = true;
                    break;
                }
            }
        }
    }
    return current.toArray(new int[0][]);
}
```

Optimized (sort + sweep)

One pass after sorting; no re-scanning needed.

```
public int[][] merge(int[][] intervals) {
    Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]));
    List<int[]> result = new ArrayList<>();
    int[] current = intervals[0];
    for (int i = 1; i < intervals.length; i++) {
        int[] next = intervals[i];
        if (next[0] <= current[1]) {           // overlaps -> extend
            current[1] = Math.max(current[1], next[1]);
        } else {                               // gap -> close current, start new
            result.add(current);
            current = next;
        }
    }
    result.add(current);
    return result.toArray(new int[0][]);
}
```

Version	Time	Space
Brute force	O(n^3)	O(n)
Sort + sweep	O(n log n)	O(n) for the result

11.2 Insert Interval

Problem. Given an array `intervals` of non-overlapping intervals sorted ascending by start, and a single new interval `newInterval`, insert `newInterval` into `intervals` so that the result is still sorted by start and still has no overlapping intervals (merging where necessary). Return the resulting array.

- **Constraints:** `0 <= intervals.length <= 10^4`; `intervals[i].length == 2`; `0 <= start_i <= end_i <= 10^5`; `intervals` is sorted by `start_i` and pairwise non-overlapping; `newInterval.length == 2`, `0 <= start <= end <= 10^5`.
- **Clarifying assumptions:** the input `intervals` array is guaranteed already merged and sorted (unlike 11.1) — you may rely on that; touching intervals (`end == start`) should be merged just like in Merge Intervals.
- **Why it's tricky:** it's tempting to just append and re-run full Merge Intervals (which works but throws away the fact that the input is already sorted). The O(n) solution requires recognizing the three linearly-ordered phases (before / overlapping / after) so a single left-to-right pass suffices with no re-sort.

Example.

```
Input:  intervals = [[1,3], [6,9]], newInterval = [2,5]
Output: [[1,5], [6,9]]
```

Intuition (diagram). Walk through in three phases: (1) intervals that end entirely **before** the new one starts — copy as is; (2) intervals that **overlap** the new one — absorb them into a growing merged interval; (3) intervals that start entirely **after** — copy as is.

```
0 1 2 3   5 6     9
  [---]   existing
    [-----]   new interval
            [-----]   existing
```

Approach (step by step)

- **Key idea / invariant:** because `intervals` is already sorted and non-overlapping, the intervals that touch `newInterval` form one contiguous run. A single pointer `i` sweeping left to right can classify every existing interval into exactly one of three phases, in order, without ever moving backward.
- **Numbered walk-through:**
- **Phase 1 (before):** while the current interval's end is strictly less than `newInterval`'s start (`intervals[i][1] < newInterval[0]`), it can't overlap — copy it to the result and advance `i`.
- **Phase 2 (overlapping):** while the current interval's start is `<= newInterval`'s end (`intervals[i][0] <= newInterval[1]`), it overlaps or touches — absorb it by expanding `newInterval` to `[min(starts), max(ends)]` and advance `i`.
- Push the (possibly expanded) `newInterval` to the result.
- **Phase 3 (after):** whatever intervals remain start strictly after `newInterval` now ends — copy them all as is.
- **Worked trace** on `intervals=[[1,3],[6,9]]`, `newInterval=[2,5]`:
- Phase 1: `intervals[0]=[1,3]`, is `3 < 2`? no -> phase 1 copies nothing, `i` stays at 0.
- Phase 2: `intervals[0]=[1,3]`, is `1 <= 5`? yes -> absorb -> `newInterval=[min(2,1), max(5,3)]=[1,5]`, `i=1`. Next `intervals[1]=[6,9]`, is `6 <= 5`? no -> stop phase 2.
- Push `newInterval=[1,5]`.
- Phase 3: copy remaining `[6,9]` unchanged.
- result: `[[1,5],[6,9]]`

Brute force

Append the new interval, then run the general merge routine on everything.

```
public int[][] insertBrute(int[][] intervals, int[] newInterval) {
    List<int[]> all = new ArrayList<>(Arrays.asList(intervals));
    all.add(newInterval);
    int[][] combined = all.toArray(new int[0][]);
    Arrays.sort(combined, (a, b) -> Integer.compare(a[0], b[0]));
    List<int[]> result = new ArrayList<>();
    int[] current = combined[0];
    for (int i = 1; i < combined.length; i++) {
        if (combined[i][0] <= current[1]) {
            current[1] = Math.max(current[1], combined[i][1]);
        } else {
            result.add(current);
            current = combined[i];
        }
    }
    result.add(current);
    return result.toArray(new int[0][]);
}
```

Optimized (three-phase sweep)

Exploit that the input is already sorted; no full re-sort needed.

```
public int[][] insert(int[][] intervals, int[] newInterval) {
    List<int[]> result = new ArrayList<>();
    int i = 0, n = intervals.length;

    // phase 1: intervals ending before newInterval starts
    while (i < n && intervals[i][1] < newInterval[0]) {
        result.add(intervals[i]);
        i++;
    }

    // phase 2: intervals overlapping newInterval -> absorb into it
    while (i < n && intervals[i][0] <= newInterval[1]) {
        newInterval[0] = Math.min(newInterval[0], intervals[i][0]);
        newInterval[1] = Math.max(newInterval[1], intervals[i][1]);
        i++;
    }
    result.add(newInterval);

    // phase 3: whatever remains starts after newInterval
    while (i < n) {
        result.add(intervals[i]);
        i++;
    }
    return result.toArray(new int[0][]);
}
```

Version	Time	Space
Brute force	O(n log n)	O(n)
Three-phase sweep	O(n)	O(n) for the result

11.3 Non-overlapping Intervals

Problem. Given an array of intervals `intervals[i] = [start_i, end_i]`, return the minimum number of intervals you need to remove so that the remaining intervals are pairwise non-overlapping.

- **Constraints:** `1 <= intervals.length <= 10^5`; `intervals[i].length == 2`; `-5 * 10^4 <= start_i < end_i <= 5 * 10^4` (each interval is non-empty: `start_i < end_i`).
- **Clarifying assumptions:** intervals that only touch at a point (`a.end == b.start`) are **not** considered overlapping (this matches LeetCode 435's convention — confirm with the interviewer since it flips the comparison from `<=` to `<`); duplicates are allowed.
- **Why it's tricky:** sorting by **start** (the habit from 11.1) doesn't work here — you'd need to know the future to decide which of two overlapping intervals to drop. Sorting by **end** time is the key insight: greedily keeping the interval that frees up the timeline soonest is always at least as good as keeping any other overlapping choice (an exchange argument / activity-selection proof).

Example.

```
Input:  [[1,2], [2,3], [3,4], [1,3]]
Output: 1               (remove [1,3]; the rest don't overlap)
```

Intuition (diagram). Sort by **end** time. Greedily keep the interval that finishes earliest — it leaves the most room for everything after it. Whenever the next interval starts before the last *kept* interval ends, it must be the one removed.

```
0 1 2 3 4
  [-]   keep (end=2)
    [-]   keep (end=3)
  [---]   overlaps -> remove
      [-]   keep (end=4)
```

Approach (step by step)

- **Key idea / invariant:** after sorting by end time, `lastEnd` always holds the end time of the most recently *kept* interval. Any interval whose start is `< lastEnd` conflicts with something already kept and is the cheapest one to discard (since it finishes no later than any interval it conflicts with, discarding it — rather than the one before it — never makes the remaining schedule worse).
- **Numbered walk-through:**
- Sort intervals by `end` ascending.
- Keep the first interval; set `lastEnd = intervals[0].end`, `removals = 0`.
- For each subsequent interval: if `start < lastEnd`, it overlaps the last kept interval — increment `removals` and discard it (don't update `lastEnd`).
- Otherwise it doesn't overlap — keep it and update `lastEnd = interval.end`.
- Return `removals`.
- **Worked trace** on `[[1,2],[2,3],[3,4],[1,3]]`, sorted by end: `[1,2],[2,3],[1,3],[3,4]`:
- keep `[1,2]`, `lastEnd=2`
- `[2,3]`: `2 < 2` ? no -> keep, `lastEnd=3`
- `[1,3]`: `1 < 3` ? yes -> overlaps -> remove, `removals=1`, `lastEnd` unchanged
- `[3,4]`: `3 < 3` ? no -> keep, `lastEnd=4`
- result: `removals = 1`

Brute force

Try every subset, keep the largest subset with no overlaps, subtract from n.

```
public int eraseOverlapIntervalsBrute(int[][] intervals) {
    int n = intervals.length;
    int best = 0;
    for (int mask = 0; mask < (1 << n); mask++) {
        List<int[]> subset = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            if ((mask & (1 << i)) != 0) subset.add(intervals[i]);
        }
        subset.sort((a, b) -> Integer.compare(a[0], b[0]));
        boolean ok = true;
        for (int i = 1; i < subset.size() && ok; i++) {
            if (subset.get(i)[0] < subset.get(i - 1)[1]) ok = false;   // overlap
        }
        if (ok) best = Math.max(best, subset.size());
    }
    return n - best;
}
```

Optimized (greedy, sort by end)

Keep the earliest-finishing interval at every conflict.

```
public int eraseOverlapIntervals(int[][] intervals) {
    if (intervals.length == 0) return 0;
    Arrays.sort(intervals, (a, b) -> Integer.compare(a[1], b[1]));   // sort by END time
    int removals = 0;
    int lastEnd = intervals[0][1];
    for (int i = 1; i < intervals.length; i++) {
        if (intervals[i][0] < lastEnd) {
            removals++;                          // this one overlaps -> drop it
        } else {
            lastEnd = intervals[i][1];            // no overlap -> keep, advance end
        }
    }
    return removals;
}
```

Version	Time	Space
Brute force	O(n · 2^n)	O(n)
Greedy, sort by end	O(n log n)	O(1) extra

11.4 Meeting Rooms II

Problem. Given an array `intervals` of meeting time intervals `[start_i, end_i]`, return the minimum number of conference rooms required so that every meeting has a room for its entire duration and no room is used by two meetings at the same time.

- **Constraints:** `1 <= intervals.length <= 10^4`; `0 <= start_i < end_i <= 10^6`. Meetings may be given in any order and may be duplicated.
- **Clarifying assumptions:** a meeting that ends exactly when another starts (`a.end == b.start`) does **not** conflict — the room can be reused instantly at that boundary; assume every meeting is

non-empty (`start_i < end_i`).

- **Why it's tricky:** this is the interval problem where sorting alone isn't quite enough — you also need to track, at every point in time, *how many* meetings are simultaneously open, not just whether the current one overlaps the previous one. That's what forces a min-heap (or a two-pointer sweep over separately sorted starts/ends) instead of a simple linear scan.

Example.

```
Input:  [[0,30], [5,10], [15,20]]
Output: 2               ([0,30] overlaps both [5,10] and [15,20], but those two don't overl
```

Intuition (diagram). Think of it as a min-heap of "when does each occupied room free up." Sort meetings by start time. For each meeting, if the earliest-freeing room is already free (its end `<=` this meeting's start), reuse it; otherwise open a new room.

```
0    5   10   15   20   25   30
[-----------------------------]   room A
     [----]   room B
               [----]   room B (reused)
```

Approach (step by step)

- **Key idea / invariant:** a min-heap `roomEndTimes` holds the end time of every room that is *currently occupied*, with the soonest-to-free room always at the top (`peek()`). The heap's size at any moment equals the number of rooms in use right then, so the answer is the heap's maximum size ever reached — which, because a meeting is only ever added after a possible removal, ends up being the heap's final size once meetings are processed in start order.
- **Numbered walk-through:**
- Sort meetings by `start` time (you must know which meeting is considered next).
- For each meeting, in order: peek the heap's minimum end time. If it exists and is `<=` this meeting's start, that room has already freed up — pop it (reuse the room).
- Push this meeting's `end` time onto the heap (occupying a room — either the reused one or a brand-new one, since a pop only happens when reuse is possible).
- After processing all meetings, the heap's size is the number of rooms simultaneously needed at the peak — return `roomEndTimes.size()` .
- **Why a min-heap and not just "the last end time":** unlike Merge Intervals, more than one room can be open at once, so you must compare against the *earliest-freeing* open room, not just the most recently opened one — that's exactly what a min-heap gives you in O(log n) per operation.
- **Worked trace** on `[[0,30],[5,10],[15,20]]` (already sorted by start):
- `[0,30]` : heap empty -> push `30` . heap = `{30}`
- `[5,10]` : peek `30` , is `30 <= 5` ? no -> can't reuse -> push `10` . heap = `{10, 30}`
- `[15,20]` : peek `10` , is `10 <= 15` ? yes -> pop `10` (reuse that room) -> push `20` . heap = `{20, 30}`
- final heap size = 2 -> rooms needed = 2

Brute force

For every meeting, count how many other meetings are active at its start time; the max is the answer.

```
public int minMeetingRoomsBrute(int[][] intervals) {
    int maxRooms = 0;
    for (int[] meeting : intervals) {
        int active = 0;
        for (int[] other : intervals) {
            if (other[0] <= meeting[0] && meeting[0] < other[1]) {
                active++;                          // other is in progress at meeting's sta
            }
        }
        maxRooms = Math.max(maxRooms, active);
    }
    return maxRooms;
}
```

Optimized (min-heap of end times)

Sort by start; a heap always holds the end times of currently occupied rooms.

```
public int minMeetingRooms(int[][] intervals) {
    Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]));   // sort by START time
    PriorityQueue<Integer> roomEndTimes = new PriorityQueue<>();     // min-heap of end tim

    for (int[] meeting : intervals) {
        if (!roomEndTimes.isEmpty() && roomEndTimes.peek() <= meeting[0]) {
            roomEndTimes.poll();                   // earliest-freeing room is free -> reu
        }
        roomEndTimes.offer(meeting[1]);            // occupy a room (new or reused) until
    }
    return roomEndTimes.size();                    // rooms still occupied = rooms needed
}
```

Alternative: sort-starts/sort-ends sweep line. Sort start times and end times separately into two arrays. Walk both with two pointers: advance the start pointer and increment a room counter, but whenever the next start is `>=` the earliest unhandled end, free a room (increment the end pointer) first. Track the max concurrent rooms. This avoids the heap and runs in the same O(n log n) but with O(1) extra space beyond the sorted arrays.

Version	Time	Space
Brute force	$O(n^2)$	O(1)
Min-heap of end times	O(n log n)	O(n) for the heap
Sort starts/ends sweep line	O(n log n)	O(n) for the sorted arrays

---

12. Stacks

The core idea. A stack is Last-In-First-Out (LIFO): you can only add or remove from the top. That single restriction turns out to be exactly the right tool for **matching pairs** (parentheses, tags, nested structures) and for **monotonic scans**, where you keep a stack whose elements stay sorted (increasing or decreasing) so you can instantly answer "what's the next bigger/smaller thing?" as you sweep through an array.

Reach for it when: you see **nested or matching pairs** (brackets, tags), you need the **most recent unmatched item**, or the problem asks for the **next greater/smaller element** relative to each position.

12.1 Valid Parentheses

Problem. Given a string `s` containing only the characters `(`, `)`, `{`, `}`, `[`, `]`, return `true` if the brackets are **valid**: every opening bracket has a matching closing bracket of the *same type*, and brackets close in the correct (properly nested) order.

- **Given:** a string `s` of bracket characters only. **Return:** a boolean.
- **Constraints:** `1 <= s.length <= 10^4`; `s` consists only of `()[]{}` characters (no other symbols, letters, or digits mixed in).
- **Clarifying assumptions:** confirm the string contains *only* bracket characters (no need to skip other characters); an empty string counts as valid (though the constraint above rules it out here).
- **Why it's tricky:** it looks like simple counting, but counting opens/closes of each type separately is not enough — `"([)]"` has equal counts of every bracket type yet is invalid because the *order* of closing is wrong. You need to track which opener is "currently active," and that's exactly what a stack's LIFO order gives you for free.

Example.

```
Input:  "{[()]}"
Output: true

Input:  "([)]"
Output: false
```

Intuition (diagram). Push every opening bracket. When a closing bracket arrives, it must match whatever is currently on **top** of the stack — that's the most recent unmatched opener. If it doesn't match, or the stack is empty, the string is invalid.

```
step  char  action                    stack(bottom->top)
----- ----- ------------------------- ------------------
0     {     push                      {
1     [     push                      { [
2     (     push                      { [ (
3     )     pop, match ( / )          { [
4     ]     pop, match [ / ]          {
5     }     pop, match { / }          (empty)
```

input `"{[()]}"` ends with an empty stack -> valid.

```
step  char  action                    stack(bottom->top)
----- ----- ------------------------- ------------------
0     (     push                      (
1     [     push                      ( [
2     )     top is '[', need ')'      ( [
```

input `"([)]"`: top of stack is `[` but the current char needs `)` -> mismatch -> invalid.

Approach (step by step)

Key idea / invariant: the stack always holds the openers that are still "waiting" to be closed, in the exact order they were opened, bottom-to-top = oldest-to-newest. The bracket on **top** is always the most recently opened, unmatched one — the only one a closing bracket is allowed to match.

1. Scan the string left to right, one character at a time.
2. If the character is an opening bracket (`(`, `[`, `{`), push it — it's now the "most recent unmatched opener."
3. If the character is a closing bracket, check the stack:
4. If the stack is empty, there's no opener to match -> invalid, return `false`.
5. Otherwise pop the top opener and check it's the *same type* as this closer. If it isn't, return `false`.
6. After processing every character, the string is valid only if the stack is **empty** — any opener left unpopped never got closed.

Brute force

Repeatedly find and remove any adjacent matching pair `"()"`, `"[]"`, `"{}"` until nothing changes; valid if the string becomes empty.

```
public boolean isValidBrute(String s) {
    String prev = s;
    while (true) {
        String next = prev.replace("()", "").replace("[]", "").replace("{}", "");
        if (next.equals(prev)) break;   // no more pairs collapsed
        prev = next;
    }
    return prev.isEmpty();
}
```

Optimized (stack)

Push openers, pop-and-check on closers.

```
public boolean isValid(String s) {
    Deque<Character> stack = new ArrayDeque<>();   // ArrayDeque: no sync overhead, no lega
    for (char c : s.toCharArray()) {
        if (c == '(' || c == '[' || c == '{') {
            stack.push(c);
        } else {
            if (stack.isEmpty()) return false;             // nothing to match against
            char top = stack.pop();
            if ((c == ')' && top != '(') ||
                (c == ']' && top != '[') ||
                (c == '}' && top != '{')) {
                return false;                               // wrong opener for this closer
            }
        }
    }
    return stack.isEmpty();     // every opener must have been matched
}
```

Version	Time	Space
Brute force (repeated collapse)	$O(n^2)$	O(n)
Stack	O(n)	O(n)

12.2 Min Stack

Problem. Design a stack data structure that supports, all in **O(1)** time per call: `push(val)` (add a value to the top), `pop()` (remove the top value), `top()` (return the top value), and `getMin()` (return the minimum value currently in the stack).

- **Given:** a sequence of operation calls (`push`, `pop`, `top`, `getMin`). **Return:** the results of `top` / `getMin` calls.
- **Constraints:** values fit in a 32-bit signed integer; up to `~10^4` – `10^5` total calls; `pop`, `top`, and `getMin` are never called on an empty stack.
- **Clarifying assumptions:** duplicate values are allowed (e.g. pushing the same minimum value twice); `getMin` must be O(1) — an O(n) rescan defeats the point of the problem.
- **Why it's tricky:** the obvious approach (track a single `min` variable) breaks on `pop` — if you pop the current minimum, what's the new minimum? You'd have to rescan, which is O(n). The trick is to make every stack frame remember its *own* correct minimum so popping never loses information.

Example.

```
push(-2); push(0); push(-3);
getMin() -> -3
pop();
top()    -> 0
getMin() -> -2
```

Intuition (diagram). A plain stack only tells you the top value. To get the minimum in O(1) without rescanning, store, alongside each value, the minimum **seen so far at that depth**. Popping just uncovers the previous (still-correct) running minimum underneath.

```
op          stack, top -> bottom, as (value,minSoFar) pairs
----------- ------------------------------------------------------
push(-2)    (-2,-2)
push(0)     (0,-2)  (-2,-2)
push(-3)    (-3,-3) (0,-2)  (-2,-2)
getMin()    -> reads min of top pair (-3,-3) -> -3
pop()       -> removes (-3,-3); now top is (0,-2)  (-2,-2)
top()       -> reads value of top pair (0,-2) -> 0
getMin()    -> reads min of top pair (0,-2) -> -2
```

Approach (step by step)

Key idea / invariant: every stack entry is a pair `(value, minSoFar)`, where `minSoFar` is the minimum of every value pushed up to and including this one. Because each entry carries its own correct minimum, popping an entry automatically "restores" the correct minimum for what's left underneath — no rescanning needed.

1. `push(val)`: compute `currentMin = min(val, minSoFar of the current top)` (or just `val` if the stack is empty), then push the pair `(val, currentMin)`.
2. `pop()`: pop the top pair as normal — the new top's `minSoFar` is already correct because it was computed when *it* was pushed.
3. `top()`: return the `value` half of the top pair.
4. `getMin()`: return the `minSoFar` half of the top pair — always O(1), no scanning.

Brute force

Plain stack for values; `getMin` scans the whole stack each time.

```
class MinStackBrute {
    private final Deque<Integer> stack = new ArrayDeque<>();

    public void push(int val) { stack.push(val); }
    public void pop() { stack.pop(); }
    public int top() { return stack.peek(); }

    public int getMin() {
        int min = Integer.MAX_VALUE;
        for (int v : stack) min = Math.min(min, v);   // O(n) scan every call
        return min;
    }
}
```

Optimized (pair each value with the running min)

Push `(value, minSoFar)` together; every op stays O(1).

```
class MinStack {
    // ArrayDeque avoids Stack's legacy synchronization cost
    private final Deque<int[]> stack = new ArrayDeque<>();   // {value, minSoFar}

    public void push(int val) {
        int currentMin = stack.isEmpty() ? val : Math.min(val, stack.peek()[1]);
        stack.push(new int[] { val, currentMin });
    }

    public void pop() {
        stack.pop();
    }

    public int top() {
        return stack.peek()[0];
    }

    public int getMin() {
        return stack.peek()[1];
    }
}
```

Version	Time (push/pop/top)	Time (getMin)	Space
Brute force	O(1)	O(n)	O(n)
Paired running min	O(1)	O(1)	O(n)

12.3 Daily Temperatures

Problem. Given an array `temps` of daily temperatures, return an array `answer` where `answer[i]` is the number of days you'd have to wait after day `i` to see a **strictly warmer** temperature. If no future day is warmer, `answer[i] = 0`.

- **Given:** an integer array `temps`. **Return:** an integer array `answer` of the same length.
- **Constraints:** `1 <= temps.length <= 10^5`; `30 <= temps[i] <= 100` (typical LeetCode bounds); temperatures can repeat.
- **Clarifying assumptions:** "warmer" means strictly greater, not greater-or-equal; if two future days tie with the current warmest-so-far, we still want the *nearest* qualifying day, not just any.
- **Why it's tricky:** the naive approach rescans forward from every index, which is $O(n^2)$. The insight is that once you know a *later, warmer* day exists for some earlier day, you never need to reconsider that earlier day again — that "resolve once and discard" behavior is exactly what a monotonic stack gives you in amortized O(n).

Example.

```
Input:  [73, 74, 75, 71, 69, 72, 76, 73]
Output: [ 1,  1,  4,  2,  1,  1,  0,  0]
```

Intuition (diagram). This is the classic **next-greater-element** pattern: keep a stack of indices whose temperatures are **decreasing** from bottom to top (a "monotonic decreasing stack"). When the current

temperature beats the index on top, that top index has just found its answer — pop it, record the distance, and keep popping while the new value still beats what's left. Then push the current index.

Approach (step by step)

Key idea / invariant: the stack holds indices whose temperatures form a strictly **decreasing** sequence from bottom to top — i.e. every index still on the stack is still "waiting" for a warmer day. Each index is **pushed exactly once** (when we first visit it) and **popped exactly once** (the moment a warmer day is found) — that's what makes the whole scan O(n) instead of O(n²): no index is ever re-examined once resolved.

1. Walk the array left to right with index `i`.
2. While the stack is non-empty AND `temps[i]` is greater than the temperature at the index on top of the stack: pop that index (call it `prevIdx`) — day `i` is its answer, so set `answer[prevIdx] = i - prevIdx`. Keep popping while this still holds, since one warm day can resolve several previous (increasingly recent) waiting days at once.
3. Once the stack top is no longer beaten (or the stack is empty), push the current index `i` — it now becomes a new "waiting" day, and the decreasing invariant holds again.
4. After the scan, any indices still on the stack never found a warmer day, so their `answer` entries stay `0` (the array's default).

Worked trace on `[73, 74, 75, 71, 69, 72, 76, 73]` (indices `0..7`):

```
i    temp   action                                stack (bottom->top, idx)
---- ------ ------------------------------------ ------------------------
0    73     push 0                               [0]
1    74     74>73: pop 0, ans[0]=1-0=1; push 1    [1]
2    75     75>74: pop 1, ans[1]=2-1=1; push 2    [2]
3    71     71<75: push 3                         [2,3]
4    69     69<71: push 4                         [2,3,4]
5    72     72>69: pop 4, ans[4]=5-4=1            [2,3]
            72>71: pop 3, ans[3]=5-3=2; push 5     [2,5]
6    76     76>72: pop 5, ans[5]=6-5=1            [2]
            76>75: pop 2, ans[2]=6-2=4; push 6     [6]
7    73     73<76: push 7                         [6,7]
```

End of scan: indices `6` and `7` remain on the stack — they never found a warmer day, so `answer[6] = answer[7] = 0`.

Brute force

For each day, scan forward until a warmer day is found.

```
public int[] dailyTemperaturesBrute(int[] temps) {
    int n = temps.length;
    int[] ans = new int[n];
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (temps[j] > temps[i]) {
                ans[i] = j - i;
                break;
            }
        }
    }
    return ans;
}
```

Optimized (monotonic decreasing stack of indices)

Each index is pushed once and popped at most once -> linear time.

```
public int[] dailyTemperatures(int[] temps) {
    int n = temps.length;
    int[] ans = new int[n];
    Deque<Integer> stack = new ArrayDeque<>();   // holds indices, temps strictly decreasir
    for (int i = 0; i < n; i++) {
        while (!stack.isEmpty() && temps[i] > temps[stack.peek()]) {
            int prevIdx = stack.pop();
            ans[prevIdx] = i - prevIdx;          // found the next warmer day
        }
        stack.push(i);
    }
    return ans;   // indices left on the stack never found a warmer day -> stay 0
}
```

Version	Time	Space
Brute force	$O(n^2)$	O(n)
Monotonic stack	O(n)	O(n)

12.4 Evaluate Reverse Polish Notation

Problem. Given an array of tokens representing an arithmetic expression in **Reverse Polish Notation** (postfix — operators come after their operands), evaluate it and return the integer result. Valid operators are `+ - * /`; each operates on the two operands immediately preceding it. Division between two integers truncates toward zero.

- **Given:** a string array `tokens`, each token either an integer (possibly negative) or one of `+ - * /`. **Return:** an integer result.
- **Constraints:** `1 <= tokens.length <= 10^4`; the expression is always valid RPN and the result fits in a 32-bit signed integer; division by zero does not occur.
- **Clarifying assumptions:** operand order matters for `-` and `/` — the token order is `a b op`, meaning compute `a op b` (not `b op a`); confirm truncating (not flooring) division is expected for negative results.

- **Why it's tricky:** it's easy to swap operand order (`b - a` instead of `a - b`) since a stack pop gives you the *second* operand first. Getting that pop order backwards silently produces wrong answers only for `-` and `/` (since `+`/`*` are commutative), which can hide the bug during casual testing.

Example.

```
Input:  ["2", "1", "+", "3", "*"]
Output: 9              ((2 + 1) * 3 = 9)
```

Intuition (diagram). Postfix is built exactly for a stack: numbers get pushed, and every operator pops its two most recent operands, computes a result, and pushes that result back — ready to be used by a later operator.

Approach (step by step)

Key idea / invariant: the stack always holds the operands that haven't been consumed by an operator yet, most-recent on top. Because RPN guarantees an operator's two operands are always the two most recently pushed values, "pop twice, combine, push result" is always correct — no parsing of precedence or parentheses needed.

1. Scan the tokens left to right.
2. If the token is a number, push it onto the stack (it's an operand, possibly used later).
3. If the token is an operator, pop the top two values — call the first pop `b` (it was pushed most recently) and the second pop `a` (pushed just before `b`). This ordering matters: the expression order was `a b op`, so the correct computation is `a op b`, not `b op a`.
4. Apply the operator to `a` and `b`, and push the single numeric result back — it's now available as an operand for a later operator.
5. After all tokens are processed, exactly one value remains on the stack — that's the final result.

Worked trace on `["2", "1", "+", "3", "*"]`:

```
step  token   action                            stack(bottom->top)
----- ------- --------------------------------- --------------------
1     2       push 2                            [2]
2     1       push 1                            [2,1]
3     +       pop 1, pop 2 -> 2+1=3; push 3      [3]
4     3       push 3                            [3,3]
5     *       pop 3, pop 3 -> 3*3=9; push 9      [9]
```

End of scan: one value remains on the stack -> result = `9`.

Brute force

Repeatedly scan the token list for the first operator, apply it, and splice the result back in — mimics evaluating without an explicit stack structure.

```
public int evalRPNBrute(String[] tokens) {
    List<String> list = new ArrayList<>(Arrays.asList(tokens));
    Set<String> ops = Set.of("+", "-", "*", "/");
    while (list.size() > 1) {
        for (int i = 0; i < list.size(); i++) {
            if (ops.contains(list.get(i))) {
                int a = Integer.parseInt(list.get(i - 2));
                int b = Integer.parseInt(list.get(i - 1));
                int res = apply(a, b, list.get(i));
                list.subList(i - 2, i + 1).clear();   // remove operands + operator
                list.add(i - 2, String.valueOf(res));
                break;                                  // rescan from the start
            }
        }
    }
    return Integer.parseInt(list.get(0));
}

private int apply(int a, int b, String op) {
    switch (op) {
        case "+": return a + b;
        case "-": return a - b;
        case "*": return a * b;
        default:  return a / b;
    }
}
```

Optimized (stack of operands)

One left-to-right pass; each operator consumes exactly the top two values.

```
public int evalRPN(String[] tokens) {
    Deque<Integer> stack = new ArrayDeque<>();   // ArrayDeque: faster, no legacy sync cost
    for (String token : tokens) {
        switch (token) {
            case "+":
                stack.push(stack.pop() + stack.pop());
                break;
            case "*":
                stack.push(stack.pop() * stack.pop());
                break;
            case "-": {
                int b = stack.pop(), a = stack.pop();
                stack.push(a - b);                       // order matters: a - b, not b - a
                break;
            }
            case "/": {
                int b = stack.pop(), a = stack.pop();
                stack.push(a / b);                       // order matters: a / b, not b / a
                break;
            }
            default:
                stack.push(Integer.parseInt(token));      // operand
        }
    }
    return stack.pop();
}
```

Version	Time	Space
Brute force (rescan + splice)	$O(n^2)$	O(n)
Stack of operands	O(n)	O(n)

Closing notes

- Spot the pattern early: matching/nesting -> plain stack; "next greater/smaller relative to position" -> monotonic stack. Say this out loud before coding.
- Narrate your plan before diving in — what goes on the stack, and what triggers a pop.
- State time and space complexity up front, and confirm it after coding (most stack solutions here are O(n) time, O(n) space).
- Walk through the example by hand, tracking the stack's contents at each step, before trusting the code.
- Handle edge cases explicitly: empty input, a single element, unmatched closers with an empty stack, and null/blank strings.
- Prefer `Deque<Integer>` (via `ArrayDeque`) over `java.util.Stack` — `Stack` extends `Vector` and carries needless synchronization; `ArrayDeque` is faster and is the recommended general-purpose stack/queue implementation.
